TECHNISCHE UNIVERSITÄT DRESDEN

**Faculty of Computer Science** Chair of Software Technology

# Multi-robot extension and evaluation of a runtime model-driven ROS 2 framework

Alexander Kassuba

Born on: 19th October 2000 in Freiberg
Matriculation number: 4857722
Matriculation year: 2019

## Master Thesis

to achieve the academic degree

## Master of Science (M.Sc.)

Supervisor
Dr.-Ing. Sebastian Götz

Supervising professor
Prof. Dr. rer. nat. habil. Uwe Aßmann

Submitted on: 17th February 2026

# Abstract

In comparison to single-robot systems, the increased complexity of multi-robot systems (MRSs) poses significant challenges for system development. Established middleware such as ROS 2 abstracts from robot-specific code, but its interfaces remain low-level, leaving the modeling of common high-level MRS concepts, distributed architectures, and adaptive behaviors largely unaddressed. Furthermore, simulation is a particularly important tool for MRS development, yet existing simulators with direct ROS integration exhibit scalability limitations and are not seamlessly integrated into the development process.

This thesis addresses these challenges by extending an existing runtime model-driven ROS 2 framework towards explicit support for high-level modeling concepts of MRSs with integrated simulation. The work focuses on enabling structured modeling of multi-robot organizations and behaviors, adaptive composition of behaviors, and improving the scalability of ROS-integrated simulation. To this end, the framework is extended with abstractions for hierarchical system structures, coordinated behavior execution based on hierarchical finite state machines and partially automatic derivation of simulation scenarios. A centralized controller is employed on top of a ROS 2 library of decentralized and composable behaviors, enabling flexible low-level behavior development with high-level adaptive composition. The resulting hybrid architecture avoids scalability limitations of fully centralized control while still providing global coordination. Using the meta-modeling system JastAdd, developers can adapt the modeling language to their specific use case. In addition, a user interface visualizes modeled system structures and behaviors at runtime, supporting transparency and human supervision. For simulation, the framework is integrated with the ARGoS simulator, which is specifically designed for large robot populations.

Results demonstrate that the extended framework can simulate more than twice the number of robots reported for Gazebo-based approaches. Resource analysis and model-set latency evaluation show that the hybrid architecture avoids typical bottlenecks of centralized MRSs. Overall, the findings indicate that runtime model-driven development and simulation of MRSs can be effectively combined within a ROS 2-based framework, providing a foundation for future work on feature and performance enhancements, formal analysis, and usability evaluation.

# Contents

# List of Figures

# List of Tables

# 1. Introduction

The vision of machines acting alongside or in place of humans has been captivating thinkers, engineers and scientists for millennia. From the ancient automata of Hero of Alexandria [125] to the programmable humanoid sketches of Leonardo da Vinci [122], time and time again ideas were conceived, even though an actual implementation was not fully possible at the time. The first implementations of real-life robotic systems, capable of coordination, real-time processing and possessing at least a limited understanding of themselves and their environments, are a development of only the last few decades, still ongoing and far from finished. The 1950s and 60s marked the industrial dawn of robotics, introducing robots like the Unimate, the first programmable robotic arm, deployed on automotive assembly lines [57]. As production capacities increased, robotics has followed suit and evolved toward multi-robot configurations, supported by advances in sensing, communication, and industrial chip manufacturing. However, realizing coordination and adaptation of larger MRSs poses a significant challenge and has been subject to further research. The field of distributed robotics started gaining traction in the 1980s [3], focusing on design [4] and implementation [92] spatially distributed and connected robotic systems. Early approaches were specific to the concrete robots and thus required systems to be re-implemented from the ground up for different use cases or robot types. Generalizing the development process is a more recent development, which started in the early 2000s, introducing middleware in the form of frameworks such as Orocos [16], YARP[78], OpenRTM-aist[1] and ROS [104] as well as its successor ROS 2 [73].

At the time of writing, Robot Operating System 2 (ROS 2) represents a de facto standard for open-source robotic software development and is supported by a large and active community. It will therefore provide the foundation for the system proposed in this thesis. In recent years, advancements in many branches of robotics and related research domains have been made. This thesis focuses on the domains *runtime modeling* and *multi-robot systems* in an attempt to create a framework prototype that allows for efficient development and simulation of mobile robotic applications. In some works, the terms MRSs and robotic multi-agent systems and are both used to describe a robotic system that employs multiple robots to perform a certain task [31]. In other literature, a robotic multi-agent system specifically

[1]There are multiple publications around OpenRTM-aist, see `https://www.openrtm.org/openrtm/en?language=en` for more information

requires agents to act with some degree of autonomy [28]. As the goal in this thesis is to develop a prototype system that consolidates aspects of both kinds of systems, these terms are used interchangeably unless specifically stated otherwise.

In this chapter, this thesis is motivated by describing current challenges that robotic applications face. From these problems, general objectives are derived in Section 1.2. Following this, an overview over the employed solutions in the prototype is given in Section 1.3. Problems (P) and goals (G) are named and labeled clearly (e.g. $\mathbf{P_1}$, $\mathbf{G_1}$) throughout this chapter to make their connections more explicit and easier to follow. To evaluate whether the goals were achieved and to what degree, an evaluation is conducted in later chapters, therefore this chapter concludes with a summary of the evaluation approach.

The prototype application presented in this thesis will build on "RuMROS" (**Ru**ntime **M**odel driven **ROS**) [117], which supports runtime modeling of robotic applications on top of ROS 2 to a certain extent. It comes with an exemplary runtime model that provides high-level actions to control multiple robots and a drone, by addressing them via ROS 2 topics. It also includes a GUI for human control and monitoring of the system. The prototype resulting from the extensions made in this thesis is henceforth referred to as RuMROS*.

## 1.1. Challenges of current robotic applications

Frameworks like ROS 2 have simplified and streamlined the development process of robotic applications by providing middleware to reduce robot dependencies. They also provide assurance to developers, that their tools and applications will be available to a wide variety of robots with only minimal changes. In the following sections, remaining problems, which ROS 2 does not currently solve, as well as general challenges for robotic systems and specifically MRSs are described. The framework this thesis builds on, RuMROS, also faces some problems, especially related to supporting MRSs, which are covered separately.

### 1.1.1. Adapting at runtime

An important property of many robotic systems is adaptivity. When deployed in agricultural contexts or search and rescue scenarios for instance, robots must be able to handle sudden events as well as gradual changes in the environment. This applies to other areas as well, but is most apparent in outdoor applications. In order to work in their respective environment, robots have to *adapt* their behavior to the situation. Keeping to the agricultural example, it may be desirable to have field tiller robots automatically adjust their work schedule according to the weather, to avoid getting stuck in mud. In other cases, manual control by human operators may also be desirable, this is discussed in Section 1.1.2.

In order to make a system robust to uncertainty at compile time, such as introduced by environmental conditions, it has to be programmed with *self-awareness* in mind. It should be able to reason about its own state and the environmental state at runtime and adapt accordingly. Self-Aware System (SAS) have been subject to research in many areas in recent years, but especially in autonomous robotics, self-awareness is a beneficial property [63]. Implementing self-adaptive behavior however is a challenging task, as it often includes many internal and external variables and has to make decisions based on system goals, which may

not be fully clear at runtime. Furthermore, it is application-specific, and for decision making, would benefit from having a richer semantic base than the relatively low-level interfaces provided by ROS 2. This is related to a more general problem present in robotics. The control and sensing interfaces for robots may be abstracted by a middleware, but are kept as low-level as possible to maximize control over the hardware while abstracting from the concrete robot hardware. In ROS 2 for instance, many robot types come with a differential drive, which can be steered with two velocity components: forward and rotational. While this is a perfectly valid approach, it means that for higher-level functionality, like driving to a location for instance, the gap between it and the lower-level interfaces has to be bridged. This required bridging of the high-level problem space (desired capabilities) to the low-level solution space (robot sensors and actuators) is a central issue ROS 2 does not fully address ($\mathbf{P_1}$).

### 1.1.2. Human monitoring and control

As robotic systems evolve, so does their level of autonomy. Failure detection and mitigation for instance allows robots to sort out defective parts on a manufacturing line [108] or adapt to unforeseen problems, requiring less human intervention. This removes additional work humans would have to put in and therefore reduces costs. However, some form of human monitoring and control is often still desired, especially in applications that are not fully autonomous, for instance in the space [113, 114] and industrial sectors [15]. To save costs, this has to be done in an efficient manner, especially in complex applications with multiple robots, as the task load for humans performing inspection and/or control is increased [118] ($\mathbf{P_2}$).

ROS 2 provides topics and services to send and receive data, as well as trigger arbitrary functionality, which is helpful for managing the system with external applications, but the terminal interface itself is rather cumbersome to use for humans, especially for large systems. For this, ROS 2 offers non-interactive visual inspection tools like rqt_graph[2] and RViz[3] on a higher level. RViz especially also allows for customization of views and even offers basic control functionality, e.g. for manually setting the position of a robot arm. Due to operating on topic level, higher level concepts, such as weather derived from rain, temperature and air pressure sensors have to be computed and made available via topics, which may also require custom message types, increasing development effort. Runtime modeling deals with the representation of such higher-level concepts while loosely coupling the lower-level system, but no general integration for the visualization of runtime models with ROS 2 currently exists ($\mathbf{P_3}$). Solutions which support inspection by humans at runtime to some degree have been developed, but they focus on modeling robot behavior and not the general internal and external system state [52, 128].

[2]For more information on ROS 2's rqt_graph, see `https://wiki.ros.org/rqt_graph`

[3]For more information on ROS 2's RViz `https://docs.ros.org/en/rolling/Tutorials/Intermediate/RViz/RViz-User-Guide/RViz-User-Guide.html`

### 1.1.3. Difficulty of scaling robotic applications to multi-robot systems

Supporting the development of general MRSs is difficult, because there are many different axes of classification [31] and thus differences in key aspects like structural organization and communication. Taxonomies for MRSs are discussed in more detail in Chapter 2. Before touching on further problems of MRS, a short motivation as to why they are important in the first place is given.

Multi-Agent System (MAS) provide a few key advantages over single-robot systems, especially in application domains that require the system to be scalable. Criteria for when an application can benefit from the advantages of a MAS have been established [94], and for introductory purposes, an example illustrating these benefits is given in the following paragraph.

Lawn-mowing robots are usually advertised with a maximum area they can realistically cover. To cover a larger area than any existing individual robot can handle (i.e. to upscale the system) there are two options: increase the coverage of the existing robot or add additional robots. The latter supports easier adaptation — if the area is extended or shrunk, robots can be added or removed. This allows for theoretically unlimited extension, while a monolithic system has technological and spatial limits that have to be pushed to extend its capabilities. MAS that can cover the same space as a monolithic single-robot system may even be cheaper in some instances, as the cost of development of individual robots and the system itself are one-time, upfront costs, scaling included in the design [94]. Having multiple robots also increases robustness of the system, as a single agent failing still allows the other agents to continue their work. Finally, multi-agent systems may have some application-specific benefits, like multiple small robots still being able to pass narrow passages on a lawn, which a larger, single-robot system cannot traverse. This also extends to other application domains. Similar benefits arise in consumer delivery warehouses where managing goods on shelves is quicker with multiple robots and increases accountability by having a single robot fulfill an order [126]. As such, multi-agent systems have been a topic of particular interest in research.

When it comes to the development of robotic applications, simulation is an essential tool to test and debug the behavior of a system in specified scenarios. Most notable for this work is Gazebo,[4] as it is designed to support ROS 2 out of the box and well-supported. For multi-robot simulations however, it does not scale well with larger numbers of robots, especially if they have to process complex sensor data [80, 88] ($\mathbf{P_4}$).

Due to this limitation, Gazebo has not been the tool of choice for multi-robot simulations, but in the past two and a half decades, a multitude of simulators have been built to be better suited for this purpose [17]. Unfortunately, many of them are no longer actively maintained and therefore only the relatively recent ARGoS 3 [99] and CoppeliaSim[5] (previously V-REP [109]) provide viable alternatives. Both can be extended and integrated with ROS 2 via plugins, though a user-friendly integration requires significant effort, this will be discussed in Section 1.3 ($\mathbf{P_5}$).

Finally, the architecture of the MRS is an important concern. Some systems are organized in a decentralized manner, distributing behavioral components across robots, instead of

[4] Gazebo is a simulator developed specifically for ROS 2. See `https://gazebosim.org/home` for more information

[5] CoppeliaSim, a robotics simulator. For more information see `https://www.coppeliarobotics.com/`

employing central control logic. In such systems, self-awareness of the system as a whole (and thus the basis for adaptivity by extension) can emerge from self-awareness of every individual robot [81], requiring only local models. In addition to their environment, every robot also has to be aware of the presence and actions of other robots that operate alongside it and adjust its behavior accordingly. In centralized systems on the other hand, a single model of the system as a whole may be desired, making it difficult to provide a general platform for modeling both types of systems ($\mathbf{P_6}$). In addition to this, some systems may also benefit from certain organizational structures like a distinct hierarchy [25], further increasing the difficulty of modeling general MRS.

### 1.1.4. Problems arising from RuMROS

To extend RuMROS to a viable multi-robot application with runtime modeling capabilities, a few issues that arise from the design of the existing system have to be addressed. Firstly, in terms of general usability, RuMROS still has some problems related to the ease of development. Iteration of prototypes can be a rather cumbersome process, as adjusting the simulation scenario requires static configuration files for communication with ROS nodes to be updated as well ($\mathbf{P_7}$). The Gazebo simulation specification is also separate from the runtime model, which is good for separation of concerns but this in turn requires the scenario specified in the runtime model to be manually matched to what is specified in the simulation ($\mathbf{P_8}$). At runtime, the above issues manifest in a hindrance to adaptation, as it is not possible to dynamically reconfigure robots ($\mathbf{P_9}$).

RuMROS also does not currently support organizational structures for modeling MAS ($\mathbf{P_{10}}$). Instead, it directly processes every robot's sensors and actuators, which raises performance concerns due to the high volume of ROS messages per robot ($\mathbf{P_{11}}$). It also introduces a single point of failure, and fails to model the structure of decentralized MRS with more individual control on each robot.

## 1.2. Objectives

The main goal of this thesis is to conceive and evaluate a prototype framework that allows developers to efficiently model high-level application concepts to create robotic applications. The system should specifically support multi-robot applications by modeling their underlying concepts, and enable their simulation in an efficient manner. In this section, central objectives, which address the previously explained problems are formulated.

### 1.2.1. Runtime modeling facilitates adaptivity

While ROS 2 provides a middleware to abstract from robot hardware, it does not provide a modeling layer for higher-level application concepts ($P_1$), as the abstraction is only concerned with hiding concrete sensors and actuators behind interfaces according to their type. In general software engineering, high-level models are created in an iterative process of specification and refinement. However, code then has to be manually written with these models on different levels of abstraction as a basis. Model-Driven Engineering (MDE) [60],

based on the Object Management Group (OMG)'s Model-Driven Architecture (MDA) [116], standardizes models at different levels of abstraction as well as as providing means for code generation. ROS unfortunately does not provide support for these concepts, leaving both modeling and implementation of abstractions to programmers. Facilitating specification of abstractions from ROS 2 while retaining the flexibility of working on a its lower level is a central aspect to the goals of this thesis. As previous works have already brought the idea of MDE to robotic applications [22], it has to be ensured that the resulting system is novel and has a unique set of features. In this regard, the main differentiating aspect of the framework presented in this thesis is $\mathbf{G_1}$: allowing developers to model the structure of the system separately from semantics. This provides additional separation of concerns and more flexibility in the development process, as the design of the system's architecture can be decoupled from its implementation.

Statically generating code from models is a good first step to ease the development process, but the problem of designing adaptive and extensible robotic applications ($P_9$) requires a solution that works at runtime. An approach that tackles both of these issues is runtime modeling, also known as *models@run.time* [7]. Robot controllers can be described using a control loop, also known as MAPE-K loop, which implements the logic for controlling an individual robot and builds the foundation for models@run.time. It takes sensor **M**easurements, **A**nalyzes them, **P**lans actions and finally **E**xecutes them by steering actuators. From measurements, it builds **K**nowledge, which is used and updated during the analysis and planning phases respectively and thus forms the basis for deciding on actions taken in the execution phase. Therefore, knowledge, implemented usually as arbitrary data structures in the language of the system, is a central piece in this type of controller. It is tightly integrated to the control algorithm itself, which in turn implements concrete behavior.

Runtime modeling abstracts from this by introducing models in place of knowledge. This is a natural step, as knowledge data structures can already perform abstraction by representing the data on a higher level, keeping track of of traveled distance and relative position instead of just storing velocity data for instance. Representing knowledge with models retains this capability, while also abstracting structurally, decoupling the data from the control loop and allowing developers to directly specify and represent higher-level concepts in a metamodel. Semantically, models can represent the system (physical parameters, current state), context (environment) and requirements and goals separately, enabling developers to independently work on different aspects of the model as a whole. This is also central to adaptivity with respect to the environment and other robots in a MRS, as modeling them can be done separately, making the otherwise complex adaptation structure more accessible. Since the runtime modeling paradigm requires higher level knowledge to be part of the runtime model, logic to compute it is also specified independently of the underlying implementation of the control loop by extension. This increases separation of concerns and robustness to changes, further facilitating scalability and manageability. In MRSs, this is especially beneficial, since there is another layer on top of the model of each individual robot — the model of the MRS as a whole — which ideally should be represented with as little entanglement as possible with lower-level code in order to be manageable. Finally, models@run.time also provide a central interface for introspection, which is separate from its other parts, creating the foundation for issues related to this, like $P_2$.

As it solves several key issues, runtime modeling should be a central part of the proto-

type application, extended to MRSs ($\mathbf{G_2}$). Although it can adapt its behavior at runtime, RuMROS has the problem of not being extensible or adaptive with respect to the actual robot setup ($P_9$), despite using runtime modeling techniques. $\mathbf{G_3}$: The prototype system presented in this thesis should have this capability, allowing for dynamic reconfiguration of organizational units at runtime.

Since human introspection is an important aspect to many systems, the prototype application should also enable it, ideally without any extra effort for developers, tackling problem $P_2$. RuMROS already supports both control and monitoring for robots via a web interface, by invoking pre-defined behaviors. It is also fully generated from the existing runtime model, eliminating extra effort for the creation of the visualization ($\mathbf{G_4}$). Executable actions as well as what is part of the visualization can be directly specified in designated classes of the runtime model. Since the multi-agent extension performed in this thesis will extend the runtime model with a model of the MRS's behavior as a whole, a visualization for it should be added ($\mathbf{G_5}$).

### 1.2.2. Hybrid system structure and simulation

A central goal parallel to the architectural and functional goals is to retain existing functionality for compatibility as much as possible ($\mathbf{G_6}$). It should still be possible to simulate robotic systems with few robots with Gazebo and make use of direct robot control from the runtime model, which the initial RuMROS offers, without changing the runtime model.

For MRSs, centralized control is not always desired ($P_6$). $\mathbf{G_7}$: It should be supported but there should also be a different way of controlling the robots with more decentralized, individual capabilities to better model systems with swarm behavior. This is a compromise for swarm systems, allowing developers to model them, despite the architecture of the system having a central runtime model by design, resulting in what will be referred to as a *hybrid system* from here on. To mitigate the problem of having a single point of failure ($P_{11}$), it should be possible for swarm developers to simply omit control actions and start local behavior on robots, while still being able to make use of the runtime inspection capabilities of the runtime model.

In order to support MRSs, their organizational concepts have to be supported in the runtime model, as well as the ROS 2 backend for developers to use. To offer individual and decentralized behavior, ROS nodes should have a set of capabilities, which can be switched at runtime. To address the performance concerns of having the runtime model steer every robot, the runtime model should be able to invoke the provided functionality, leaving computational effort to the ROS nodes. Supporting this also requires structural changes to the runtime model, as in addition to individually commanding robots, it should be possible to also multicast command messages, addressing multiple robots at once. This means that the runtime model has to support organizational units, such as hierarchies, as well as their nested composition, in order to address $P_{10}$. This way, MRS can be modeled and controlled efficiently, while offering different degrees of autonomy/control.

In addition to modeling, simulation is also an important concern for development and testing. To mitigate the problem Gazebo has with larger numbers of robots ($P_4$), another simulator should be used for multi-agent systems ($\mathbf{G_8}$). The previously described issue of

many available simulators no longer being maintained narrows down candidates significantly. On top of this, there are four main aspects that are desirable: performance, extensibility, openness and feature support. Good performance, even for larger systems is a core goal, as otherwise there is no reason to switch from Gazebo, which is extensible via plugins, open-source and has sufficient feature support. The other criteria are not strictly necessary, but may be detrimental to an otherwise well-suited choice. For instance, if a simulator fulfills nearly all criteria, but only supports a very limited set of robots and is not extensible ($P_5$), this combination would prevent it from being a suitable option. These criteria are further evaluated in Section 1.3, and a choice for the prototype system is made based on it.

To address the problem of Gazebo's separate, static scenario specification ($P_8$), $\mathbf{G_9}$: proper integration of the simulator with RuMROS is also important. It should be possible to at least partly derive the simulation scenario from the runtime model in order to ease iteration in development, as the model has knowledge of how many robots are used, what types they are, where they are positioned, etc.

## 1.3. Solution

This thesis builds on an existing system, RuMROS, as previously described, which already achieves some of the goals for few-robot systems,[6] but several limitations become apparent regarding scaling to MRSs, as it was not initially designed with such systems in mind. In this section, an overview of the solutions to the problems described in Section 1.1 is provided, with respect to the objectives specified in Section 1.2. Following this, a set of key research questions central to this thesis is formulated.

### 1.3.1. Robust multi-robot modeling architecture

RuMROS uses a novel approach to runtime modeling, which is described in detail in Chapter 4. It makes use of JastAdd [50], allowing developers to model their robotic applications as an Abstract Syntax Tree (AST). The specification of its structure and semantics are entirely separated, which already achieves $G_1$ — the goal of robust static modeling capabilities. This makes achieving desirable properties in software engineering in general, like code reuse and separation of concerns easier for developers, and it is further amplified by the aspect-oriented approach JastAdd has. It allows for easy addition of semantically characterized aspects and even rewriting of existing functionality in new ones. This improves manageability and extensibility, which are particularly important for scalability, a key aspect of MRSs, to which this existing system is extended in this thesis.

At runtime, the AST used in RuMROS can receive robot sensor data and compute velocity commands to control robots, which are then sent to the robots. It achieves runtime-modeling capabilities ($G_2$) for few-robot systems, but has to be extended for MRSs in order to support its capabilities. For this purpose, the prototype supports the concept of *groups* of robots, which can be addressed as one logical unit via commands. Additionally, these groups can be

[6] Few-robot systems in the context of this thesis describes robotic systems, which support multiple robots but don't scale well and don't support concepts of larger MRSs like organizing robots into structures. Therefore, they are not suitable for development of general multi-robot applications that have to scale.

nested, allowing for hierarchical organization of robots, making the system more manageable and scalable, not only for development but also at runtime. The control commands serve two distinct purposes: enabling control over organizational units, and addressing performance concerns. The former works towards the goal of extending control over robots, which the existing runtime model already provides for individual robots, to the organizational concepts of MRSs. By adding parameters with default values to commands, the level of control over each group is adjustable by both developers and human monitors, who supervise the application state via the web Interface. The performance concerns are addressed by not having to receive sensor data and compute movement commands for a large number of robots within the runtime model. Instead, functionality is shifted to the ROS layer of RuMROS, which is then invoked from the runtime model. This way, processing is more localized and commands are invoked less frequently.

To implement this, instead of making use of the generic ROS nodes offered by Gazebo like RuMROS does, robots have to implement a set of basic movement patterns and swarm behaviors. Therefore, a ROS node is developed, that can be ran on all robots. Only having a single ROS node that offers behavior is a deliberate choice, with the goal of avoiding additional development effort and bad code reuse in mind. If the node was specific to a robot type, extending the application to support novel kinds of robots would involve some form of re-implementation of behavior for this type, based on differences in sensors, actuators and physical parameters. Instead, the node abstracts from the robot type by introducing common sensor classes, which adapt slight differences of concrete implementations to a broader sensor data type. Behavior classes can then be specified on top of this in a modular way, only requiring a certain type of sensor, which is adapted to concrete robots on lower layers. In summary, this localizes control loops but makes them respond to commands from the runtime model. Thus, a certain level of central control is still possible, but much more loosely coupled than in the initial framework. This addresses the goal of achieving a hybrid system structure ($G_7$) by adding decentralized elements steered by a central controller. A detailed description of the developed ROS 2 node is given in Chapter 4.

With this, static and runtime modeling capabilities are covered, but the goal of adaptivity ($G_3$) remains to be solved, as the initial runtime model is fully static. To support adaptive behavior at runtime, the group structure is partially dynamic, as it allows robots to change their group association at runtime. This reconfiguration at runtime however is only a capability that adaptive systems should have, and does not provide developers with the means to model adaptive behavior in a structurally well-defined manner. For this, a *behavior model* is introduced, which enables programmers to specify the system's global behavior at runtime as a model based on a Hierarchical Finite State Machine (HFSM). On a high level, it consists of nodes representing behavior calls and transitions that enable the behaviors under specific conditions. These conditions include time and arbitrary conditional logic, allowing for a change in behavior, and thus adaptation, based on internal, external and computed parameters available in the runtime model.

### 1.3.2. Multi-robot simulation and backwards compatibility

To enable efficient simulation of MRSs ($G_8$), ARGoS 3 [99] will be used as a simulator for many-robot simulations. It was designed to support large-scale swarms specifically, its au-

thors stating that simulation of up to 10000 robots at a speed 40% faster than real-time is possible [99]. Therefore it offers better performance than the other promising candidate presented in Section 1.1, CoppeliaSim. As a tradeoff, CoppeliaSim comes with more runtime tools, while ARGoS only supports basic start, pause, reset and fast-forward functionality. In terms of static functionality, ARGoS supports all the features used by the initial Gazebo example scenario that RuMROS comes with (obstacles, areas, textures, text and more). CoppeliaSim supports these, as well as higher level features like motion planning and forward/inverse kinematics, but these features are not required, as ROS 2 already provides libraries for them.

As for extensibility, CoppeliaSim comes with a ROS plugin out of the box while ARGoS offers more flexibility with respect to performance, as a custom plugin can leave out aspects not used by the application and increase simulation performance. This leads to an overall increased integration effort for ARGoS, compared to CoppeliaSim. The latter is not fully free and open source in contrast to ARGoS, which also leads to ARGoS offering better extensibility. It can support any robot type via custom plugins and was designed with user extension in mind, offering an abstraction layer for sensors and actuators to more easily build plugins for new robots. It also allows for easy specification of simulations and even simulation-specific plugins to execute custom code, e.g. to draw areas or provide logic for pickup of items. This, in combination with better performance, lead to it being the simulator of choice for this project.

Integrating ARGoS with RuMROS in order to make iteration during development easier ($G_9$) requires additional effort and careful design of the runtime model. ARGoS natively loads a simulation scenario from an XML configuration file, which developers have to manually alter every time the scenario changes. For the prototype application, an automatic configuration generator will be employed, which derives the static specification of a concrete scenario from the runtime model. It includes a template loader and an extensible library of predefined environments (within template configuration files) that can be used as a basis in order to avoid cluttering the runtime model with simulation code. In case developers prefer to use configuration files directly, they can also fully specify their scenario in a configuration file this way.

Finally, to achieve the orthogonal goal of backward compatibility $G_6$, the runtime model for MRS is provided as an extension to the exemplary runtime model RuMROS comes with, keeping changes to a minimum. This is simplified by JastAdd's aspects and rewrite language features, allowing new semantically related components to be added easily. The structural specification of the runtime model used for both the MRS extension and few-robot simulations with Gazebo. It is extended to include concepts central to multi-agent systems, like organizational units, as described in previous sections. As the concrete specification of the runtime model's structure is too complex for the scope of this introduction, it is explained in more detail in Chapter 4. RuMROS* allows developers to choose the type of system they are modeling when launching RuMROS. For few-robot systems, Gazebo can be used for simulation and for multi-agent systems, ARGoS is recommended.

### 1.3.3. Human introspection

As mentioned in Section 1.2, RuMROS already includes an interface that achieves the goal of enabling human monitoring and intervention ($G_4$) for few-robot systems. In RuMROS* it is further extended to include MRS structures, by showing robot-group relations for instance. In addition to this, a graph-based overview of the behavior model has been added, which also enables supervisors to start and stop the model, addressing $G_5$.

## 1.4. Evaluation

The evaluation conducted in this thesis focuses on assessing the feasibility of simulating large MRS using the developed prototype framework. As simulation constitutes a central aspect of the implementation, all evaluations are performed in an environment simulated with ARGoS. The primary objective of the evaluation is to analyze system performance and scalability on typical consumer hardware. To this end, a representative warehouse scenario is used, in which multiple groups of mobile robots execute coordinated behaviors defined in the behavior model. The number of robots is increased incrementally across test runs in order to identify scalability limits and observe system behavior under growing computational load. Performance measurements are obtained on both a mobile and a desktop machine to capture differences between resource-constrained and more powerful systems. During each test run, the simulation is executed for a fixed duration, while component-level metrics such as CPU utilization, memory consumption, and loop timings for behavior model and ARGoS are recorded per component. To complement system-level measurements, middleware-level behavior is examined using tracing tools provided by the ROS 2 ecosystem. This allows inspection of message frequencies, callback durations, and communication delays. Following a presentation of the results, an analysis is conducted to identify limiting factors as well as components that constitute potential performance bottlenecks. From this, suggestions for future optimizations are derived.

# 2. Background

In this chapter, the underlying framework and basic concepts used in this thesis are explained. This includes the theoretical foundations required for runtime modeling, Self-Adaptive Systems (SASs) and behavior modeling, as well as an overview of ROS 2, JastAdd [50] and RuMROS [117]. Regarding MRS, an overview of existing system categories and taxonomies is given, with additional focus on swarm systems, as they employ many of the behavioral patterns implemented in this work.

## 2.1. Theoretical foundations

In the early 2000s, the MDA was proposed by the OMG as an approach to software development. In previous years, standards like the Common Object Request Broker Architecture (CORBA) [9] were introduced to address the challenges of system interoperability, platform independence, and architectural separation of concerns. However, a core issue remained unaddressed — Middleware proliferation. In order to adapt arbitrary systems to CORBA or other standards for distributed systems, enterprises often had to resort to using multiple different middlewares, prompting the development of MDA [116]. It proposes to solve this problem by introducing three levels of modeling — the Computation Independent Model (CIM) layer to capture the system's environment and requirements without structural details, Platform-Independent Model (PIM) layer to represent the system functionality independently of any technological platform and finally the Platform-Specific Model (PSM) layer to include platform-specific details of the implementation. From this proposal, MDE was derived as a method to integrate models into all stages of the development process. It treats models as primary artifacts and integrates them tightly in the software engineering process, such that artifacts like code snippets can be directly traced back to models. For the frameworks used is thesis, Model-Driven Development (MDD) [77] is also relevant. It focuses on transformations between the modeling layers with the final goal to enable code generation from lower layer models like PSMs.

In order to create models, Domain-Specific Languages (DSLs) are often used. CORBA has the Interface Definition Language (IDL) for the specification of application interfaces for instance, enabling code for ports in different implementation languages to be generated via IDL compilers. This reduces coupling to concrete implementation platforms by abstracting

from underlying languages, but does not fully eliminate the problem of middleware proliferation, because modeling every possible scenario would require IDL compiler implementations for every language. One approach to mitigate this would be to allow for growth by providing language extensions that facilitate adaptation to other languages. While this would overall decrease the number of languages and consequently middlewares developers have to use for any given system, they would still have to wait for an extension for their concrete scenario to be made, using multiple middleware frameworks in the meantime.

As this significantly increases development effort, an alternative approach is to allow developers to modify the language itself to suit the needs of a concrete application. This need for modeling languages was recognized by the OMG in the 2000s. The Meta-Object Facility (MOF) specification[1] recognizes three modeling levels (M1, M2, M3) above the level M0 that represents the real world. Level M1 encompasses concrete models, created in their respective modeling language, which resides on level M2. On the final level, M3, languages themselves are modeled. This is the final level, as languages capable of modeling languages contain constructs to model themselves, thus eliminating the need for another level. As each layer abstracts from the layer below, the greek word "meta", which means (self-)describing is often used. M1 contains models, M2 meta-models (models describing models) and M3 meta meta-models (models describing meta-models, or languages). This is relevant to any application that works with models for which existing languages are not expressive enough, including the system proposed in this thesis.

### 2.1.1. Runtime modeling and Models@run.time

A central concept of MDE is to employ models at every step of the software development process. Runtime modeling extends this idea by aiming to make use of models during execution of code as well. With model transformations and a proper toolchain, code generation from models is possible according to MDE, but the resulting code no longer has a clearly separated and abstract view on modeled concepts at runtime. In contrast to this, runtime models are maintained separately from and synchronized with the executing system, thereby enabling dynamic monitoring, adaptation, and decision-making capabilities based on a high-level representation at runtime.

The practice of integrating models as active, first-class elements within a running system is also known as models@run.time [7]. In this paradigm, models serve as causal representations of the system and its environment. This means that changes to the real-world system are reflected in the model, and conversely, modifications to the model can influence system behavior. According to Blair et al. [10], models@run.time are characterized by several key features:

- *Causal Connection*: The runtime model is dynamically linked to the system, maintaining an up-to-date representation of its state and enabling behavioral adaptation on model level.
- *Abstraction*: Models abstract certain aspects of the system (e.g., configuration, performance, context) to support higher-level analysis and control.

[1] The MOF specification from 2006 can be found at `https://www.omg.org/spec/MOF/2.0/PDF`.

- *Purpose at runtime*: These models are directly manipulated during execution, typically to support adaptation, optimization, or fault recovery.

This means, that in contrast to reflection, which also enables systems to reason about their own state and adjust accordingly, models@run.time are not intrinsically connected to the computation model. They tend to be higher-level and are not based on the solution space, but the problem space. Causal connections represent the way of bridging solution space to problem space. While this coupling is not strict by any means, as runtime models are supposed to be an abstraction of the computation model, it is desirable that runtime models are intrinsically linked to their MDE counterparts. This is to facilitate integration in distributed systems by keeping models consistent even at runtime by design, thus helping to mitigate issues like middleware proliferation. In summary, Blair et al. define a model@run.time as follows:

> A model@run.time is a causally connected self-representation of the associated system that emphasizes the structure, behavior, or goals of the system from a problem space perspective [10].

In general systems, the way runtime models are implemented is very diffuse, as there are a wide variety of purposes and application domains [6, 121]. Existing attempts to categorize them are presented in Chapter 3 in order to identify gaps in research.

### 2.1.2. Self-aware and self-adaptive systems

Self-adaptiveness is a key property enabled by runtime modeling. In Section 1.2.1, a widely used architecture to enable it in robotic systems — the MAPE-K loop — was already described. Originally introduced in the context of autonomic computing [61], it can be enhanced with runtime models to enable a range of self-* capabilities, including self-awareness and self-adaptivity [7]. In this section, concrete definitions for systems with these properties are given. Specifically, it is also explained how they can be applied in the context of robotic systems.

#### Self-aware systems

Self-awareness of computing systems has been a topic of interest in multiple domains, including robotics, but in literature, different definitions are used [19, 63, 67]. This lack of a clear definition prompted engineers and researchers from multiple different fields to come up with a consensus definition [63] for general self-aware computing systems. It states the following:

> Self-aware computing systems are computing systems that learn models capturing knowledge about themselves and their environment (such as their structure, design, state, possible actions, and runtime behavior) on an ongoing basis. In addition to this, they reason using the models (e.g., predict, analyze, consider, and plan) enabling them to act based on their knowledge and reasoning (e.g., explore, explain, report, suggest, self-adapt, or impact their environment) in accordance with higher-level goals, which may also be subject to change.

As this definition explicitly states a need to model not only a system's structure and design, but also runtime behavior, it implicitly requires runtime modeling for a system to be self-aware. Within self-aware systems themselves, there can be a varying degree of self-awareness. Systems "themselves" and "the environment", as stated in the definition can include many different factors. Lewis et al. [67] suggest a classification of these different degrees of computational self-awareness based on Neisser's levels of human self-awareness [85]. They differentiate between 5 levels:

1. *Stimulus awareness* - The system is aware external stimuli and can react to them
2. *Interaction awareness* - The system can learn that both its own actions and external stimuli are part of interactions with the environment and other systems
3. *Time awareness* - The system can amass knowledge from previous events and likely future events
4. *Goal awareness* - The system can obtain knowledge of its own goals, preferences and constraints
5. *Meta-self awareness* - The system can obtain knowledge about its own awareness levels and how they are applied

In order to implement a robotic system that supports the first three levels, techniques like reflection and introspection may be sufficient, along with data structures for storing sensor data over time. However, goals are part of the problem space, requiring a higher level of modeling than the system level. Therefore, only an approach that is aware of this level, like Models@run.time is appropriate to reach the two final levels.

While the definition given above includes context-awareness in self-awareness, this is not always the case in other literature. Lewis et al. also differentiate between *private* and *public* awareness [67], as is already hinted by the consensus definition above. Private awareness (also called self-awareness, but in a more narrow sense than what the consensus definition above states [7, 97]) is awareness related to a system's own state. Public awareness (also called context-awareness [7, 97]) on the other hand is concerned with the environment. In robotic systems, in order to obtain private awareness, robots require sensors and models capable of monitoring and reasoning about their own condition and state, for instance, odometry for movement detection or grip force sensors for properly handling gripping of different objects [19]. Similarly, external sensors are required to monitor the environmental state and additionally in MRSs, the state of other robots.

### Self-adaptive systems

Self-awareness (in the broad sense, including context-awareness) has been identified as a prerequisite for self-adaptiveness [97]. This is also evident in the above definition of self-awareness — A self-adaptive system is a special self-aware system. Not only does it monitor and reason about the system's context (public awareness) and itself (private awareness), but it also incorporates specific adaptation objectives or goals into the reasoning process and ultimately acts based on the result [7, 97]. Again, this definition is broad and even differ throughout literature. It could be argued that almost any computing system adapts to changes of its own state and external inputs, according to logic with simple `if-then-else`

statements for instance. Petrovska suggests to to narrow down the definition, by stating that the system has to be explicitly designed to support adaptivity. For this purpose, the following have to be specified:

- The concrete system function that is to be adapted,
- the conditions (external parameters and internal system states) for adaptation
- the goals the adaptation mechanism has.

The logic that performs said adaptation should then be built in accordance with these aspects and decoupled from the logic that implements the function of the system that is to be adapted. For robotic systems, self-adaptation a central, innate concept, due to the following reasons:

- Robots have certain capabilities, limited by their hardware and thus may offer a distinct set of functions that can be explicit targets for adaptation,
- robots are Cyber-Physical System (CPS) that interact with their environment and potentially other robots, and come with sensors that enable them to do so,
- many robots can sense their own state (via supporting sensors and inference, e.g. odometry for moving robots, grasp strength sensors for robotic arms, etc.),
- Existing frameworks, like ROS 2, enable separation of sensing, reasoning and actuation due to the abstractions they perform as middleware. Specifically, ROS 2 allows to put adaptation logic in a separate node for an explicit, decoupled implementation from system functionality.

In order to implement self-ware and self-adaptive behavior, several suggestions regarding system architecture and design patterns have been made [63, 68]. In general self-adaptive systems, there are a wide variety of implementation approaches and techniques [64]. As the focus of this thesis lies on a model-based implementation approach, they are not discussed in detail. However, some systems discussed in Chapter 3 may be based on them, so a brief overview of the system types identified by Krupitzer et al. [64] is given in the following section.

In addition to model-based approaches, like models@run.time [7], there are control theory-based, architecture-based, reflection-based and many other approaches. In control theory-based SAS, a control loop, like the MAPE-K loop is used to implement adaptation logic. Architecture-based systems model the architecture of themselves and adapt it according to their adaptation plans or goals. Reflection-based systems reason about their own state by making use of reflective language features, allowing them to inspect and potentially also modify instances of constructs like class, method and field. Beyond this, they found service-oriented, nature-inspired, learning, formal modeling and agent-based approaches, as well as miscellaneous approaches and systems that facilitate adaptation via general methodologies of software engineering, like component-based systems. In robotic systems, and especially decentralized ones, agent-based and nature-inspired approaches are of interest. Some of the concepts nature-inspired systems encompass, like flocking, as well as the core of agent-based SAS — individual agents (or robots) being able to act *autonomously* and *cooperate* [64] — are discussed in Section 2.3.

### 2.1.3. Behavior modeling

The system proposed in this thesis allows programmers to directly model the behavior of robots in an adaptive manner within the runtime model. At the core of such a model lies a task switching algorithm, which determines when a task should be exchanged for another. In this section, Finite State Machines (FSMs) and more specifically their hierarchical extension HFSMs are introduced, as they are used for switching behaviors in RuMROS*. Finally, Behavior Trees (BTs), a more recent approach that improves on HFSMs is discussed. Further approaches to this, such as Petri Nets [96], are beyond the scope of this thesis, but will be discussed in Chapter 7 in the context of future work as they offer additional useful features.

FSMs are derived from finite automata [105] and, on a high level, are defined by a set of states and transitions that link one state to the next. A FSM has a defined starting state and can only pick one of possibly multiple transitions per state and thus can only be in one state at a time. In the context of modeling behavior, the current state of a FSM corresponds to the currently executing behavior, while transitions could be annotated with conditions, pointing to the next behavior to execute when the associated condition has been met. In a way, this is reminiscent of "goto"-statements or conditional jumps in code [55], as switches occur by changing the code that is currently executed conditionally. An advantage of this approach is that it is simple, compared to other approaches like BTs while being expressive enough to be widely used in systems [5]. The main drawback of FSMs is their lack of modularity, as in order to be expressive, any given state may have a large number of transitions, which can become cumbersome to manage. HFSMs aim to alleviate this problem by grouping states into hierarchies [55]. This increases modularity, as the number of transitions on each state is reduced by introducing sub-states. Modeling complex tasks is facilitated by such a structure, as this abstraction allows for a divide and conquer strategy in this regard.

While HFSMs are expressive and more modular than FSMs, they still have the drawback of producing potentially complex graphs, making control flow traces more complex and graphical representation more difficult. BTs on the other hand exclusively work with trees, allowing them to keep good modularity, while encouraging hierarchical structures and facilitating visualization [21]. At their core lies the concept of switching between *Action* nodes, which represent a certain behavior or task based on sibling *Conditional* nodes and their parent nodes. Action and conditional nodes are leafs in a BT, while their nonterminal parent nodes can be *Sequence* and *Fallback* nodes. Conceptually, behavior trees recognize three reasons to switch to another task — successful termination, failure and interruption. Since links between nodes of a BT do not model transitions between tasks directly, there is no set of conditional transitions to be evaluated. Instead, while running, a BT is evaluated starting from the root in a depth-first manner, beginning to execute the leftmost action. While control flow is propagated downwards this way, node states are propagated upwards from the currently running action to the root of the BT. While an action is running, the current action keeps executing until finished or failed, propagating the corresponding state upwards (interruption is handled differently, this is explained in the next section). After returning "success" or "failure", handling depends on the parent node. If part of a Fallback node and successful, this parent node is considered finished, moving on to the next sibling for execution in the BT. On failure however, a Fallback node invokes the next child action node (sibling to the right of the failed action node), conceptually housing alternatives or

fallback actions in case an action fails. When part of a sequence node on the other hand, the next sibling is executed on success only, while skipping the rest of the sequence on failure.

Interruptions are modeled by utilizing conditional nodes as leftmost child nodes of fallback nodes. A conditional node does not execute behavior, i.e. directly commanding actuators, causing a change in the environment. Instead, it evaluates a condition with every tick and then hands over control flow to the next sibling action node if the check fails. Therefore, it may only return "success" or "failure", not "running". Conceptually, the whole sub-tree of fallback parent node, conditional node and sibling action node represents an interrupt. The conditional's sibling defines the interrupt behavior that is executed when the condition fails, while on success, the parent fallback node naturally hands over control flow to the next sub-tree, continuing execution without interruption.

This way, behavior can be modeled hierarchically, avoiding superlinear growth of transitions with an increasing number of states. Continuous behavior can also be modeled, by restarting evaluation when the rightmost action node finishes, executing the leftmost action again, stopping only if an action node that runs forever is reached. In comparison to FSMs or HFSMs, BTs are less expressive (without further extensions), since they cannot use internal variables to factor in past decisions and change their behavior accordingly. However, they are more reactive, as they restart execution from the root at each new input (i.e. when an action node reports a different state). The main benefit that ensues from this is easier adaptation at runtime. Due to their structure, BTs also have a more manageable linear complexity compared to FSMs, which are at best quadratic in complexity [55].

## 2.2. Underlying frameworks

The main framework used to implement the solutions given in Section 1.3 is RuMROS. As it uses ROS 2 to control robots on lower layers and adds runtime modeling capabilities via JastAdd, it is detailed after an overview of these underlying components.

### 2.2.1. ROS 2

The Robot Operating System 2 (ROS 2) is an open-source framework for building modular and distributed robotics applications. Its predecessor, ROS, already provided a modular framework with hardware abstraction, but was lacking production-grade features and performance [73]. With ROS 2, released in 2017, these issues were addressed, increasing its practicality for real-time distributed systems. It provides a collection of tools, libraries, and runtime components to support the development of robot software, focusing on scalability, real-time performance, and cross-platform compatibility (C++, Python and on both Linux and Windows) [73, 75].

A central concept to ROS 2 are nodes — independent computational units that can be developed, deployed, and executed separately. A node can perform a discrete function, such as sensor data processing, control, or actuation, or multiple of them at the same time. Designing these nodes which monitor and steer robots is up to developers. ROS 2 supports node composition, allowing multiple nodes to be loaded and executed in the same process. This reduces overhead and improves performance in resource-constrained systems [73, 75].

Separating sensing nodes from computation and actuation nodes makes a system modular and enables it to be distributed across different devices. To enable a potentially distributed network of nodes, ROS 2 supports multiple types of inter-node communication, which form the interfaces of each node:

- *Topics* implement a publish/subscribe model, asynchronously and anonymously sending and receiving data. A node publishes typed messages to a named topic, and other nodes can subscribe to receive this data, forming a many-to-many relationship. This mechanism is typically used for continuous data such as sensor streams or control signals.
- *Services* provide a request/response communication model for synchronous interaction. One node offers a service that another node can call and await a reply from. Services are typically used for configuration or queries.
- *Actions* support goal-oriented tasks that provide feedback and can be canceled. As they are non-blocking, they are suitable for goals such as navigation or manipulation tasks [73].

ROS 2 offers a set of functions to setup the required objects within a node, in order to make use of these communication interfaces. If a topic does not exist when a node subscribes or publishes to it, it is created dynamically, allowing order of execution to vary without causing problems. services and actions on the other hand have to be declared and their nodes have to be launched in order to use them.

To implement these communication schemes, ROS 2 is built on top of the Data Distribution Service (DDS) standard [91]. DDS is a peer-to-peer publish/subscribe middleware that offers configurable Quality of Service (QoS) parameters, enabling control over message delivery reliability, timing, and resource usage [73]. This middleware abstraction layer provides decoupling between ROS 2 and the underlying transport protocol as well as operating system. Several DDS implementations are supported, including Fast-DDS[2] and CycloneDDS[3] as the most popular options. These will be relevant in Chapter 5, as they can have a significant impact on performance [75].

Finally, in addition to middleware, ROS 2 also offers algorithms and developer tools. Algorithms are helper libraries, that provide functionality which is commonly used in robotics. This most notably includes Simultaneous Localization and Mapping (SLAM) via slam_toolbox [72] and motion planning with MoveIt [20]. As for tools, ROS 2 includes a variety of both command-line and graphical utilities for configuration, node launching, introspection, visualization, simulation and more. The most important tools central to development, debugging and visualization are:

- The *ros2* command-line utility suite for interacting with nodes, topics, services, and actions. It enables launching nodes, viewing and publishing topic messages, as well as calling services and invoking actions for instance.
- *colcon* as the primary build tool, with support for multi-package workspaces.
- *rosbag2* to help with development, debugging as well as performance evaluation by

[2]Fast-DDS source code: `https://github.com/eProsima/Fast-DDS`
[3]CycloneDDS source code: `https://github.com/eclipse-cyclonedds/cyclonedds`

recording and replaying messages.

- *rviz2* for 3D visualization of robot data and state.
- *rqt* as well as *rqt-graph* for further introspection and visualization of communication.

### 2.2.2. JastAdd

In the prototype application developed in this thesis, JastAdd[4] is used to enable runtime modeling capabilities. Originally, it was designed as a meta-compilation system [50], to facilitate writing compilers. As is common in compiler systems, it represents the internal structure of the language as an AST.[5] In this work, JastAdd is used as a runtime modeling system, as it offers some beneficial functionality, the usage of which in the context of runtime modeling with ROS is discussed in more detail in Section 2.2.3.

On a high level, JastAdd works with Reference Attributed Grammar (RAG) specifications, and comes with a compiler, which translates them directly to Java code, which can then be used in arbitrary Java applications. It offers M3-level meta-modeling capabilities with code generation, enabled by two distinct languages that allow developers to separately model structure and semantics of an application. The meta-model of the structural part is specified in a language designed for RAGs [49]. As it specifies the objects that the semantic specification works with, it lives on the meta meta-modeling level (M3) of the meta-modeling stack and is also called a meta-language. For the semantic part, a DSL with Java-like syntax is used, which composes the objects defined by the concrete RAG. Instead of specifying the meta-model, it specifies concrete models for an application scenario and therefore is an M2-level language. Both RAGs and the modeling DSL are discussed in the following sections.

#### Reference Attributed Grammers

RAGs build on Attribute Grammars (AGs), which were introduced in 1968 by Donald Knuth [62] as a way to declaratively describe context-sensitive properties of language constructs. These attributes can be automatically computed, as long as any given grammar specification is non-circular. Reference Attributed Grammars are an extension to AGs, that allows attributes to be references to AST nodes that can be followed, similarly to pointers in C. This overlays a custom reference graph over the parse tree of any concrete AG and thus permits it to be circular. The main idea of this extension is to make the grammar more expressive and more in line with modern object-oriented languages like C++, C#, Rust or Java, which support pointers natively [49].

While RAGs are as expressive as circular AGs, support for noncontainment relations, which are also part of modern languages, is still lacking. For this purpose, Mey et al. proposed another extension — Relational Reference Attributed Grammars (RelRAGs) [79]. They describe a fundamental mismatch between RAGs and conceptual models — general models are represented as graphs, while RAGs are intrinsically tree-structured. Additionally, RAGs lack direct support for modeling bidirectional relations, which is natively supported by

[4] JastAdd is a meta-compilation system. For more information, see `https://jastadd.cs.lth.se/web/`.
[5] The AST used by JastAdd additionally supports node references.

many modeling languages, like the Unified Modeling Language (UML) for instance. Mapping bidirectional noncontainment relations to plain RAGs requires significant effort and introduces overhead in form of either manually defined lookup or unmanaged redundancy of two opposing unidirectional relations. Relational RAGs provide additional syntax to specify noncontainment relations, and are therefore used to specify the constructs of the runtime model in RuMROS.

Listing 2.1: Exemplary relational RAG for a university

```
// Definitions with only containment relations
University ::=
    students:Student*
    lecturers:Lecturer*
    courses:Course*;

abstract Person ::=
    <name:String>;

Student : Person ::=
    <matId:int>
    <courseOfStudy:String>;

Lecturer : Person ::=
    <staffId:int>
    <chairName:String>;

Course  ::=
    <title:String>;

// Noncontainment relations
rel Student.enrolledCourse* <-> Course.enrolledStudent*; // enrolled
rel Lecturer.teachingCourse* <-> Course.assignedLecturer; // lecturing
```

Listing 2.1 shows an exemplary RAG for the structural definition of a university with students, courses and teachers. JastAdd supports inheritance and abstract AST node types, as the target language, Java, supports these features natively. This makes it easy to define a tree structure of the university with all relevant objects, without having to write duplicate containment relations, as the name of a person can be inherited for both students and lecturers. Containment relations (the JastAdd developers also call them components) are defined with `Nonterminal ::= Component1 Component2;`, allowing any number of component attributes to be specified. Components can additionally be annotated as follows:

- `component-name:type` to name the component relation, while indicating the type of the component,
- `[Component1]` to indicate that the component is optional,
- `Component1*` to indicate that the component is a list of *Component1* objects,
- `<Component1:type>` to declare the component as a terminal attribute of the given type (String by default, if type is omitted) and
- `/Component1:type/` to declare the component as a nonterminal component.

Terminal components, in the context of a context-free grammar, describe attributes that are simply assigned a value of a basic type [62]. Nonterminal attributes on the other hand are computed. In JastAdd, if the grammar defines a nonterminal component via

`/Component1:type/`, its computation has to be explicitly specified in the DSL for aspect-oriented behavioral extensions. Nonterminals spcified before `::=` are treated as AST nodes and translated to Java classes.

In order to process relations, defined via the `rel` keyword, the RelAST preprocessor[6] is required. It enables the definition of noncontainment relations, which are translated to regular JastAdd syntax in a preprocessing step. In the example, the *enrollment* relation allows students to enroll into multiple courses. As the relation is bidirectional, courses also hold references to their enrolled students. References are manifested as either Java reference variables, or Java lists of reference variables, if `*` is used on the attribute to indicate multiplicity. The *lecturing* relation is a many-to-one bidirectional relation, allowing lecturers to teach a number of courses, whereas each course can only have one lecturer. When the JastAdd compiler is called to convert the resulting RAG of the preprocessing step to Java code, these relations are manifested in methods like `List<Course> Lecturer.getTeachingCourses()` and `Lecturer Course.getLecturer()`, which developers can use.

### DSL for aspect-oriented behavior definitions

As JastAdd uses Java code as its compilation target, the syntax of the modeling DSL is similar to it and allows for direct integration of arbitrary Java code into compiled classes. This can be seen as the imperative aspect of a model. Attribute declarations on the other hand are declarative and JastAdd allows them to be specified separately to maintain good separation of concerns. Furthermore, the DSL follows an aspect-oriented approach, allowing developers to specify semantic extensions in the form of computed attributes and references in separate aspects. This principle follows the Aspect-Oriented Programming (AOP) paradigm and enables several benefits [35], including:

- Better maintainability by reducing code scattering, as cross-cutting concerns are grouped in aspects,
- improved modularity, as aspects are semantically coherent modules and
- better code reuse, as modular aspects can be reused in different models.

The syntax of the DSL considers aspects as primary objects. They can contain new attributes, equations and rewrites of an existing aspect's attributes. As the underlying structure is an AST, attributes can be computed in a top-down or bottom-up manner, which the DSL supports via the `inh` and `syn` keywords respectively.

[6]The RelAST preprocessor is a result of the work of Mey et al. [79]. It is open-source and available at `https://bitbucket.org/jastadd/relational-rags/src/master/`

Listing 2.2: Exemplary JastAdd semantic extension aspect, which adds a computed timetable attribute

```
aspect TimeTable {
  syn String Student.getTimetable() {
    List<Course> courses = getEnrolledCourses();

    String timetable = "Timetable for " + getname() + ":\n";
    for (Course c : courses) {
      timetable += c.getday() + " " + c.gettime() + " - " + c.gettitle() + "\n";
    }
    return timetable;
  }
}
```

Listing 2.2 shows an exemplary semantic extension to the relational RAG for the university scenario provided earlier. It covers the declarative aspect mentioned earlier and should therefore be written in a `JRAG` file. The `TimeTable` aspect adds a synthesized attribute that represents the timetable of an individual student to the student AST node. The definitions inside an aspect can make use of the functionality (manifested in methods by the JastAdd compiler) offered by the node they are declared for. In this example, the methods `Course.getday()`, `Course.gettime()` and `Course.gettitle()`, as well as `Student.getname()` and `Student.getEntrolledCourses()` are automatically generated by declaring them as terminal attributes in the RAG. The JastAdd compiler directly weaves the aspect code into the corresponding classes of the AST. It also satisfies the cross-cutting mitigation that AOP offers, as an aspect can mix declarations for attributes and rewrites of multiple AST node types. The `TimeTable` aspect for instance may be extended by adding a method to the University node, which lists the most popular timetables for students in each grade in the future. For this, the `University` node has to be extended by the respective attributes. In order to represent the current year or grade of the students for this purpose, either the relational RAG can be updated to include it as a containment attribute or the aspect can declare it. In the latter case, the extension is kept fully separate, but has to manually declare the otherwise generated access methods. As briefly mentioned before, it is also possible to add imperative parts, which perform computations that don't yield AST attributes or node references, like logging course enrollments to a file for instance. They can be declared in a separate `JADD` extension, which, similarly to a `JRAG` file, also groups additions for possibly multiple different classes into aspects.

### 2.2.3. Runtime Model-driven ROS (RuMROS)

The framework this work extends upon, RuMROS, consists of three parts, which can be seen as the main layers of the system, as they build on each other and have clearly defined communication interfaces. An overview of them is given in Figure 2.1. The key feature of RuMROS is to allow the specification of a meta-model via JastAdd, which is then used to generate a runtime model and web UI to monitor and control the system at runtime. The runtime model controls robots via ROS 2 topics, allowing the system to benefit from the extensive toolchain, robot support and simulation capabilities ROS 2 offers. The runtime modeling capabilities allow developers to model behavior that is both adaptive at runtime as well as adjustable by human operators, although the initially provided exemplary model

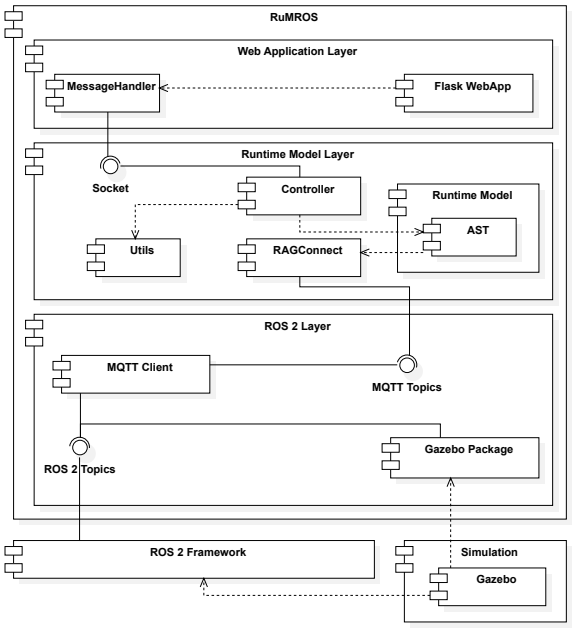


Figure 2.1.: Top-level architecture of RuMROS

concentrates on demonstrating the latter. Composition and invocation of behavior is up to developers, since RuMROS does not provide constructs for modeling them in a structured manner. In the following paragraphs, the structure of RuMROS is described and an overview of the initial runtime model is given.

### ROS layer

On the lowest layer, the ROS nodes of the robots reside. RuMROS includes a simulation scenario definition for Gazebo, with two Turtlebots,[7] a rover and a drone in an arena with some specified sub-areas. Gazebo creates the respective ROS nodes for each robot, which directly expose sensor data and actuator controls via topics. ROS messages are converted to MQTT[8] messages via the ROS component node offered by the *mqtt_client*[9] component. This node provides a bidirectional communication port for the runtime model layer above it to connect to, allowing it to receive sensor data and send commands to actuator endpoints via ROS topics.

### Runtime model layer

The runtime model layer is a standalone application containing three logical main components:

[7] More precisely, the Turtlebot3 waffle is used, see `https://www.turtlebot.com/turtlebot3/` for more information.

[8] MQTT is a widely used, topic-based protocol for IoT messaging. More information on the concrete specification can be found at `https://mqtt.org/`.

[9] mqtt_client is an open-source ROS 2 package. Source code is available at `https://github.com/ika-rwth-aachen/mqtt_client`.

- An AST representation of the runtime model
- RAGConnect[10] [112], which connects the AST to the ROS layer at runtime via MQTT
- A controller, which initializes the other components and sends model updates to the web UI, as well as receives actions from the web UI

The controller is a statically coded Java application, which, upon launching, sets up the communication interfaces to RAGConnect [112] on the lower layer as well as to a UDP server on the upper layer. The runtime model itself is fully generated from a meta-model, which is specified at compile time with JastAdd [50]. The UDP server periodically retrieves and serializes the runtime model and sends it to the web application. It also receives commands to update the model, when an action is performed on the UI, and processes them accordingly.

### RAGConnect

RAGConnect, on a high level, handles updating nodes of the runtime model with data from incoming MQTT messages from the ROS layer, as well as sending MQTT messages with values of updated nodes to the ROS layer. In order to do this, RAGConnect uses a configuration file, which is specified in a DSL that is similar to a RAG, but also allows Java code to be included. An overview of the syntax is given in Listing 2.3.

Listing 2.3: Grammar of the RAGConnect configuration file, taken from Schöne et al. [112]. Definitions made by the developer are enclosed in angle brackets, plain symbols are reserved keywords. Symbols ending with -name correspond to a type definition in the corresponding RAG

```
<port> ::= <direction> <options> <target > using <mappings>;
<direction> ::= send | receive
<options> ::=  | indexed | with add | indexed with add
<target> ::= <nt-name>
| <nt-name> . <child-name>
| <nt-name> . <attr-name> (<type-name>)
<mappings> ::= <mapping-name> , <mappings> | <mapping-name>
```

Central to RAGConnect is the concept of a port — an entity which is connected to external systems. These ports are unidirectional and can either send or receive data. Endpoints for this data have to be nonterminal components, either by themselves or within the context of a parent nonterminal, to narrow down selection. This allows for particular attributes of the AST to be designated as receivers or senders of data. Senders are enriched with RAGConnect methods in a preprocessing step, allowing them to propagate new values on update to an external system, while receivers are given methods to update their values upon receiving data from an external system. The connection to the external systems is done via MQTT, by mapping endpoints specified in the RAGConnect configuration file to MQTT topics. This is done in another configuration file and explained further in Chapter 4. Finally, in order to properly update the data, mappings can be specified. RAGConnect automatically handles mapping data from MQTT to AST

[10] RAGConnect is an open-source project that allows JastAdd AST nodes to be connected to communication ports. Source code is available at `https://jastadd.pages.st.inf.tu-dresden.de/ragconnect/`.

nonterminals and vice versa for primitive types. For advanced types, mappings can be defined with `<mapping> maps <type> to <nt-name> {: <Java code> :}`. This allows developers to process custom or advanced formats, like JSON or XML. Since arbitary Java code can be executed, mappings also allow for further computations, RuMROS for instance uses mappings to unmarshal JSON orientation data of the robots. It comes in the form of quaternions, so further computations are required to extract only the rotation around the z-axis, which corresponds to a robot's heading direction. In RuMROS, RAGConnect is configured to receive and update every robot's pose (position and orientation) in the runtime model. To actuate robots, ROS velocity commands are used. The appropriate nonterminals are configured to be sent to ROS.

### Development pipeline

In order to run their robotic application with RuMROS, developers need to specify the RelRAG (`Model.relast`), JastAdd behavioral definitions (`JRAG` and `JADD` files) and RAGConnect configuration file (`Model.connect`) in order to compile the application. The provided exemplary model includes them for the given Gazebo simulation scenario. First, the RAGConnect preprocessor generates an extended specification from the given RelRAG and RAGConnect configuration file. The RelAST preprocessor then translates it into a regular JastAdd specification. Finally, the JastAdd compiler is invoked to produce Java code, which in turn can be compiled and run with the Java Virtual Machine (JVM). The runtime model is then ready to be loaded by the provided Java controller. RuMROS comes with a bootstrapping script, that launches it and all other parts of the system, including the web UI and Gazebo simulation.

### Runtime model

The RelRAG of the provided exemplary application is shown in Listing 2.4. It specifies nonterminals for objects related to the runtime model, like `Robot` with concrete robots as its subclasses, `Area` and data structures for state, velocity and position of robots. Therefore, it models both robot functionality and (to a limited extent) also the environment. For controlling the robots, the model contains structures like `Action`, `Input` and `Result`, which are used to define high-level actions with parameters. These are then displayed on and can be executed from the web UI. All nonterminals relevant to the model itself are grouped under the `Model` node, which is required for the controller to update the web UI properly. Noncontainment relations are used to link states and positions, that are part of the model, to each individual robot or drone.

Listing 2.4: The RelRAG of RuMROS's meta-model

```
Model ::=
    Turtlebot*
    Rover*
    Drone*
    Area*
    State*
    Action*
    PositionReference*;

abstract Robot ::=
    <id:int>
    <name:String>
    /Velocity2D/;

rel Robot.CurrentState -> State;
rel Robot.TargetPosition? -> PositionReference;

Turtlebot : Robot ::= PosePositionReference;
Rover : Robot ::= OdomPositionReference;

Drone ::=
    <id:int>
    <name:String>
    OdomPositionReference
    /Velocity3D/;

rel Drone.CurrentState -> State;
rel Drone.TargetPosition? -> PositionReference;

abstract PositionReference ::=
    /Position/;

PosePositionReference : PositionReference::=
    Position;

OdomPositionReference : PositionReference ::=
    Position;

DefaultPositionReference : PositionReference::=
    Position;

Position ::=
    <x:Double>
    <y:Double>
    <z:Double>
    <theta:Double>;

Velocity2D ::=
     <speed:Double>
     <rotationSpeed:Double>;

Velocity3D ::=
     <horizontalSpeed:Double>
     <verticalSpeed:Double>
     <rotationSpeed:Double>;

Area ::=
    <id:int>
    <name:String>
    <xPos1:Double>
    <yPos1:Double>
    <xPos2:Double>
    <yPos2:Double>;

State ::=
    <id:int>
    <name:String>
    <speedFactor:Double>
    <angleTolerance:Double>;

Action ::=
    <id:String>
    <label:String>
    Input*
    Result;

Input ::=
    <id:String>
    <label:String>
    <type:String>;

Result ::=
    <type:ActionResultType>
    <message:String>
    <displayMillis:int>;
```

In JastAdds semantic part of the model specification, RuMROS defines concrete actions robots can perform. Robots can drive and drones can fly to an area and this can be canceled. This behavior is also directly implemented in `JRAG` files and part of the runtime model. When an action is selected on the UI, the controller of the runtime model invokes it, changing the state of the robot it was invoked on. RAGConnect is configured to periodically send velocity commands to the robots via the computed nonterminal `/Velocity2D/` on `Robot` and `Drone`. The method that implements the computation acts like a per-robot state machine, returning zero velocity when the robot is idle. When driving to an area, it uses the current pose, as received from ROS, to compute forward and angular velocities. RAGConnect then sends them to the ROS node, steering the robot to the

target area. When the *cancel* action is selected for any robot or drone on the web UI, the state machine switches back to the idle state and sends zero velocities, stopping the robot.

#### Human introspection layer

On the final layer on top of the runtime model, the web application for human introspection resides. It is implemented in Python 3 using Flask[11] and runs as a standalone application, like the runtime model. The backend consists of a message handler, which receives the model in the form of JSON data from the runtime model layer, as well as a controller. The latter orchestrates communication between the introspection layer and runtime model layers and handles web requests. The web frontend displays the data of the runtime model AST nodes as tables, shown in Figure 2.2. The UI also displays the actions developers specified in the runtime model, along with fields to select and set parameters. These actions can be executed by users of the web interface, which causes the controller to send action data including parameters back to the runtime model layer's server via UDP. While the UI of the web application is fully generated, it requires model data to be of a certain structure. The meta-model (RAG) has to define a `Model` node and action structures such that they can be executed on a target robot. The `Result` nonterminal is required to display errors that may occur in the controller of the runtime model application when an action is invalid.

## 2.3. Multi-robot systems

In order to extend RuMROS to an application capable of properly handling certain MRSs, it is important to distinguish different kinds of such systems. In the following sections, an overview over general MRSs[12] is given and existing taxonomies and axes for classification are laid out. Swarm systems as special decentralized systems are discussed in particular, as their design methods are reflected in RuMROS*.

### 2.3.1. Taxonomies and classifications

In the past, a number of different axes have been suggested to classify MRSs [18, 23, 31, 42, 65, 76]. Cao et al. [18] focused on *cooperative* MRSs, proposing the main axes of group architecture, resource conflict resolution and origin of cooperation. In their terms, group architecture includes control architecture (centralized, decentralized) as well as differentiation (homogeneous, heterogeneous) and interaction structure (interaction via environment, sensing or communication). These are axes that can be applied to general MRSs. Resource conflict resolution and origin of cooperation on the other hand are viewpoints related to cooperative systems in particular. Dudek et al. [32] expanded on this, refining the existing

[11] Flask is a Web Server Gateway Interface (WSGI) application framework for Python, available at `https://flask.palletsprojects.com/en/stable/`.

[12] "Multi-robot systems" in this section explicitly means systems that employ multiple robots. "Multi-agent systems", where robots have their own functionality and some degree of autonomy are referred to explicitly in this section, if the term applies in the more narrow sense. This is to avoid confusion with literature that explicitly uses the terms.

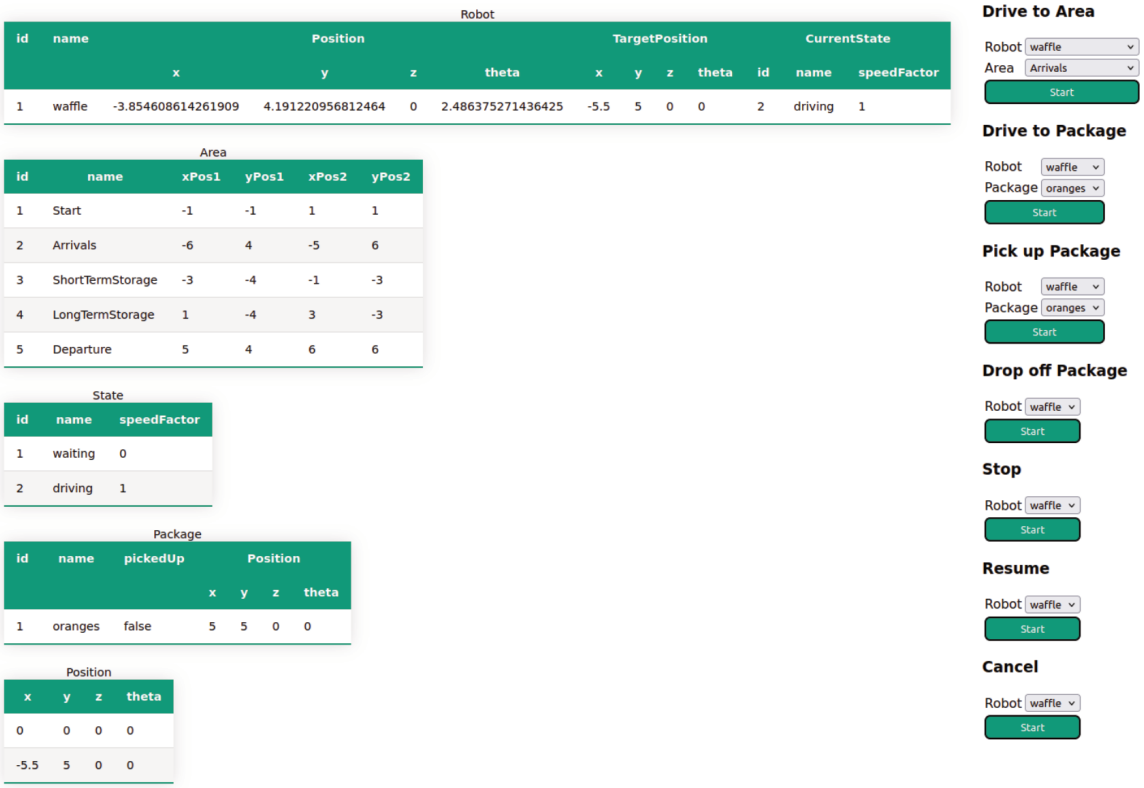


Figure 2.2.: The GUI of RuMROS, taken from [117]. It showcases its passive UI elements to display data tables, as well as active elements for parametric executable robot actions.

axes, while focusing on the design of the system, in order to make principled design decisions easier. In systems with explicit communication, they additionally differentiate range, topology and bandwidth of the provided infrastructure. Notably, they also classify systems according to their size (in terms of number of robots) and dynamic reconfigurability. Mazdin and Rinner proposed a simplified task-oriented taxonomy based on task type, coordination and organization [76] as independent layers. It is depicted in Figure 2.3.

They classify tasks that require multiple robots for execution as multi-robot tasks. These can be decomposed into sub-tasks (which is a common workflow for MRSs [107]) and tightly or loosely coordinated. In tightly coordinated systems, the sub-tasks require a certain level of organization and thus synchronization via some form of interaction (most often communication) between robots. Loosely coordinated systems on the other hand can execute sub-tasks separately and independently, without any form of synchronization.

In order to coordinate multiple robots, different behavioral patterns can be used. Gautam and Mohan propose the additional axes of *mapping and localization*, as well as *coordination and control techniques* [42]. The former is required as a basis for coordination and control techniques, which in turn implements the aforementioned behavioral patterns, like flocking, which was first defined by Reynolds with heuristic rules [106]. These techniques are often applied in swarm systems, which are defined and discussed in Section 2.3.2.

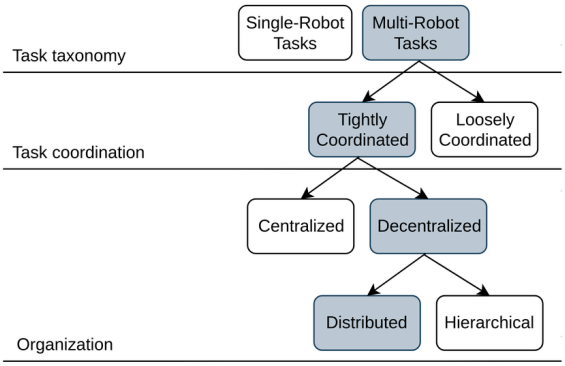


Figure 2.3.: Taxonomy of MRSs by Mazdin and Rinner [76] according to type of task

As systems have evolved over the years, even more types have emerged. In a recent survey of MRSs [23], in addition to centralized and decentralized control architectures, distributed and hybrid architectures have been identified as further classes. In centralized systems, one central entity controls the actions of all robots, whereas in decentralized ones, either each robot has its own independent controller or robots are assigned groups with one group controller each. Distributed control on the other hand is similar to a decentralized architecture, while specifically requiring a form of interaction structure that coordinates (and thus distributes) control over the system as a whole. Hybrid architectures combine any of the aforementioned types, for instance by having a central controlling entity, but with a limited degree of influence, as some of the control architecture is outsourced to individual robots. In terms of communication they differentiate between implicit, explicit and uninformed systems. Implicit communication includes the previously mentioned communication via the environment (also called stigmergy [47]) as well as via sensors. Explicit communication includes all previously described methods where robots employ some form of direct communication with neighbors, all other robots, group controllers, etc. Uninformed systems do not employ any form of communication at all.

### Cross-cutting aspects and consolidation

Some aspects frequently mentioned in literature are cross-cutting existing taxonomies. Autonomy for instance is one such aspect, as in centralized systems, individual robots have a low degree of autonomy or none at all, whereas in decentralized systems, each robot may execute its own behavior in at least a limited way. Kugele et al. therefore propose different levels of autonomy: non-autonomous, intermittent autonomous, eventually autonomous, and fully autonomous [65]. Non-autonomous systems require user input at all times to control their operation, for instance via a remote control. In intermittently autonomous systems, the system can operate autonomously between user inputs. In such systems, user inputs may be commands to execute specific behavior, i.e. part of the logic the user would perform in a non-autonomous system is shifted to control logic of the robots. In eventually autonomous systems, there is a learning component. Initially, user input is required, but this is reduced as the system learns the behavior it should exhibit. Fully autonomous systems require no user input at all and can adjust their behavior by themselves, according

to external and internal context and depending on their objective. Self-adaptation (and other self-* properties as well) is another cross-cutting concern, as it requires some degree of autonomy [97].

Parker also differentiates between more general types of interaction in MRSs [93], related to awareness of other robots and meta-information, like system goals and objectives of other robots. This cross-cuts self-awareness and requires runtime modeling. Figure 2.4 shows the classes she suggests: collective, cooperative collaborative and coordinative systems.

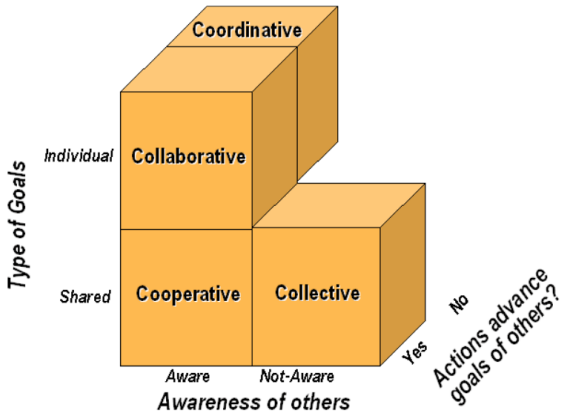


Figure 2.4.: Interaction types in MRSs according to Parker [93]

In collective systems, robots are not aware of each other, but they share goals and their actions are beneficial to other team members. This if common in swarm systems. Cooperative systems add awareness of other robots to collective systems. Such systems are often used to perform joint tasks where robots may work on different sub-goals. Collaborative systems also have awareness and the actions of robots benefit the others, but the robots have individual goals instead of a shared goal. This, however, requires the individual goals to be compatible (for instance, a robot moving a box from A to B and another with the task of moving it from A to C would be incompatible). The final type, coordinative systems are systems where robots are aware of each other but they have different goals and their actions do not benefit others. This is often the case when robots that are part of different systems share a workspace, and have to coordinate their actions in order to avoid conflicts and collisions.

In summary, there are many different taxonomies of MRSs. Some attempt to cover general systems, while others focus on a certain viewpoint. In this thesis, in order to classify RuMROS* later on, previous taxonomies are consolidated in an overview for collective, cooperative, collaborative and coordinative MRSs. It is depicted in Figure 2.5 and includes all elements of previously mentioned taxonomies, as well as cross-cutting aspects.

### 2.3.2. Holons and swarm robotics

In Chapter 1, problems and corresponding goals for the prototype framework developed in this thesis were analyzed. One particular problem is the performance of the system with respect to scaling. Centralized systems, this includes the current version of RuMROS, have

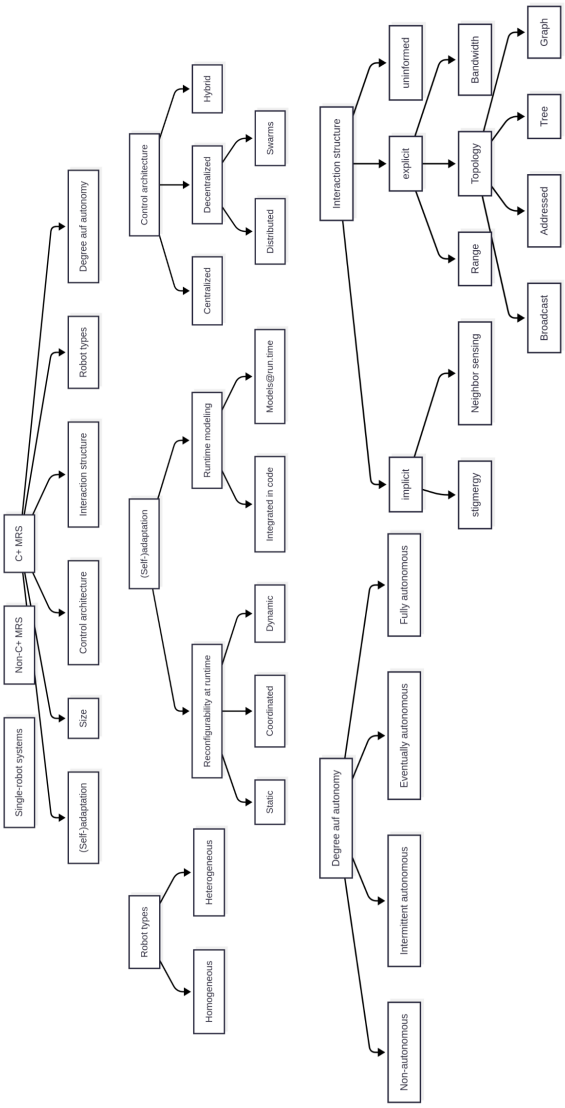


Figure 2.5.: Consolidated taxonomy of MRS on the basis of [18, 23, 32, 42, 65, 76]. C+ is used as an abbreviation for collective, cooperative, collaborative and coordinative systems.

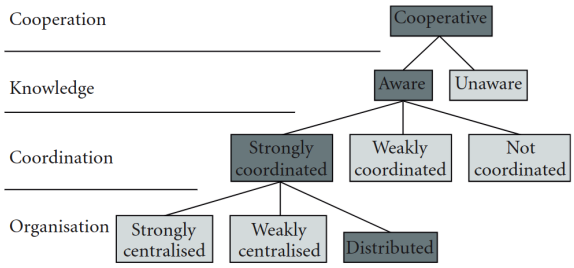


Figure 2.6.: Robotic swarm systems in the taxnonomy of Iocchi et al. [54] by Navarro et al. [83]. Dark grey marks the type of system corresponding to a swarm robotic system.

performance limitations according to the computing power of the central controller. This problem is intrinsic to the architecture and also occurs in non-robotic systems [56]. To mitigate this problem, it was established that the prototype system should at least partially support decentralized behavior, by having a hybrid control architecture. *Robotic swarms* are a class of systems that has particularly high demands with respect to the degree of decentralization, so they are typically implemented with a fully decentralized architecture. As this offloads the main part of computations to individual nodes, behaviors used for evaluation in Chapter 5 will resemble swarm behaviors, in order to test the scalability limits of RuMROS*. In this section, the structure of robot swarms as well as typical behaviors they exhibit are explained to provide a baseline.

*Swarm robotics* deals with the problem of coordinating large groups of robots through the use of local rules [83]. The inspiration for swarm robotics comes from insects in nature. Often organized in large colonies, they lack global coordination, but can exhibit higher level behavior by collaborating on a smaller scale. This local collaboration, mimicked in swarm systems by local coordination rules allows global behavior to emerge, known as *emergent behavior* [69]. According to Kounev et al.[123], emergent behavior can also be described by decentralized (or distributed) self-awareness, as the awareness of the system as a whole is also distributed across individual actors.

Swarm systems can be classified using the taxonomy by Iocchi et al. [54], which is similar to the consolidated classification depicted in Figure 2.5. This was done by Navarro et al. [83] and is shown in Figure 2.6. They describe robotic swarms as cooperative, aware, strongly coordinated and distributed (strongly decentralized) systems.

In terms of structure, complex natural systems often exhibit hierarchical structures [115]. However, larger organizational structures in nature, like a school of fish, are composed of individual organisms, which are in turn composed of organs and furthermore cells. On every individual level in this hierarchy, the subjects are self-contained systems. This is described by *holonic systems*, which have containment as the main relation between levels of the hierarchy [26]. Therefore, holonic systems are recursively self-contained hierarchical systems. The subjects on each level in such a system are called *holons*. Holons have a dual nature. For one, they are self-contained and independent, resembling a black box to outside observers and thus represent a unit of abstraction. This property is also called its *whole* nature. On the other hand, a holon also has a *part* nature, as it is integrated into larger

organizational structures and is subject to its influence.

In practice, robotic swarms, unlike their biological counterparts, are not always hierarchically organized. In literature, *groups* are often mentioned as non-hierarchical organizational units [8, 27, 30, 83]. They are the most basic form of organization, simply containing robots as peers, without any form of hierarchy. Another form of organization in cooperative MRSs are heterogeneous groups, not in the sense of containing different types of robots, but the robots having different roles. As in nature, like with flocks of birds, group formations with explicit *leaders* and *followers* have been examined [56]. Navarro et al. however exclude them from their definition of a robotic swarm [83], while others, to the best of this thesis' author's knowledge, do not explicitly mention them.

### Swarm behaviors

Over time, applications of swarm system have been researched and as a result, a collection of "standard problems" with associated

standard behaviors to tackle them have been established. Beni provides such a list [8], grouping tasks under the domains of pattern formation, environmental entities and complex tasks:

- Pattern formation
  - Aggregation
  - Self-organization into a lattice
  - Deployment of antennas, sensors, maps, etc.
  - Covering of areas
  - Mapping of the environment
  - Creation of gradients
- Tasks related to an environmental entity
  - Goal searching
  - Homing
  - Finding the source of a chemical plume
  - Foraging
  - Item retrieval
- Complex tasks
  - Cooperative transport
  - Mining
  - Shepherding
  - Flocking
  - Containment of oil spills

This list is not exhaustive, but illustrates the most common application domains and nature of their corresponding tasks. In this thesis, the complex task of flocking is of importance as a selected exemplary task to illustrate and test the functionality and limits of the system, as it combines multiple aspects of other tasks. In general, it means individual robots interact, for instance, creating and holding a formation, while sharing and working towards a given goal [89]. Said goal could be a navigation task for instance.

# 3. Existing systems and state of the art

In this chapter, the current state of the art regarding the most important research areas of this thesis are presented. These areas include (self-)adaptivity, runtime modeling and robotic systems, with focus on MRS. The following sections outline the key branches of each research area that are relevant to the system developed in this thesis, in order to identify potential gaps that this application may help to bridge.

## 3.1. Self-adaptivity in current systems

For general systems, development methods, architectural patterns and behavioral patterns have been used to implement adaptive behavior [51]. In the following sections, a brief overview of them is given, after which they are compared to RuMROS* and how it intends to allow for self-adaptive behavior.

### 3.1.1. Development methods

Surveys show that self-adaptivity and other related self-* properties, such as self-organization, have been topics of interest, independently of robotics [51, 64]. Hrabia et al. [51] found that a variety of processes to develop self-adaptive software have been suggested and evaluated. For agent-oriented systems,[1] model-driven methodologies have been explored. The ADELFE process [12], centered around a specialized, agent-oriented modeling language based on UML, provides modeling tools to design adaptation with respect to other agents. However, for self-* properties, only placeholders are provided. The actual design process of adaptive behavior is not addressed. TROPOS4AS [82] considers adaptive behavior explicitly, by utilizing goal models extended with annotations for environmental conditions and failure handling. This improves on ADELFE by providing more guidance for the design process of adaptive behavior, but only addresses adaptation with respect to internal state.

[1] Agent-oriented systems are systems composed of multiple separate entities (agents) that can act independently, thus having a certain degree of *autonomy*. In such systems, agents may also exhibit C+ behavior that emerges from the individual behaviors of the collective.

### 3.1.2. Architectural patterns

In order to design concrete self-adaptive systems, some architectural patterns, which enable adaptivity, have been described. Introducing a mediator [43] between internal system components or agents enables system parts to self-organize and thus to adapt to each other. An observer-controller architecture [14] can be used to implement behavior in a way that is similar to a MAPE-K loop. The observer collects data on the system's state, the controller then analyzes it, e.g. by utilizing metrics to compare against a goal, and then picks an action to adapt accordingly. While there are several other approaches to this [51], the main problem with them regarding self-adaptivity, is that only general guidelines for the design of self-adaptive systems are given. Such structures facilitate the implementation of adaptivity, but concrete implementation of data collection, metrics and reaction is left to developers, which seems insufficient [51].

### 3.1.3. Behavioral patterns

More concretely guiding the design process of adaptive behavior are the design patterns of de Wolf and Holvoet [24] as well as Fernandez-Marquez et al. [38]. The purpose of behavioral patterns is to provide a standard way of implementing behavior to tackle general and commonly occurring, often biologically inspired tasks in multi-agent systems. The patterns aiming to solve these tasks can be categorized into different levels of abstraction: basic patterns, composite patterns and high-level patterns [38]. For instance, flocking and foraging are common high level-tasks a swarm system has to perform. In order to accomplish this, basic patterns like repulsion and aggregation are necessary. When flocking, agents form a formation, which requires them to keep a certain distance to neighbors, in turn requiring neighbor sensing and computing a force field that is the sum of repulsive and attractive forces. Therefore, this task requires a form of self-adaptation: adjusting one's own movement according to observations of other agents. When foraging or exploring, agents should spread, also requiring repulsive forces. In addition to this, communication could be required to actively seek out unexplored areas or lead fellow agents to a discovered resource, which again could be realized with flocking behavior. This illustrates that these patterns can be composed to implement complex adaptive behaviors, easing development of them by employing the divide-and-conquer principle.

In contrast to development methods and architectural patterns for SAS, behavioral patterns provide a more fine-grained and detailed way of implementing self-adaptive behavior. Their main drawback is that since these patterns are biologically inspired, they are not universally applicable, as they are primarily targeted to decentralized or distributed systems, such as swarms. They imply a decentralized adaptation logic, that leaves adaptation of the system as a whole as emergent behavior. This increases the robustness of the system as a whole to failures of individual agents [51]. On the flip side, if global adaptive behavior is very complex, it may be still be difficult to design, even with patterns. In order to make sure that emergent behavior of pattern-based systems is in fact what was desired in the design and implementation phases, it is also vital to perform tests via simulation [34].

### 3.1.4. Self-adaptivity in RuMROS*

RuMROS* natively offers beneficial features for the implementation of self-adaptivity due to its runtime model-centric architecture. The causal connection between the model and the underlying system enables adapting the state of the model to also adapt the system itself in turn. This allows adaptation logic to be implemented in the model, thus benefiting from having more information than what is available on a robot-level. This information may also be preprocessed by the runtime model to represent high-level concepts of the application, simplifying adaptation code.

For systems with decentralized behavior, the framework developed in this thesis aims to provide a set of behavioral patterns for self-adaptation. While it does not solve the problem of leaving the behavior designer up to the composition of these components, it offers infrastructure to make this process more comfortable. This includes implementing behavioral primitives in an extensible and mostly hardware-independent way on the lower-level side, while providing functionality that facilitates self-adaptation on a higher level. In the runtime model, the composition of the system is modeled in a dynamic way, including the hierarchical composition of agent groups, allowing them to be adjusted at runtime. This allows the system to adapt not only by partially or wholly changing its behavior, it also allows for structural adaptation (or more specifically self-organization).

In other works, decentralized patterns have been implemented in the context of robotic systems [58, 101], a more concrete comparison of RuMROS* to *ROS2Swarm* [58] is given in Section 3.3.1.

## 3.2. Approaches to runtime modeling

Adaptation at runtime is one of the major challenges of using models in the development of robotic software [22, 121]. However, research in this area is still mostly focusing on analysis and design phases, while only few papers deal with models at runtime. Due to the lack of broad research in the application domain of robotics and in order to better categorize the approach proposed in this work, an outline of general runtime modeling approaches is given in this section. It is focused on works compatible with the definition of models@run.time [10]. Finally, current robotic systems that use runtime models are briefly covered in Section 3.2.2.

### 3.2.1. Models@run.time in general systems

In 2019, Bencomo et al. [6] conceived a taxonomy to classify approaches to models@run.time with seven main axes. It is shown in Figure 3.1 and classifies existing runtime models according to their modeled artifacts, type, purpose and applied techniques. Szvetits and Zdun also came up with a taxonomy [121] prior to this, which differentiates between model type, objective, architecture and technique. Therefore, it differs from the taxonomy of Bencomo et al. by emphasizing on architectural types instead of separating modeled artifacts from type. The structure of the following paragraphs follows the taxonomy of Bencomo et al., as it is more recent, but the findings of the literature review conducted by Szvetits and Zdun are included as well.

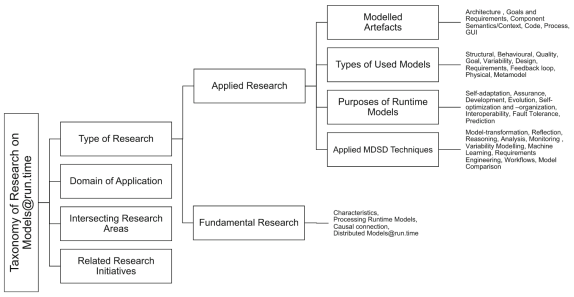


Figure 3.1.: Taxonomy of Models@run.time by Bencomo, Götz and Song [6]

### Modeled artifacts

The main subjects of models, i.e. modeled artifacts, cover different levels of abstraction. Most systems employ *architectural models*, which represent the state of the system's highest-level components and connections. Other high-level approaches include *goal or requirement models*, as well as *process models* that model the dynamic state, i.e. interactions of components over time. Some systems also employ models on *component* level, focusing on their individual state and disregarding connections and interplay. As the lowest level of abstraction, they identified the code level. Models on this level deal with some form of intermediate representation of the system's code structure. This includes abstract syntax trees, whose models are covered in Section 3.2.1 in more detail, as the approach used in this system also builds on this notion. They also include *context models*, i.e. models focused on the environment of the system, as well as *GUI models*. In the classification of [121], the model types are also chosen based on modeled artifacts, but they are not differentiable along the abstraction axis. Most notably, they additionally differentiate based on the paradigm according to which the models are implemented. In addition to AST-based models they mention *state machine models*, and more importantly for this work, *feature and aspect models*. Widely used, "traditional" aspect-oriented programming approaches like AspectJ[2] offer extensions of classes with methods and behavior, but they do not offer a way for structural definitions, i.e. specification of the class structure. There are frameworks like CaesarJ,[3] offering structural specification, in this case in the form of virtual classes, but unfortunately it has not been updated in over 10 years by now and thus isn't widely used. JastAdd [50], which the system proposed in this thesis uses, has a dedicated DSL for the specification of the model's structure and is actively maintained. This enables the code of the entire model to be generated.

### Model types

Bencomo et al. [6] have a more abstract notion of the model type. They found models which reflect the static structure and state of a system or its components, i.e. *structural models* to be the most common type. *Behavioral models* on the other hand focus on dynamics, describing actions based on the current state. Further approaches include capturing

[2]AspectJ is an aspect-oriented extension to Java, for more information see `https://eclipse.dev/aspectj/`.
[3]CaesarJ is a Java-based language that facilitates modularity, see `https://caesarj.org/index.html` for more details.

non-functional properties (*quality models*), *system goals* and possible variants of a system (*variability*). There are also models that deal with synchronizing models from the design phase with a running system [120] and on a more general level, models that specify how the system is modeled — *runtime meta-models* [66].

Gaps in research on runtime modeling systems become apparent when looking at what combinations of model type and modeled artifacts concrete systems use [6]. The most common combination by far are systems that use structural models for the system's state on an architectural level, with behavioral models of most types of modeled artifacts being the second most researched type. In terms of type, due to the nature of JastAdd, the system proposed in this work has the capabilities to model both system structure and behavior at runtime, forming a hybrid approach. As JastAdd uses AST-based models, it slots into the code-based model category in terms of modeled artifacts. However, the architecture of the system is part of the structural definition. At runtime, the AST represents the state of system components (robot's positions and behavior), as well as relations between them (group assignment), thus technically also modeling architecture. As a hybrid of types and modeled artifacts, the system employed in this work is novel and aims to provide modeling capabilities more extensive than those of its composing individual types or modeled artifacts. In theory, for modeling hybrid MRS,[4] structural and behavioral modeling are necessary when behavior is executed by the central controller. When each individual agent performs a task by using their own behavior however, the central runtime model predominantly has to deal with the system's structure.

### Purpose of models

Aside from abstraction, an inherent property of models, the most common purpose of runtime models is to facilitate the construction or operation of a self-adaptive system [6]. This is also the case for the framework proposed in this work, RuMROS*. Related to this aspect is another category with significantly less research: systems that can self-organize. RuMROS*'s runtime model is able to change the organizational structure of the underlying MRS, enabling self-organization. In addition to this, it integrates a GUI operable by humans, allowing the system to be monitored. It also offers dedicated simulation functionality derived from the model, slotting it into the category of *monitoring, simulation and prediction* by Szvetits and Zdun [121]. Systems, which combine runtime modeling with human control, like FlexBE [128], as mentioned in Section 1.1.2, are rare and a more recent development. To the best of this thesis' author's knowledge, the runtime model of RuMROS* offers a unique combination of structural and behavioral modeling with an aspect-oriented programming approach, that enables the previously mentioned properties.

Both surveys [6, 121] also identified a variety of techniques used in the implementation of the runtime models, including model transformations, reflection/introspection, reasoning, analysis, monitoring, variability modeling, machine learning, autonomic control loops (e.g. MAPE-K) and more. Relevant to this thesis are the following:

[4] Hybrid MRS in the sense of structure as described in Chapter 2: a centralized controller exists but individual agents have some degree of autonomy and can perform certain tasks on their own without requiring additional external inputs.

- *Analysis* - The employed model can access robot information depending on configuration (e.g. pose, velocity), process it and perform reactions,
- *Introspection* - The model backend uses introspection to serialize the model, making the UI as independent as possible from arbitrary concrete models,
- *Monitoring* - RuMROS* explicitly captures observations of the system by specifying ports for sensor data. This and further derived data are accessible in both the runtime model and monitoring UI.

Other approaches [11, 87, 128], which employ generic or generated user interfaces based on runtime models have been explored. RuMROS has a generic UI, which displays the runtime model and offers actions by communicating with the backend of the runtime modeling application, which in turn uses introspection techniques to decouple the model from it. This enables desirable properties like good separation of concerns and strong modularity, as well as making the UI adapt to the runtime model without any code changes. Compared to approaches that focus solely on generating UIs from runtime models [11, 87], it is less flexible. This is due to UI elements being assigned to model elements in a fixed manner (tables for non-terminals as well as parameter inputs and buttons for actions) as opposed to using dynamic mapping rules for instance. While this limits the customizability of the UI (without changes to UI code), RuMROS* focuses on structuring and displaying the model itself in a way that presents the information accordingly. Since it performs analysis and monitoring, it can be adapted to compute and display virtually any metric monitoring personnel requires.

### Model architecture

Regarding model architecture, the system proposed in this work falls into the category of systems with *model-aware middleware* [121]. This means that the model is part of a central entity, which maintains, continuously re-evaluates and enables access to the model for other (potentially external) components. Compared to *monolithic* architectures, where system components are not distributed, this provides additional flexibility for integrating new components. Existing approaches that share this architecture with RuMROS* often focus on dynamically exchanging components or rearranging their overall structure at runtime [121]. The framework prototype created in this work does not focus on dynamic addition and removal of robots or significantly enriching monitoring capabilities compared to the existing UI. The monitoring connections are static and determined at compile time. However, dynamic reconfiguration of the organizational structure that the runtime model contains is possible. System adaptation by reconfiguration has been explored previously in combination with runtime modeling, for instance with goal-based runtime models [127] and meta-models based on design knowledge, environment parameters and goals [129]. To the best of this thesis' author's knowledge however, enabling such functionality with an AST-based model in a robotic context has not yet been explored.

#### AST-based models

The idea of using AST-based models at runtime has previously been explored by Nierstrasz et al. [86]. They argue that representing high-level models with ASTs allows software to adapt to changing requirements or context in a more fine-grained way, while still keeping the model separate from actual code. In the approach they describe, they model high-level software concepts like classes using meta-objects and connect them to an AST via node links. The AST itself contains code for concrete objects that represent the current application. It is processed by a JIT compiler, allowing for dynamic code aspects, enabled by changing the AST and links to meta-objects. This improves on the previously named static AOP languages like AspectJ, as weaving can not only be done at compile time, but also at runtime. To implement the actual model, the authors use a DSL to cleanly separate the model from code and maintain a causal connection between them. This work focuses on exploring the degree of feasibility of modeling, monitoring and controlling a MAS with an AST-based runtime model in the application domain of robotics. This follows up on the research directions Nierstrasz et al. suggested [86] to support adaptive software: bringing models closer to code via DSLs and enabling model-centric development by treating models as first class objects.

### 3.2.2. Runtime modeling in robotic systems

To the best of this thesis' author's knowledge, research that explicitly focuses on models@run.time in robotic systems is still scarce. There are systems, which use a form of models@run.time to realize self-adaptivity, like MROS [13], however, their focus lies on modeling architectural variants, while the runtime model of RuMROS focuses on representing the state of a system of mobile robots. For similar purposes, failure monitoring [70] and runtime verification [36] solutions have been explored, which appear to use runtime models, but they lack focus on the modeling approach itself. A system similar to RuMROS, that aims to explicitly provide runtime modeling capabilities is GRuM [124], as not only does in include a runtime model, but it also offers additional infrastructure for human monitoring, as RuMROS does. Unfortunately, their model only represents the system's state and thus does not qualify as a runtime model in the sense of models@run.time, as the causal connections between the model and the system are only one-way. With respect to providing an AST-based runtime model in the spirit of the models@run.time paradigm for controlling and monitoring a MRS, RuMROS should be unique to the best of the author's knowledge.

## 3.3. Robotics: Current frameworks and hybrid multi-robot systems

Over the past two decades, a variety of software solutions for building robotic applications have been developed [16, 73, 78], which are still maintained and in use today [95]. This section briefly introduces these general middleware frameworks for robot control, while the following sections deal with concrete solutions for facilitating the creation of MRSs specifically. Finally, novel approaches to simulation of MRSs in recent years are compared to what RuMROS offers.

In the domain of service robotics, use of robotic frameworks in companies and real-world applications has been surveyed in 2020 [41]. It was found that ROS is the most widely used framework by far (88.5%), followed by ROS 2 [73] and OROCOS [16] around the 20% mark and finally YARP [78] and SmartSoft [110] at around 5% usage. In the industry, ROS 2 is used about four times less than ROS because most companies did not feel the need to upgrade and change their running systems so far. Both however offer extensive support for existing robotic systems, have mature development and management toolchains and provide the means to organize a system in a modular way. OROCOS [16] also strives to be modular by organizing applications in a component-oriented way. However, it is aimed at real-time applications and does not include built-in simulation capabilities. YARP [78] does not offer robotic algorithms like ROS, ROS 2 and OROCOS do, but encourages distributed systems through the use of components with ports and connections, described with an Interface Definition Language (IDL). SmartSoft [110] makes use of a component model which is centered around communication patterns. It features loose coupling of components, allowing for dynamic extensibility and reconfiguration at runtime, which is not an explicit concern of the previously mentioned systems.

SmartSoft can be seen as an implementation of the abstract model-driven robotics development framework RobMoSys [95], which is a project funded by the European Union to kickstart a more general industrial platform for robotics [111]. The RobMoSys[5] methodology centers around model-driven development of robotic applications. For this purpose, they suggest a set of industrial-grade development and modeling tools, including a set of Papyrus-based[6] modeling DSLs for instance. In addition to this, they provide standard roles[7] and architectural patterns[8] for the development process.

This thesis builds on some of the core principles of RobMoSys, since runtime modeling is used. RuMROS* aims to implement behavior modules, allowing for separation of behavioral composition from the implementation of concrete behaviors. This in turn emphasizes the separation of RobMoSys roles like behavior developer and system architect. Some of the architectural patterns, like communication patterns for instance, are also passively employed throughout RuMROS*, as ROS 2 is used, which already provides robust implementations for them. The choice for the middleware framework falls on ROS 2 as it is open-source, actively developed and offers a more extensive ecosystem of tools than its competitors, including built-in simulation. It is chosen over the industry standard ROS, as the node-management and communication infrastructure it provides is much more mature for distributed systems, compared to its predecessor. Since ROS 2 does not specifically implement dedicated modeling features, and there is no complete implementation of RobMoSys to this day, we propose a custom, dedicated runtime modeling system built on top of ROS 2 as a standalone component. Making the runtime model compatible with standard middleware is an important step to ensure a greater degree of applicability to different systems and domains. Some robotic applications have already taken this step [58, 98], and are discussed, among others, in the following sections.

[5] The RobMoSys wiki can be found at `https://www.robmosys.eu/wiki/`.

[6] More information on the Papyrus modeling languages can be found at `https://gitlab.eclipse.org/eclipse/papyrus/org.eclipse.papyrus-robotics/-/wikis/home`.

[7] `https://www.robmosys.eu/wiki/general_principles:ecosystem:roles`

[8] `https://www.robmosys.eu/wiki/general_principles:architectural_patterns:start`

### 3.3.1. Frameworks for multi-robot systems

In MRSs, robotics middleware has additional challenges to address. ROS 2 provides communication infrastructure via topics, but in decentralized systems, specialized network organization structures, like Peer-To-Peer networking (P2P) are often desired [53]. The development of MRSs itself also brings new hurdles in comparison to to single-robot systems. On the basis of communicating (this may also be implicit), organizational structures may need to be formed and algorithms that implement the behavior of the system are required in turn. In the following sections, existing frameworks for the development of MRSs and their main capabilities are presented. Following this, approaches to implementing runtime modeling in a MRS are compared to RuMROS*. Finally, current hybrid MRSs are explored, in order to identify gaps in research, which this system may help to cover. An overview of the systems covered in this section, including RuMROS*, is given in Table 3.1.

Table 3.1.: Overview of relevant multi-robot frameworks

| **Framework** | **System Type** | **Architecture** | **Approach** | **Middleware** |
|---|---|---|---|---|
| ROS2Swarm [58] | Decentralized | Layered (movement, collision avoidance) | Bottom-up (behavior primitive library) | ROS 2 |
| jSwarm [44] | Centralized | Service-oriented with dynamic task scheduling | Mixed (top-down constraints + primitives) | None |
| Buzz [98] | Decentralized | Distributed with situated communication | Mixed (top-down swarm primitives + single-robot primitives) | ROS |
| HiSARM [59] | Hybrid | Layered and HFSM-based, uses static model transformations | Mixed (top-down composition + primitives) | None |
| RuMROS* | Hybrid | Modular, aspect-oriented, with runtime model | Mixed (top-down composition + primitives) | ROS 2 |

For decentralized systems in particular, there are two general approaches to designing them with self-adaptiveness in mind, in accordance with Diaconescu's idea of a holon's dual nature [26], as described in Section 2.3.2: top-down and bottom-up [51]. When employing a top-down approach, the system is designed starting from a global perspective. From this view, finer substructures are iteratively derived until all system units are described with sufficient detail. In contrast, bottom-up approaches focus on the design of the individual robots first and foremost. They do not program the system from a global perspective, but rather design individual robot behavior in such a way, that more complex, global behavior emerges from

it, when multiple robots running the local behavior are deployed. The framework developed in this thesis employs a mixed approach, with focus on bottom-up behavioral primitives, while task allocation at runtime is performed in a top-down manner. Therefore, behavioral primitives of bottom-up systems, as well as mixed systems are examined in the following sections.

### Bottom-up approaches

From the perspective of autonomic computing, where individual agents operate with a certain degree of autonomy, [61], decentralized MRSs are composed of individual agents that each execute their own behavior. This is especially apparent in swarm systems, in which there may be no explicit communication at all between robots. Decentralized behavioral patterns have been implemented for robotic systems with focus on foraging [101], although this behavior is not supported by the prototype developed in this thesis. An implementation of more general collective behavioral primitives is provided by *ROS2Swarm* [58]. It is designed for mobile robots that operate with ROS 2 and provides several standard behaviors, e.g. for aggregation, dispersion, flocking and more, which were briefly discussed in Section 2.3.2. These behavioral primitives can be composed and are executed by each individual robot separately. The framework uses a layered architecture, with movement patterns on the outer layer and logic to prevent collisions on the inner layer. Both subscribe to ROS 2 topics to receive sensor data, most prominently from a Light Detection And Ranging (LiDAR) sensor. When a movement pattern is active, it calculates a drive command, which is handed to the inner layer, that checks against the LiDAR data and determines whether an object is too close or not. If it detects a possible collision, it instead calculates a drive command around the object, otherwise it passes on the movement command to ROS 2. The implementation of patterns themselves is extensible, as all patterns share a common superclass, which provides the implementation structure. The behaviors themselves are based on a number of algorithm families depending on the concrete pattern [23], but they all have in common that their objective is to achieve emergent behavior.

A comparison of implementation approaches and algorithms of behaviors is beyond the scope of this thesis, but an overview of the algorithms used in RuMROS* is given in Chapter 4. Architecturally, it also provides a library of behaviors for general mobile robots, which is extensible by design and implemented with ROS 2. It is less comprehensive than the functionality that ROS2Swarm provides and does not offer a hardware protection layer like ROS2Swarm does, as it is intended to be a proof of concept for applicability in the field of swarm robotics. Extensibility however is an important factor for actually being used in real-world applications, therefore the framework developed in this thesis instead includes a layer for better hardware abstraction. This allows sensors, which collect similar data, like an array of circularly arranged distance sensors and a rotating LiDAR to be used interchangeably with the provided behaviors, making it more flexible to use with hardware that differs slightly from what the behaviors were designed for. ROS2Swarm also uses a set of static configuration files for behaviors, thus not allowing behaviors to change at runtime. RuMROS in contrast offers a more dynamic approach to its ROS 2 nodes, offering dynamically exchangeable, parametric behavior primitives.

### Mixed approaches

Among the approaches Hrabia et al. analyzed [51], jSwarm [44] stands out, as it is designed for swarm systems while featuring a central controller, similar to the framework proposed in this thesis. In contrast to RuMROS* however, which implements behavioral primitives in ROS 2 nodes and subjects them to composition in the runtime model, their approach uses a service-oriented architecture. Each node offers local services, according to its hardware, which can be called by a global scheduler, that is part of the central controller. Instead of having a separate entity for centralized services like RuMROS* does, they select one of the existing entities of the system to provide them, via communication and a leader-selection algorithm. All of this is enabled by the networking infrastructure, which allows nodes (robots) to discover each other and offered services. jSwarm is a mixed approach, since nodes offer very basic behavior, like movement to a position. In contrast to pure bottom-up approaches however, it dynamically schedules tasks in a top-down manner, like RuMROS*.

In order to run a robotic application, services that are capable of implementing the given code have to be selected. This is done by an analyzer in the distributed services component, which constructs a dependency graph of all actions performed by the application. In addition to code, programmers can also specify separate logical constraints, allowing for specification of when and where an action should be performed. These constraints are included in the dependency graph, which is then given to a scheduler, that plans the actions both in space and time. It also takes system constraints into account, including sensor and actuator inaccuracies. Finally, the derived schedule is assigned to individual execution units, which perform the actions as scheduled, by receiving appropriate commands.

RuMROS* does not dynamically assign robots to groups based on tasks. Instead, organizational units are primary objects of modeling, explicitly specified in the model. Dynamic organization is realized by dynamic group allocation, as well as dynamic task allocation. Additionally, RuMROS* takes a different approach to coordination of robots, as its specification of constraints on behavior (e.g. distance adjustment between robots) is limited to adapting parameters of the given behavioral primitives and their composition. In exchange, it aims to provide better guidance in designing composite behavior, as the existing primitive operations available to programmers are more complex than those of jSwarm.

jSwarm models system aspects at runtime, in order to dynamically pick robots to perform a task. However, the model is not explicit or separated from code. RuMROS* on the other hand explicitly models the current group structure and available robots and tasks, which in turn allows it to offer a management interface for human monitoring, in contrast to jSwarm. In jSwarm, the local services each node provides are also custom made, not benefiting from well maintained and widely supported hardware abstraction middleware solutions like ROS 2 for robot control.

Another mixed approach is Buzz [98], which deals with swarm programming. It is an interpreted extension language, that directly runs on robots via the Buzz Virtual machine (BVM). By design, the BVM is loosely coupled to sensors and actuators, allowing it to be adapted to existing robotic frameworks, like ROS[9] [90].

Like the solution developed in this thesis, Buzz treats swarms as first class objects but

[9] An existing implementation of the BVM for ROS can be found at `https://github.com/MISTLab/ROSBuzz`.

furthermore supports set operators like union, intersection or difference. This enables tasks to be assigned on a per-swarm basis. RuMROS* also allows for the definition and dynamic modification of swarm groups. In addition to this, it supports hierarchical organization of swarms, allowing not only a per-swarm task allocation, but also nested allocation to groups of swarms (i.e. to branches in the swarm hierarchy). Both systems are designed to support heterogeneous swarms, although the authors of Buzz do not specify whether a swarm itself can contain different robots. RuMROS* supports groups of heterogeneous robots to a limited extent, i.e. as long as they offer similar sensor types and capabilities that support the assigned behavior. This is enabled by the additional sensor abstraction performed on the ROS 2 layer, allowing similar robots to execute the same tasks in a heterogeneous group.

In terms of communication, Buzz offers uses a distributed ad-hoc network, that builds on situated communication [119], allowing robots to communicate within line-of-sight. It also includes distributed, local key-value storage for information exchange. RuMROS* on the other hand directly makes use of the networking capabilities that ROS 2 offers by overlaying a bidirectional central message bus with generic message types for arbitrary communication between both robots and the runtime model. Buzz uses the network in place of stigmergy, while RuMROS* implements behavior passively, via neighbor-detection. The robot-to-robot communication it offers however, is intended for the implementation of more complex tasks or generally, systems with centralized communication, which is beyond the scope of this work and not implemented in the prototype.

Listing 3.1: Exemplary use of Buzz for swarm group assignment and task allocation, taken from [98]

```
# Group identifiers
AERIAL = 1
GROUND = 2
GRIPPERS = 4
# Task for aerial robots
function aerial_task() { ... }
# Task for ground-based gripper robots
function ground_gripper_task() { ... }
# Create swarm of robots with fly_to symbol
aerial = swarm.create(AERIAL)
aerial.select(fly_to)
# Create swarm of robots with set_wheels symbol
ground = swarm.create(GROUND)
ground.select(set_wheels)
# Create swarm of robots with grip symbol
grippers = swarm.create(GRIPPERS)
grippers.select(grip)
# Assign task to aerial robots aerial.exec(aerial_task)
# Assign task to ground-based gripper robots
ground_grippers = swarm.intersection(GROUND + GRIPPERS, ground, grippers)
          ground_grippers.exec(ground_gripper_task)
```

The Buzz language is JavaScript-like and offers both functional and imperative constructs for group management, neighbor interaction, communication and virtual stigmergy. The example they provide for group management can be found in Listing 3.1. With JastAdd, RuMROS* can also manage groups at runtime, start and stop per-group tasks and perform coordination of tasks in time, i.e. task composition. The exemplary runtime model of the prototype does not use neighbor interaction or virtual stigmergy, but as a programming framework allows the provided communication channels to be used to implement similar

behavior. Buzz supports modularity via the specification of classes, modules and functions. RuMROS* extends on this, by additionally including aspect-oriented features. On top of this, the explicit runtime modeling approach provides a more comprehensive and central way to capture runtime status information, extensible and supporting human introspection.

*HiSARM* [59] is one of the latest mixed approaches, and explicitly targets development of hybrid MRS. Its main goal is to ease the development process of multi-robot applications, by allowing the specification of tasks on a high level and in a robust way. This is supported by automatic code generation, communication and hardware abstraction as well as a modular architecture with behavioral primitives for composition.

The framework the authors propose consists of three layers and builds on a model transformation workflow. On the upper level, programmers write a high-level definition of the application, which includes team formation, possible robot states and transitions, as well as services. Services are defined by composing primitive actions the robots can perform, which are provided by a database on the lowest layer. Their exemplary definition of a fire response scenario is shown in Listing 3.2. To support code generation, which is performed by the middle layer in a sequence of steps, the database also includes simulation information, robot information and more. The behaviors (services) used in the high-level description are composed of basic behavioral primitives, which are part of a library of algorithms on the lowest layer. This layer also contains a database with robot specifics, simulation and architecture information, as well as mappings from algorithms to low-level robot code, thus abstracting from hardware and facilitating extension with new robots.

Listing 3.2: Fire response example in HiSARM's high-level DSL, taken from [59]

```
# Team Formation
{
  OnSiteResponse: ePuck searcher[7]
  EvacuationSupport: p3dx supporter[3]
}
# Transition Definition
Main.OnSiteResponse {
  case(Wait):
    catch(FIRE): mode = Divide
    catch(REMOTE): mode = RemoteControl
  case(RemoteControl):
    catch(AUTO): mode = Wait
  case(Divide):
    catch(AISLE_ARRIVED): mode = Exit
  default: mode = Wait
}
ModeTransition.Distribute(TARGET) {
  case(Move):
    catch(ROOM_ARRIVED): mode = RoomCheck
  case(RoomCheck):
    catch(SEARCHED): mode = MoveToAisle
  default: mode = Move(TARGET)
}
ModeTransition.Search {
  default: mode = Search
}
ModeTransition.Extinguish {
  default: mode = Extinguish
}
...

# Mode Definition
Mode.Divide {
  group(Room1, min=2):
    ModeTransition.Distribute("ROOM1")
  group(Room2, min=3):
    ModeTransition.Distribute("ROOM2")
  group(Room3, min=2):
    ModeTransition.Distribute("ROOM3")
services:
  MonitorEnvironment
}
Mode.RoomCheck {
  group(Search, min=1):
    ModeTransition.Search
  group(Extinguish, min=1):
    ModeTransition.Extinguish
  ...
}
Mode.Search(AREA) {
  services:
    SearchItem(AREA)
}
Mode.Move(TARGET) {
  services:
    MoveToTarget(TARGET)
    MonitorEnvironment
}
```

```
Mode.Wait { ... }
Mode.Extinguish { ... }
Mode.Exit { ... }
Mode.MoveToAisle { ... }
...

# Service Definition
MoveToTarget(TARGET_POSITION) {
  result = moveTo(TARGET_POSITION)
  if (result == "SUCCESS")
    throw ROOM_ARRIVED
} repeat(5 SEC)
SearchItem(ROOM) {
  result = searchItem(ROOM)
  if (result == "SUCCESS")
    throw SEARCHED broadcast
    } repeat(5 SEC)
...
```

The middle layer transforms the high-level code into a set of task graphs (one per robot), which utilizes the low-level algorithms declared in the high-level specification. It has inputs and outputs connected to ports, from which the communication structure is generated, as well as a generated controller, which composes the algorithms according to the high-level specification. The controller is based on a HFSM and implements the specified states and transitions. From this, the robot specifics of the database are utilized to eventually generate machine code for each robot type, which is then distributed across all robots.

RuMROS* also uses a HFSM-based representation for global system behavior, but in contrast to HiSARM, represents system structure with an AST-based runtime model. Furthermore, HiSARM structurally supports hierarchical groups and offers a library of behavioral primitives which depends on sensor types, abstracting from hardware like RuMROS* does. While their system is in a more advanced development stage than the prototype presented in this thesis, RuMROS* has two distinct advantages by design: runtime modeling and its ROS 2-based architecture. HiSARM uses HFSMs, but they are statically compiled into code at runtime, thus losing the benefits of a dedicated runtime model, such as human introspection, which RuMROS* provides. Since HiSARM is not based on one of the widely used robotic middlewares, it also can't make use of existing toolchains for analyzing the system's logical structure and communication, which ROS 2 enables. With their custom implementation of communication, integration into existing systems that use common middleware may also be more difficult.

### 3.3.2. Simulation and exploration of scenarios

Between testing and formal verification approaches, simulation is one of the most widely used techniques to design the behavior of MRSs or evaluate a system's hardware requirements [84]. In this work, simulation is also the tool of choice to experiment with the system and explore combinations of basic behaviors. The choice of simulator for the system, ARGoS 3 [99], has already been motivated in Chapter 1. In this section, the focus lies on selected approaches that offer features to aid the design of a system by automating or enhancing simulation as well as how this work compares to them.

Focusing on providing a system to assess robustness of a MRS, Harbin et al. proposed the ATLAS framework [48]. It provides a DSL for specification of functional and non-functional requirements of the mission, as well as architecture, behaviors and capabilities of robots. Code generation is then used to derive the three main components of the system — algorithmic templates for the implementation of behavior, interfaces that reflect the specified system structure and build on robotic frameworks like ROS for execution and simulation, and middleware for communication between both parts. The algorithmic templates provide

a framework for developers to implement concrete high-level behaviors, while the custom middleware enables insights into performance of the system by recording metrics related to goals specified in the DSL.

In comparison to RuMROS*, ATLAS has a similar architecture, which employs a DSL with code generation and builds on systems running ROS. However, this work focuses on providing a runtime modeling system for MRSs, which provides more flexibility, as the modeling language can be structurally extended by developers by altering the RAG. ATLAS on the other hand focuses on design space exploration, by providing language features like explicit variation groups for different system components. This allows their system to use model transformations to generate (and via certain algorithms select the most promising) concrete, executable mission specifications that employ different variants, while conforming to constraints on components or environment. In RuMROS*, both the specification of system variants and their execution in simulation have to be done manually by developers. However, since its modeling language is specified by JastAdd and support for modeling simulation scenarios is already supported in the current system, introducing these concepts may be feasible in future work. In terms of capturing metrics however, RuMROS* has the advantage of offering a human monitoring and control interface for observing live data. Since it is based on a runtime model, it also offers more comprehensive ways of interacting with or further processing captured data. While ATLAS permits logging and leaves further computations up to the developers, RuMROS can compute higher-level metrics directly at runtime.

A different approach to exploring various system configurations is the recently developed language Scenic [39]. It specializes on providing language features for specifying probabilistic distributions over objects and agents in space, as well as agents' behavior over time, subsequently allowing to derive concrete simulation scenarios by sampling from the distribution. Like ATLAS and RuMROS*, it supports Gazebo[10] for simulation, as well as other simulators like CARLA [29] for more specific use cases, like autonomic driving. Scenic offers language constructs that facilitate the modeling of different types of scenarios: generic ("a car on the road"), structured ("a badly-parked car"), fuzzy ("a car near 1.2 x 4m") and concrete ("a car at 1.2 x 4m") [39]. This reduces the amount of code required for each type, easing development effort. For simulation, RuMROS* uses ARGoS [99] and integrates it with the JastAdd runtime model, allowing for specification of certain simulation scenario features like environmental objects, robot placement, robot types and more. This allows developers to leverage the capabilities of ARGoS, without having to work with its configuration files. Since both RuMROS* and ARGoS are not focused on system variants, there are no dedicated configuration distribution sampling mechanisms. However, ARGoS does offer introducing noise into sensor data to make the simulation more realistic, as well as allowing for objects and robots in the scene to be stochastically distributed. While offering such functionality, it is not as sophisticated as Scenic's implementation, which explicitly focuses on this aspect. In contrast to Scenic, since ARGoS is configuration-based, modeling the spectrum of scenarios is not covered by additional keywords or operators in the DSL. Instead, it makes use of templates, providing an exemplary, extensible library of scenarios, which can be loaded and modified in the modeling language.

[10] More information on Scenic and its currently supported simulators can be found at `https://scenic-lang.org/` and `https://docs.scenic-lang.org/en/latest/simulators.html`.

Listing 3.3: An exemplary car scenario defined with the Scenic language, adapted from [39]. Some definitions, like for the *Pedestrian* class are omitted, if they do not illustrate relevant language features.

```
class Car:
    position: Point on road
    heading: roadDirection at self.position

behavior FollowLaneBehavior():
    while True:
        throttle, steering = ... # compute controls
        take SetThrottleAction(throttle),
                SetSteerAction(steering)

behavior AvoidPedestrian():
    threshold = Range(4, 7)
    while True:
        if self.distanceToClosest(Pedestrian) <
                threshold:
            strength = TruncatedNormal(0.8, 0.02,
                    0.5, 1)
            take SetBrakeAction(strength),
                    SetThrottleAction(0)
        else:
            take SetThrottleAction(0.5),
                    SetBrakeAction(0)

scenario CarOnSide(gap=0.25):
    precondition: ego.laneGroup._shoulder != None
    setup:
        spot = OrientedPoint on visible
                ego.laneGroup.curb
        parkedCar = Car left of spot by gap, with
                behavior AvoidPedestrian(20)

scenario Main():
    setup:
        model scenic.simulators.gta.model
        ego = Car with behavior FollowLaneBehavior
        compose:
            do CarOnSide(), CarOnSide(0.5)
```

The syntax of Scenic is shown in Listing 3.3. While reminiscent of Python, it also includes classes for distributions as well as special keywords to specify the used simulator, agent behaviors, constraints, composable scenarios, operators for various offsets or distances and many more not shown in the example. Their approach to composing behaviors differs from RuMROS*, which uses a HFSM to model complex system behavior, whereas Scenic composes behaviors in an imperative way. Since RuMROS* uses JastAdd, which allows general Java code to be executed, this is also possible in the prototype framework, but offering a more structured way to model behavior has the advantage of allowing for visualization and human introspection of the model. When it comes to assigning behaviors, RuMROS* offers more features related to MRSs, as behaviors do not have to be individually assigned to each robot, but the general approach centered around parametric adjustment for individualizing preset behaviors is similar to Scenic.

# 4. Implementation of the proposed system

In this work, RuMROS*, a hybrid multi-robot runtime modeling framework for mobile MRSs based on RuMROS [117] is proposed. This chapter provides an overview of its layered structure, as well as more detailed insights into the most important components and design considerations for the hybrid MRS extension implemented in this thesis, with respect to the concepts and systems introduced in chapters 2 and 3, respectively.

## 4.1. System overview

An overview of the Top-Level Architecture (TLA) is given in Figure 4.1. It follows the already layered structure of RuMROS, although some components have been modified and several have been added to meet the goals outlined in Section 1.2. As previously explained in Section 1.3, RuMROS already partly implements solutions for few-robot systems, like enabling robust runtime modeling ($G_1$, $G_2$) and human introspection ($G_4$). In these aspects, the existing parts were modified to include concepts of hybrid MRS and are covered in the following sections. To achieve the goals that RuMROS does not partially solve, like structurally supporting hybrid MRS ($G_7$), dynamic reconfiguration, supporting SAS features ($G_3$) and many-robot simulation ($G_8$, $G_9$), new components have been added.

In addition to the already existing ROS package for simulation with Gazebo, a ROS 2 workspace, which enables the simulation of hybrid MRSs has been created on the ROS layer. It consists of multiple packages, mainly a generic ROS node, which comes with several basic behaviors for decentralized MRSs and a package that bridges ARGoS [99] to ROS 2. All packages are explained in detail in Section 4.2.

On the runtime modeling layer, the model has been extended to support specification of simulation scenarios, as well as to support MRS concepts like groups, communication infrastructure and more. The new *Behavior Composition* module enables performing actions based on observations programmatically. This way, automated and self-adaptive behavior can be specified within the runtime model, more on this in Section 4.3.4. RuMROS in contrast only supported performing actions via the human monitoring GUI, which has been

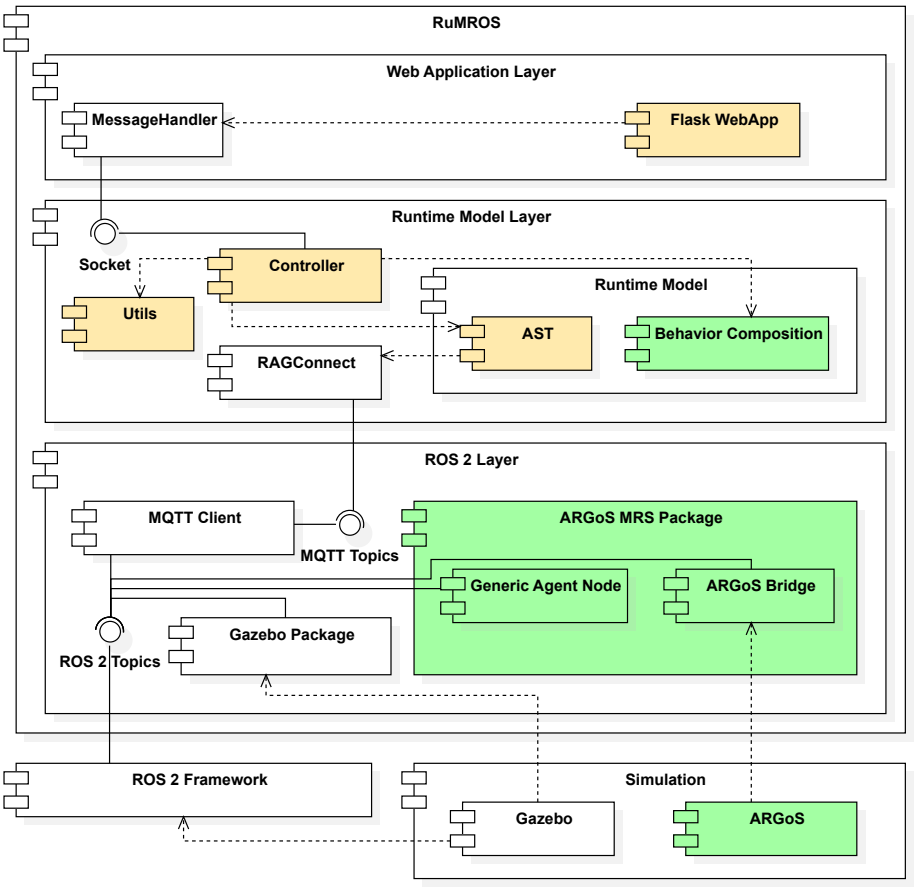


Figure 4.1.: The top-level architecture of RuMROS*. Components that were added to the existing framework, RuMROS, are marked green, while modified components are marked orange.

adjusted to display MRS concepts in a better way (Section 4.4).

## 4.2. ROS layer

The Gazebo workspace contains an exemplary simulation scenario, which includes Turtlebots,[1] RaspRovers[2] and a built-in generic drone. It is fixed and part of the already existing framework, RuMROS. In the following sections, the focus lies on the workspace for the multi-robot extension performed in this thesis, which contains the following main packages:

- `argos3-ros2-bridge` - A set of ROS nodes that bridge the ARGoS simulator to ROS 2
- `swarm_controller` - A node that is designed to run on each robot, offering behavioral primitives and a communication interface with the runtime model

[1] `https://emanual.robotis.com/docs/en/platform/turtlebot3/overview/`
[2] `https://www.waveshare.com/wiki/RaspRover`

- `rumros_msgs` - Constains all custom message definitions of the system

In addition to these packages, which are described in the following sections, the workspace also contains `simple_launch` and `lidar_vis`. `simple_launch`[3] is a helper package that simplifies launch code, facilitating the initialization of a potentially large number of ROS nodes during the startup sequence, when simulating on a single machine. `lidar_vis` is an exemplary custom utility package, designed to help with development and debugging by visualizing LiDAR data of a robot in a real-time graph.

### 4.2.1. Generic agent node

The runtime modeling example provided by RuMROS, as previously mentioned, supports Turtlebots, Rovers and a Drone. Due to time constraints, the MRS extension for simulation with ARGoS only supports two robot types — Turtlebot (version 3, model *Burger*[4]) and Foot-Bot[5] — in order to demonstrate the framework's capability of modeling heterogeneous systems. Robot type, as well as unique robot identifier (ID) and group ID are required parameters to initialize the robot node. The runtime model, which sets up the simulation scenario and manages the system, creates configuration files that contain robot IDs associated with group IDs and types during startup, subsequently allowing a ROS launch script to initialize every robot node according to the specified setup automatically. From the ID of the robot and its type, topic strings are derived for initializing the communication with other robots or the runtime model via ROS topics. Currently, the node has publishers and subscribers for actuators and sensors respectively, as well as three main communication channels — `/swarm<swarm_id>/cmd_behavior` for switching a group's behavior, `/rumros/cmd_ctrl` for control commands addressed to individual robots and `/rumros/cmd_ack` for sending packets in the opposite direction, mainly acknowledgments. The way communication is organized is described in more detail in Section 4.3.3.

Apart from asynchronous communication callbacks, the main controller of the robot node itself is designed as an internally and externally switchable state machine, with the presently activated behavior and its parameters representing the current state. Per default, an idle state is selected, which does not process any sensor data or send actuator commands to the robot. While running, the node calls a `tick()` function at a specified interval, which in turn calls a similar function on the currently selected behavior object to obtain a ROS *Twist* message, used to actuate the differential drive of the robot. Therefore, the node is generic with respect to the robot type, as long as its drive actuator can be steered by Twist messages. For other types of drive actuators, developers have to alter the tick function in the node and behaviors to output different message types based on the robot type, which is easily possible, as the node is written in Python, a dynamically typed language.

Since the main controller of the node treats the currently selected behavior as part of the state machine's state and calls functions on it accordingly, it makes use of the gang-of-four (GOF) State pattern [40]. The node also contains a library of behavior patterns and a sensor abstraction layer, which are both designed to extensively utilize further patterns to make

[3] https://github.com/oKermorgant/simple_launch
[4] https://emanual.robotis.com/docs/en/platform/turtlebot3/features/
[5] https://dgarzonramos.github.io/robotics101/fbot/

the node as extensible as possible.

### Extensible behavioral patterns

RuMROS* provides the following basic parametric movement patterns:

- *Idle* - The robot does nothing.
- *Drive* - The robot moves according to the provided linear and angular velocities.
- *Moving* - The robot, given a pose sensor, navigates to a target position and orients itself in a target orientation.
- *Random Walk* - The robot traverses the environment in a random pattern, adjustable with parameters.
- *Dispersion* - The robot, given a LiDAR-like sensor and part of a group or swarm, diffuses in the environment, avoiding obstacles and other robots.
- *Flocking* - The robot, given a LiDAR-like sensor and a pose sensor, organizes itself into a formation with other robots in the area, characterized by attractive and repulsive forces. The robots then jointly move towards a target location.

The organizational structure of the provided movement patterns is hierarchic, similar to ROS2Swarm [58], although the exemplary behavior is limited to movement-related patterns in RuMROS*. The current implementation only provides one layer, with all behaviors sharing a superclass that defines the interface. However, since each behavior is implemented in its own class with a uniformly defined interface, behaviors can be chained to create more complex behaviors. As in the context of runtime modeling, the composition of behaviors is intended to be done in the runtime model, complex chained behaviors are not currently implemented in the ROS node. Regarding the categorization of patterns by Beni [8], referenced in Section 2.3.2, the implemented patterns cover the categories *pattern formation* and *complex patterns*. Regarding *tasks related to an environmental entity*, the authors of ARGoS provide examples[6] that show how behaviors like foraging can be implemented purely in the simulator, but due to time constraints, this has not yet been adapted to RuMROS* in this work.

The interface itself allows subclassing behaviors to define a heterogeneous list of sensors, from which data can be retrieved by provided methods. Internally, it makes use of the GOF Observer pattern, in order to register tick methods for all sensors, allowing data to be retrieved asynchronously. The hierarchical structure also has the benefit of reducing effort for supporting new sensor or actuator types. In order to extend a behavior with a new sensor type beyond the currently supported pose and LiDAR-like sensors for instance, the respective methods have to be added and registered in the behavior superclass only, after which every concrete behavior may make use of the provided data if it chooses to. Concrete behaviors are implemented in subclasses via the TemplateMethod pattern, allowing each subclass to override a hook that is called in the superclass. This fixes the output data

[6] `https://github.com/ilpincy/argos3-examples`

interface and allows behaviors to be started and stopped gracefully by an external actor via the superclass interface. Finally, the superclass provides GOF Observer functionality to its subclasses, allowing concrete behaviors to register and notify a list of callback methods on different channels. This enables reactions to events, like notifying the runtime model, once moving to a selected position is reached for instance.

### Sensor abstraction

The sensor abstraction layer defines an abstract `Sensor` class, which abstracts from the interface ROS 2 provides, as concrete sensor subclasses are initialized in the main controller and handed over to concrete behaviors. This way, behaviors can deal with data directly, by retrieving it via a simple callback. It implements GOF Observer behavior, such that subclassing sensors may also notify subscribing classes when new sensor data arrives for more complex use cases. From the `Sensor` class, concrete sensors may be derived directly, that can hold sensor data and perform computations on it. Its main purpose however, is to derive intermediate sensor type classes, from which concrete, hardware-specific sensors can be derived in turn. The hardware-specific classes can then implement the GOF Adapter pattern to convert the data into a standard format defined by the intermediate type class. RuMROS* provides exemplary adapters for LiDAR-like sensors, which each adapt the data from an array of distance sensors of the Foot-Bot and the LiDAR sensor of the Turtlebot 3 to a common interface. This allows behaviors to receive sensor data of the consolidated format, eliminating the need to adjust each behavior for every new robot type. Instead, a single adapter has to be written for a given sensor type in order to use a new sensor of an existing type. Hardware parameters can be incorporated in the adapters, enabling constraint checks in behaviors, for instance to test whether a given sensor has enough range to sense obstacles within the specified perimeter.

### Behavior implementation example: Flocking

As it is the most complex behavior currently implemented in the framework, the flocking algorithm shall serve as an example on how emergent behavior can be written with a bottom-up approach. Like the patterns described in Section 3.1.3, it is based on force fields, implementing a version of algorithm 2 by Olfati-Saber [89]. On a high level, it balances three interaction terms among agents: separation, cohesion, and alignment. Each robot only uses local sensing with LiDAR-like sensors (no explicit communication), such that global coordination emerges purely from local interactions with other agents. This is a basic implementation, for more advanced use in the field, Olfati-Saber also includes an algorithm that can differentiate between obstacles and other agents via additional communication or sensing.

Simplified Python code for the implementation of the algorithm as used in RuMROS* is available in the appendix, Listing A.1. It senses neighbors using a LiDAR-like sensor, which delivers data via the sensor abstraction layer as a point cloud in local two-dimensional space, with polar coordinates. To derive relative neighbor positions, coordinates are converted to cartesian, followed by a robust, density-based clustering [37]. After this, the three main forces are computed.

Separation and cohesion (spacing force): Each agent enforces a minimum inter-agent distance by computing a repulsive force, if neighbors come closer than a prescribed spacing. Conversely, if neighbors are farther away but still within sensing radius, an attractive component pulls the agent back toward the group. This is realized through a spring-like potential function, linear in distance offset, where equilibrium is reached at the defined spacing parameter.

Alignment: To maintain formation without collisions, each agent estimates the velocity of neighbors by comparing consecutive cluster positions over time. The required data structures for this can be held by the implementing behavior class, such that the `tick()` method may access them, since it itself is called periodically and cannot hold any data. It then computes a correction force such that its velocity tends to match the average neighbor velocity. To smooth out robot movement and and avoid oscillations, the velocity estimate is dampened with an exponential smoothing term.

In addition to the three main components between agents, the algorithm includes a heading force, in order to guide the entire flock towards a common goal. This is derived from the vector between the agents current position (as obtained by a pose sensor) and the global target location, converted into the local robot coordinate system. This ensures that despite decentralized interaction rules, the group progresses toward the desired destination with each step.

Finally, the overall control input is computed as a superposition of spacing, alignment and heading forces, weighted by parameters. The result is then clipped to a given maximum velocity. Finally, the force vector is converted into a ROS Twist command suitable for differential drive robots, ensuring that the physical platform can follow the computed trajectory.

### 4.2.2. Simulation and ARGoS bridge

For simulation, both ARGoS and Gazebo can be used, although Gazebo is meant for few-robot systems, due to the previously mentioned performance constraints. In the exemplary runtime model provided by RuMROS*, Gazebo is used to simulate a centralized, few-robot MRS in a warehouse scenario with multiple areas. For systems with larger numbers of robots (a concrete quantization of this is determined in Chapter 5), ARGoS should be used instead. This allows developers to either leverage the extensive support for modeling scenarios, areas, ROS-supported robots and more that Gazebo offers, or work with a more performant simulation solution for large MRSs, albeit with a more limited feature set compared to Gazebo. The scenario specification of ARGoS is directly integrated into the runtime model (more on this in Section 4.3.2). Compared to Gazebo, which has a fixed scenario definition that has to be updated manually, this has the advantage of facilitating iteration of scenarios during development. In the following section, an overview of how the bridge package[7] is implemented, in order to make ARGoS work with RuMROS* and ROS 2, is given.

The ARGoS configuration may include up to three types of external extension libraries: robot controllers (one per robot or shared), loop functions and QT[8] user functions. A robot

[7]The bridge package is based on existing code from CPS Konstanz that bridges the Foot-Bot to ROS 2. Their code can be found at `https://github.com/CPS-Konstanz/argos3-ros2-bridge`.

[8]QT is an open-source GUI framework, see `https://github.com/qt/qt5` for more details.

controller defines the behavior of the robot and is loaded from a shared library written in C++. Loop functions and QT user functions are loaded from different C++ libraries and are used for continuously running functions during the simulation and GUI drawing functions respectively. In order to bridge ARGoS to ROS 2, the ROS 2 bridge package compiles a number of different libraries — one controller for each robot type and one each for loop and QT user functions. Each controller converts simulated sensor readings to ROS 2 messages and incoming messages to performing actuator actions. Since sensors and actuators differ for each robot type, the controllers are separate and in order to support new robot types, an additional controller has to be added. To facilitate extensibility, the package includes a common superclass for controllers that provides all methods for the lifecycle of an ARGoS controller and performs the setup for basic sensors and actuators of mobile robots, like positioning and differential drive. In the configuration file, controllers are specified once and then linked to an arbitrary number of robots, such that each simulated robot has its own node, similarly to how Gazebo handles its nodes.

The loop functions interface of ARGoS allow for arbitrary code execution during simulation and can be used to call various functions ARGoS provides, e.g. to draw on the floor of the simulated arena. The creators of the system also provide examples,[9] showcasing their use, keeping track of resources in a foraging scenario and allowing robots to pick up virtual items on the floor for instance. The provided exemplary runtime model of RuMROS* makes use of areas, which are drawn on the floor via loop functions. The corresponding `loop_functions` node in the XML configuration is used to pass custom parameters for them as well as QT user functions. Drawing on the simulation window itself is also supported by ARGoS, in world space, on specific entities or in screen space. This is used in the example provided with RuMROS* to optionally draw robot and corresponding group names over each robot in the scene. Since the whole package has access to ROS 2, it can actively observe a ROS 2 topic for group changes to update the visualization accordingly.

## 4.3. Runtime modeling layer

The runtime modeling layer consists of the runtime model specification in JastAdd [50], a RAGConnect specification for ROS 2 connections and a Java application that contains the main controller as well as utility packages. The latter are mostly used for build tasks related to ARGoS, required for supporting the specification of ARGoS simulation scenarios within JastAdd, as well as automated configuration of the `mqtt-client` ROS package, derived from the setup in the runtime model.

As mentioned in Section 2.2.3, the specification of the runtime model has a structural part — the RelRAG — and a semantic part — the `JRAG` and `JADD` aspect extensions native to JastAdd. When compiling, the RAGConnect and RelRAG specifications are preprocessed into a RAG with extensions to resolve noncontainment relations. Subsequently, the resulting artifacts as well as all native JastAdd aspects written by programmers using the system are handed to the JastAdd compiler, which generates the final AST classes in Java. The generated code does not have to be altered manually and can directly be used, therefore the process works in accordance with the MDD paradigm, where development work is predom-

[9] `https://github.com/ilpincy/argos3-examples`

inantly done on models.

Since the system is designed to be used with either ARGoS or Gazebo, a build flag can be set to build for either simulator. While the RelRAG as well as general aspects are shared, extension aspects can be specified to only be included in the build for a specific simulator, by prefixing their filenames with *Argos* or *Gazebo*. As communication methods may also differ, the RAGConnect configuration is fully separated.

The built AST classes are used in the Java controller, which constitutes the entry point of the runtime model application component. When launched, the controller creates the root node of the model and initializes it by calling an `init` method specified within the model. It also loads a JSON configuration file for connections to `mqtt-client` and associates the given MQTT topics with the appropriate RAGConnect methods generated by the compiler, initializing the connections. While running, the controller periodically serializes the model to JSON, which is fully generic and independent of specified classes, as JastAdd internally uses Java's *Serializable* interface.[10] The serialized model is then then sent to the web application layer for display, via a UDP socket. The same socket is also used to receive messages from the web application in order to update the model when actions are performed. These updates in turn trigger RAGConnect to send messages to the ROS 2 layer which then performs them on the actual robot or in simulation. This mechanism implements the bidirectional *causal connections* in the sense of the models@run.time paradigm [10], while also adding an additional layer for human introspection on top, which abstracts from the lower level concepts. Adaptive behavior of robots at runtime can also be specified, this process is described in more detail in Section 4.3.4.

The controller is, for the most part, generic in the sense that it is independent of the specification of the majority of the AST's structure. In order for the basic components of RuMROS* to work however, it enforces some structural constraints. The root AST node must be `Model`, which has to contain a list of `Action`s that can be performed by the web UI with defined `Input`s and `Result`s. In addition to this, in order to support functionality like sending commands to individual robots to adjust the group structure or await results of `Action`s for instance, a `CommandManager` node and `CommandMsg` structure are required.

In the following sections, the most important parts of the structural, semantic and communication specification of RuMROS*'s runtime model AST are described in detail. Finally, the approach to modeling behavior and self-adaptation is explained.

### 4.3.1. Structural specification

The part of the structural specification purely related to the runtime model of the MRS itself is shown in Listing 4.1. For the most part, these AST node specifications are unchanged compared to the existing model (Listing 2.4) in order to keep the few-robot example intact. In the exemplary scenario of Gazebo, robots, as well as the drone can maneuver around a warehouse, which is modeled using different rectangular `Area`s. These, in addition to robots, which are represented by a `PositionReference`, `Velocity` and `State`, were direct child nodes of the `Model` node. In order to change the behavior of robots via the web UI, the RelRAG also contains a list of `Action`s with customizable `Input`s and displayed `Result`s.

[10] `https://docs.oracle.com/javase/8/docs/api/java/io/Serializable.html`

Listing 4.1: The exemplary partial RelRAG of RuMROS*'s meta-model. Communication-related parts are omitted, as they are described in detail in Section 4.3.3.

```
Model ::=
    SwarmGroup*
    Area*
    State*
    SwarmUnit*
    PositionReference*
    Action*
    CommandManager
    PendingResult:AwaitableResult*;

SwarmGroup ::=
    <id:int>
    <name:String>
    /CurrentState:State/;

abstract SwarmUnit ::=
    <name:String>
    <id:int>
    /SendCommand:CommandMsg/;

abstract Robot : SwarmUnit ::=
    /Velocity2D/;

rel SwarmGroup.SwarmUnit* -> SwarmUnit;
rel SwarmGroup.AggregatedState -> State;
rel SwarmGroup.SwarmGroup* -> SwarmGroup;

rel SwarmUnit.CurrentState -> State;
rel SwarmUnit.TargetPosition? -> PositionReference;

Turtlebot : Robot ::=
    PosePositionReference;
Footbot : Robot ::=
    PosePositionReference;
Rover : Robot ::=
    OdomPositionReference;

Drone : SwarmUnit ::=
    OdomPositionReference
    /Velocity3D/;

Position ::=
    <x:Double>
    <y:Double>
    <z:Double>
    <theta:Double>;

abstract PositionReference ::=
    /Position/;

PosePositionReference : PositionReference::=
    Position;

OdomPositionReference : PositionReference ::=
    Position;

DefaultPositionReference : PositionReference::=
    Position;

Velocity2D ::=
    <speed:Double>
    <rotationSpeed:Double>;

Velocity3D ::=
    <horizontalSpeed:Double>
    <verticalSpeed:Double>
    <rotationSpeed:Double>;

Area ::=
    <id:int>
    <name:String>
    <xPos1:Double>
    <yPos1:Double>
    <xPos2:Double>
    <yPos2:Double>;

State ::=
    <id:int>
    <name:String>
    <speedFactor:Double>
    <angleTolerance:Double>
    Parameter*;

Action ::=
    <id:String>
    <label:String>
    Input*
    Result;

Input ::=
    <id:String>
    <label:String>
    <type:String>
    <defaultValue:String>;

Result ::=
    <type:ActionResultType>
    <message:String>
    <displayMillis:int>;

abstract Parameter ::= <name:String>;

StringParameter : Parameter ::=
    <value:String>;

IntParameter : Parameter ::=
    <value:Integer>;

DoubleParameter : Parameter ::=
    <value:Double>;

CommandManager ::= ...;
```

In order to allow direct access to data structures of the runtime model and avoid unnecessary traversal of the tree, the root node directly holds them. The native support for noncontainment relations, introduced by Mey et al. [79], facilitates this, especially for nodes like `State`, which logically belong to `Robot`s and would thus naturally be nested and scattered further down the AST. This also allows for reusing the same objects without duplicating them in different sub-trees, simply by referencing them:

```
rel SwarmUnit.CurrentState -> State;
rel SwarmGroup.AggregatedState -> State;
```

For supporting larger MRS, organizational structures were introduced. In RuMROS*, `Robot`s are `SwarmUnit`s that are organized into `SwarmGroups`. The whole system may then contain any number of these groups. This allows behavior to be adapted for multiple robots at once, facilitating organization and handling of larger MRS. This is further enhanced by the relation `rel SwarmGroup.SwarmGroup* -> SwarmGroup;` which allows groups to be nested, enabling hierarchical structures, as often present in natural swarms [115] and holonic systems [26].

The RelRAG extensively uses noncontainment relations to reduce the AST's complexity and enable modeling of structures that would be cumbersome to implement without them. Another important modeling feature of JastAdd is support for *nonterminal attributes*, which can be specified with slashes: `/CurrentState:State/;` . Such an attribute is fully synthesized by a method specified in `JRAG` files, allowing complex nonterminals to be computed according to a specific algorithm. Architecturally, they can be used in conjunction with RAGConnect to specify the structure (interface) of communication endpoints explicitly within the RelRAG, leaving implementation up to the semantic part as well as the RAGConnect specification. This works for both receiving and sending ports and is used in the provided exemplary runtime model to send state changes and individual robot commands to, as well as receive the position of robots from the ROS 2 layer. The concrete design of these endpoints is up to developers, which is important for supporting hybrid systems. The provided meta-model illustrates how to compute and send individual actuation commands, like `Velocity2D` in the Gazebo simulation part for this purpose. The concepts of decentralized or distributed MRS on the other hand are represented by ports that send parametrized commands based on group `State`, on the basis of which local behavior is executed on the ROS 2 layer when compiling for ARGoS. In hybrid systems, these concepts can be mixed. The provided meta-model facilitates this by also including communication endpoints (`PendingResult`, `SendCommand`) for sending structured commands to individual robots.

### 4.3.2. Semantic specifications

Every semantic specification in JastAdd has to be implemented inside a named aspect. How code related to certain concerns is distributed among aspects is fully up to developers, as JastAdd does not enforce any constraints with respect to which parts of the RelRAG may be part of the specification in a particular aspect or not. In the provided exemplary runtime model, for illustrative purposes, there are both aspects that group code related to certain structures of the model, like `Action` or `Position` implementations and cross-cutting aspects related to entire extensions. For Gazebo, the *DroneLoitering* aspect for instance implements a functional extension that implements orbiting behavior for the drone in a single file.

Within aspects, Java code, including support for arbitrary libraries, is used for programming. Fully native Java methods can be specified in `JADD` files, but computations on the AST model are performed predominantly on attributes of each node type specified in the RelRAG. Synthesized attributes compute values bottom-up from a sub-tree of the AST (including noncontainment references). This is sufficient for most node-intrinsic computations, as child nodes and references required for them are, by design, a part of the parent node that performs them. However, since the model holds the main data structures as top-level nonterminal nodes, inherited attributes are essential to propagate information downwards in the AST, so that nodes performing computations on a lower level may access them. These main concepts are illustrated by a simplified version of the `State` aspect, which implements lookup by name in the model and makes it available in `SwarmUnit` and `SwarmGroup`:

```
aspect State {
    syn State Model.getStateWithName(String name) {
        return java.util.stream.StreamSupport.stream(
            this.getStateList().spliterator(), false)
            .filter(s -> s.getname().equals(name))
            .findFirst().get();
    }

    inh State SwarmGroup.getStateWithName(String n);
    eq Model.getSwarmGroup(int i).getStateWithName(String n) {
        return this.getStateWithName(n);
    }

    inh State SwarmUnit.getStateWithName(String n);
    eq Model.getSwarmUnit(int i).getStateWithName(String n) {
        return this.getStateWithName(n);
    }
}
```

### Implementation of behavior

The entry point to the JastAdd implementation of the runtime model is the `Model.init()` method, called by the controller. It creates all objects of the runtime model and assigns them a starting state, depending on the scenario specified by the developer. In the provided model, this includes `Area`s, `Robot`s and `SwarmGroup`s, communication-related nodes and the data structures for the web interface: `Action`s and `Input`s. From this, the ARGoS simulation scenario is partially derived, this is described in Section 4.3.2. Once created, actions and inputs are transmitted to the UI by the controller for display. Once an action with certain parameters is selected by an operator, a method corresponding to the name of the action is called via reflection in the controller. These callback methods can be supplied in a JastAdd `JADD` extension file and the provided exemplary model includes several for simulation with both ARGoS and Gazebo.

The approach of RuMROS* to actions means that arbitrary Java or JastAdd methods can be called from the UI, easily extensible via hook methods or overrides in another JastAdd

aspect. Without further extensions, the callbacks currently modify data structures of the model, setting a robot's `State` for instance. When RAGConnect is configured to monitor a concrete node or node class of the AST, the calls to setters are injected with hooks, allowing RAGConnect to notice the change and send messages to ROS 2 nodes accordingly. This is the case for MRSs simulated with ARGoS, sending state updates with parameters to set behavior on a given group of ROS 2 nodes. Directly steering the robots is implemented with Gazebo, in this case an action still triggers the state change, but RAGConnect is configured to send Twist commands, generated from a synthesized nonterminal attribute implementation method in `RobotImpl.jrag`.

While it is up to developers to design the concrete implementation of behavior for few-robot systems, the provided example gives some guidance, implementing robot behavior as a state machine, similar to a system steered by a MAPE-K loop. Sensor data of ROS 2 nodes can be connected to AST endpoints, allowing it to be analyzed. The given model receives the pose of robots to compute movement commands to reach a target destination when a "drive to area" command is executed for instance. Therefore, it includes all parts of a MAPE-K control loop, with knowledge being provided by the AST itself.

The provided actions are different for ARGoS and Gazebo, since their capabilities and purposes differ. With ARGoS, larger MRSs are meant to be simulated, therefore the model does not send robot actuation commands directly. Instead, for better performance and supporting decentralized MRS, the state of a `SwarmGroup`, as well as parameters are sent to the corresponding group of ROS 2 nodes. To support more fine-grained control, the runtime model can also directly address each robot individually, which is required for changing the group of a robot at runtime for instance. These communication structures are described in Section 4.3.3.

### Scenario specification and ARGoS Integration

In a `JRAG` file, the initialization of concrete objects from AST node classes related to real-world objects or concepts, like robots, areas and groups takes place, from here on referred to as *model initialization*. This is always part of the scenario specification, whereas the second part, which is concerned with the simulation setup for ARGoS, is omitted when compiling for Gazebo. AST node instances are created like normal Java objects, as they are compiled to Java, with their components, specified in the RelRAG, as constructor parameters. After actions and states are created according to the functionality of the ROS 2 layer or runtime model (depending on whether the framework is used to directly interact with robot actuators or to make use of local behavior), areas with associated coordinates are created and added to the model. Then a group hierarchy is created. Each group is given a unique identifier and name, and groups can be added as sub-groups to other groups, allowing nesting. Finally, the `Turtlebots` and `Foot-Bots` are created, given an initial behavior, placed at a starting position and assigned to groups. Since groups define the behavior a robot has, it can only be part of one group at a time.

The second part of the scenario specification is a native Java extension in a `JADD` file, since it is only executed during a pre-processing step in the startup phase of the framework. It makes use of functionality inside the Java utility package of the runtime modeling layer to automate the creation of configuration files for mqtt-client and ROS 2 launch, as well as

facilitate the creation of an ARGoS configuration file. Both mqtt-client and ROS 2 launch need to be aware of the number of robots of each type, as well as the number of groups and robot-group mappings. The former are specified as static variables in the `Model` node with this aspect, whereas group mappings are derived from the model initialization scenario.

As mentioned in Section 4.2.2, ARGoS loads extensions from user-provided libraries. In the case of RuMROS*, robots of a common type share the same type of controller, which is provided by the ARGoS bridge on the ROS 2 layer, connecting to ROS 2 topics in order to send sensor data and receive actuation messages. The provided exemplary model automatically adds XML configuration nodes for the robots at their given positions as specified in the model initialization scenario, with the controller(s) from the bridge. ARGoS also allows for custom configuration entries that can be used to pass data to the controllers, loop or QT user functions. This is used to add the areas specified in the model initialization, which are then drawn on the floor object by the loop functions. There is also the option to add further parameters, allowing robot and group names to be drawn above them in the simulation if desired. This is implemented with a generic interface in the utility package, such that more parameters and functionality can be added to the bridge without having to change the parameter setters in the runtime model. Finally, objects can be added to the arena. ARGoS natively supports basic objects like cylinders and boxes, for which the utility package provides appropriate placement functions. Placement can be done in two ways: manually specifying each object at a specified position, or distributing a given number of objects in a specified space with a single distribute XML node. Both ways also work for placing robots, although in the distribution case, the initial positions of robots as specified in the runtime model are overridden by the placement that ARGoS picks from the distribution. The initial positions will be set in the runtime model until the first pose messages are received from the simulated ROS 2 nodes, updating them with the ones given by ARGoS.

A full configuration may also include further configuration nodes for camera placement, physics engines, and simulation parameters like tick speed. In order to make working with different scenarios more comfortable, RuMROS* also allows an XML configuration template to be specified within the `ArgosInit` aspect. It is used as a basis, within which the programmatically specified configuration nodes are embedded. This allows for creating a library of scenarios with different objects, parameters and camera angles already set up in the arena, easing programming effort. RuMROS* provides some basic scenarios with its exemplary runtime model for reference.

### 4.3.3. Communication specification

To connect the runtime model to the ROS 2 layer, RAGConnect is used exclusively. It requires two sets of configuration files, separate for compiling for Gazebo and ARGoS: A JSON configuration for class-topic mappings and a configuration in the RAGConnect DSL for concrete AST attribute mappings. The JSON configuration contains a list of inputs and outputs, associating AST classes to MQTT topics with message types. This is required to initialize RAGConnect in the runtime model controller. The configuration also allows for the specification of wildcards in topics for object identifiers, like robot or group IDs, as well as constraints with respect to the environment of the node, only making connections if the node has a specific parent node for instance.

The RAGConnect DSL specification on the other hand specifies the concrete attributes which are received and sent, commonly referred to as *ports*. The provided exemplary model receives the pose of robots and sends either the current group state when compiled for ARGoS or robot movement commands when compiled for Gazebo. Additionally, commands for individual robots can be sent and received on a bidirectional message bus, which is implemented directly in JastAdd as an overlay topology over ROS 2 topics. This can be used to address individual robots instead of groups to change the group of a robot or move to a position for instance. Similar structures can also be used for modeling a hybrid MRS with communication between robots, allowing for flexible topology.

The concrete specification of a RAGConnect port varies, depending on its type (differing for list attributes for instance) and most importantly, context.[11] To receive the position of a robot in all `Position` objects throughout the AST, a context-free receive port may be specified with `receive Position;`. However, depending on sensor type, some robots may use odometry to estimate the pose while others use different means to obtain a more precise position. Therefore, context is required and the runtime model differentiates between them, reflected in the connection specification: `receive PosePositionReference.Position using PoseToPosition;` (analogous for `OdomPositionReference`). In addition to the context, this statement makes use of a RAGConnect mapping specification, allowing to define a function *PoseToPosition*, which is used to convert the received data to a `Position` object. This is necessary for all non-basic types, and in this example scenario, allows converting a received ROS 2 pose message to a position that can then be used to update a corresponding `Position` node of a robot in the model (and similarly for ROS 2 odometry messages). In practice, more complex message types like ROS 2 messages, have to be received as byte arrays: `PoseToPosition maps byte[] b to Position {: ... :}`. The framework includes auxiliary functions within the utility package to convert from and to ROS 2 messages for the used message types, and further ones may be implemented there by developers.

### 4.3.4. Composition of behaviors

Section 2.1.3 introduced HFSMs and BTs as key mechanisms for task switching on an agent. The behavior model implemented in this work needs to primarily deal with groups of agents, but also individual robots for group switching for instance. Since the model should also be visualized on the web UI, a BTs would be preferable to HFSMs, further supported by JastAdd, which natively deals with trees and thus facilitates implementation of such structures. However, the required coordination between robots poses a significant challenge for an implementation with behavior trees, as they are designed to be independent for each agent. A group waiting for a unit to switch or two groups moving to a rendezvous point after both completing a task would not be possible to implement with just BTs, as they don't allow for dependencies on other trees natively. It would be possible to implement a single behavior tree, which commands the whole application, but this would make it monolithic, partly taking away from the advantages of modularity and good visualization that BTs would otherwise have over HFSMs. Another option would be to implement a blackboard,[12] which

[11] An overview of the specification options of RAGConnect can be found at `https://jastadd.pages.st.inf.tu-dresden.de/ragconnect//dsl/`.

[12] Blackboards are a key-value storage that all nodes of a BT can access. They are commonly used in BT libraries such as BehaviorTree.CPP (`https://www.behaviortree.dev/docs/tutorial-basics/tutorial_`

stores global knowledge across all group trees for instance. The drawback of this approach is that it also introduces arbitrary links, as these variables may be set by one group's tree and used in conditional checks of another group's tree. These references are implicit, making traceability as or even more difficult than with HFSMs. For the reasons outlined above and due to time constraints, the current version of the behavior modeling system compromises between BTs and FSMs by utilizing a modified HFSMs-based approach for the behavior model. This implementation additionally allows state transitions to multiple states. Since each state represents a group behavior, not the system's global behavior, this is required to express parallel execution. For modeling hierarchical MRSs, it also provides a higher level of expressiveness than basic BTs.

The HFSM implementation if fully done with JastAdd, extending the RelRAG by an additional `BehaviorModel` node which keeps track of the current time and running state and maintains a list of top-level model nodes, active nodes and startup nodes. Nodes can either be composite nodes or terminal (leaf) nodes, from here on referred to *Composite Behavior* (`CB`) and *Basic Behavior* (`BB`) respectively. `CB`s in particular can be nested within each other, allowing the creation of hierarchical structures. They manage how contained `BB`s are executed (unless sub-blocks are directly linked to by other parts of the model). By default, they execute contained behaviors in a sequence, inspired by a BT's sequence nodes. Alternatively, they can be set to execute all contained blocks at the same time when activated.

Every behavior node also has a set of conditional transitions linking it to the next nodes. When a behavior has multiple transitions, they can be set to all fire on their respective conditions or in a mutually exclusive way — only firing the first transition that evaluates to true, similarly to how *alternatives* work in behavior trees. The current implementation provides three exemplary sub-types of transitions: timed transitions which fire either after a certain time elapses or at a specific point in time and custom conditional transitions, which fire when an arbitrary predicate evaluates to true. These types can be extended directly in the RelRAG or by providing helper classes for custom predicate transitions. The provided exemplary model has a set of auxiliary functions that can fire transitions on finishing an awaitable action, behavior block and model or environment state like area occupancy for instance. This allows for the implementation of self-adaptive behavior, with respect to all parameters the runtime model has access to as well as further computed ones. More information on the concrete scenario of the provided model can be found in Section 4.4.

When the model is executed at runtime, either automatically on launch or manually from the web interface, the startup `CB`s are executed first, recursively evaluating contained behaviors according to their mode (parallel or sequential) until reaching leaf `BB`s. These blocks reuse `Action`s already specified in the runtime model, contain parameters to customize them and target them at a group or unit. There are no rules enforcing a `CB` to only contain behavior for a certain group or unit, but it is advised to use `CB`s to group behavior in either a semantically coherent (i.e. grouping by functionality) or structurally coherent (i.e. grouping by controlled groups) manner to keep the specification more readable and manageable. Depending on how the transitions are set up, the model continues execution indefinitely when containing loops or stop when reaching a stop behavior, which is usually linked to the *idle* action.

`02_basic_ports/`) or frameworks like Unreal Engine (`https://dev.epicgames.com/documentation/en-us/unreal-engine/behavior-tree-in-unreal-engine---quick-start-guide`).

## 4.4. Human monitoring layer

The monitoring layer, or web GUI retains the functionality of RuMROS — viewing tables of runtime model data structures and executing actions. In addition to this, as both actions and tables can grow substantially in number, separate views for both of them have been added. The main contribution of this work, however, is the interface for the behavior model, shown in Figure 4.2. It is not as flexible as the mostly generic existing part of the UI, since it has to rely on specific concepts like nodes and transitions of certain types, but still allows a more narrow degree of extensibility by utilizing superclasses of behavior and transition types where possible. The current prototype displays a node diagram of the concrete HFSM-like behavior model specification and allows the model to be started and stopped. The graph display itself uses Cytoscape.js[13] for visualization, which allows for automatic arrangement of nodes, as well as panning, zooming and node repositioning. It is a robust library that could also allow for displaying additional information or triggering behavior on selecting nodes in future extensions.

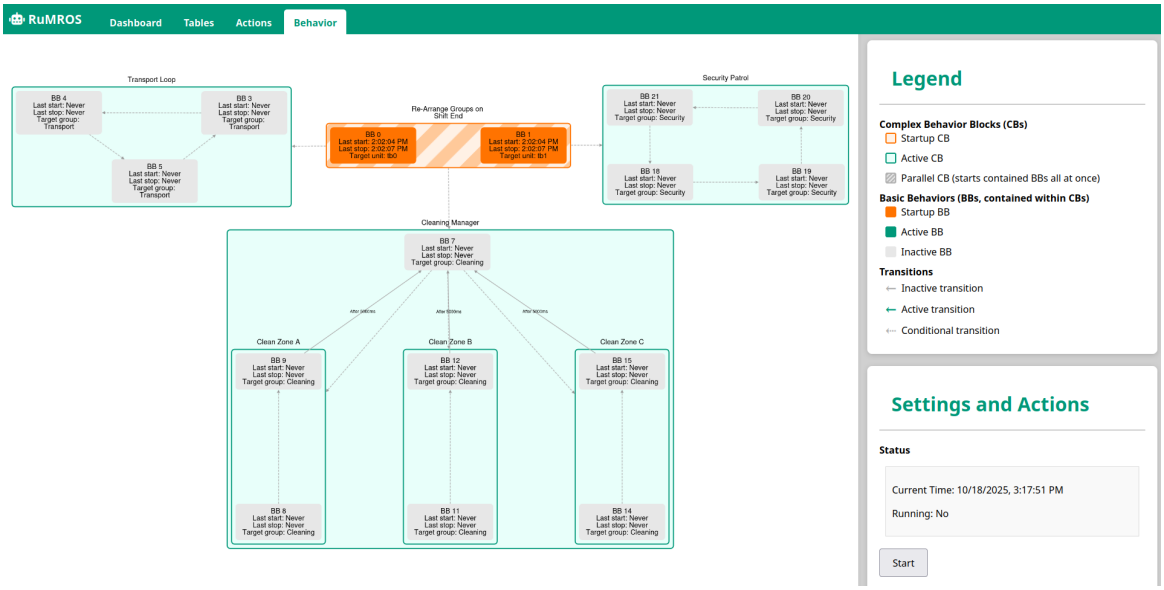


Figure 4.2.: The exemplary behavior model of RuMROS* as shown on the web GUI while idle

The exemplary model describes a fictional warehouse scenario, where robots are divided into groups related to transport of goods, cleaning and security. It showcases the current organizational features of the model, but does not implement actual transportation behavior robots may have, like picking up goods for instance, as RuMROS* is currently limited to to movement actuators within the scope of this thesis. With orange, startup behaviors are marked, here the first two contained leaf behaviors cause two robots to change their group from transport to cleaning and security respectively, when rush hour ends and less transportation robots are required. They start in parallel, as their parent `CB` is configured as such. It uses custom conditional transitions to activate the cleaner and security group

[13] Cytoscape.js is an MIT-licensed graph theory library for analysis and visualization of arbitrary graphs. More information can be found at `https://js.cytoscape.org/`.

behaviors once all contained leaf behavior blocks are finished. Following this, cleaner and transport robots, including the additional two robots that changed groups, start their behavior, which is a sequence of movement or flocking commands, switching behaviors once robots reach a certain area. This way, complex movement patterns can be implemented: transport robots move to designated warehouse areas sequentially while security robots patrol the perimeter. The GUI shows all currently active `BB`s as green boxes, while inactive blocks are grey. The cleaning robots also utilize the positional data of robots, which the runtime model has access to, however, in contrast to transport and security robots, they also perform computations on the position of the transport group in order to ensure they move to a warehouse that is currently vacant. This is modeled via the `CB`s "Clean Zone A-C", which consist of a block that moves them to the respective area, stops and then returns control after a short time (simulating cleaning) to the outer manager `CB`. The manager itself utilizes mutually exclusive conditional transitions to pick a vacant area, starting the corresponding cleaning behavior. In order to avoid cluttering the UI, conditional transitions are displayed as dashed, but do not have additional information attached to them, as predicates can get arbitrarily complex depending on the use case. This is a drawback of using a HFSM-like structure versus behavior trees in terms of visualization complexity. On the flip side, even custom conditional transitions are explicit and directly reflected in the structure of the model, whereas visualizing dependencies in behavior trees that make use of blackboards do not have this information explicitly available.

## 4.5. Summary

Regarding the definition of a Self-Adaptive System, outlined in Section 2.1.2, RuMROS* forms an understanding of the system under management and the environment by specifying explicit connections to robot sensors. By performing further computations on them, higher level knowledge is built, which is made available in the runtime model. While no learning mechanisms are implemented, it is possible to apply reasoning within the limited scope of its specification, by employing conditional reactions to internal and external events. Therefore, the provided exemplary model works on level 1 of Neisser's awareness scale [85], although this can be extended up to level 3 with the current architecture, due to it's support for reflection, and is up to developers to implement for their respective system. Goal awareness on the other hand is implicitly encoded in the logic and would require further linking to development artifacts in the sense of MDD. The adaptation mechanism is designed to be explicit and due to the surrounding HFSM-like model more structured than imperative programming, following Petrovska's suggestion [97]. However, she also mentions explicit goal modeling, which should be subject to future work.

In terms of the MRS taxonomy conceived in Figure 2.5, the framework is heterogeneous with respect to robot types, works according to the models@run.time paradigm and is reconfigurable in a limited (coordinated) manner. Its control architecture is hybrid, while the interaction structure can vary according to implementation of the concrete system, allowing for all types of structures, although only implicit, neighbor-sensing interaction has been tested in the prototype. This also influences the degree of interaction according to Parker [93], which on the ROS level is collective if robots do not sense other robots, cooperative if they do so and fully collaborative if they also share information. Due to the use of a

runtime model, which adjusts behavior according to information from all robots however, on a high level, the system is always collaborative, or at least coordinative, if groups have different objectives instead of sharing a common goal. The degree of autonomy also depends on implementation details. As there is no learning component but the system can react to given scenarios and react accordingly, the degree of autonomy that is possible to model with RuMROS* arguably lies between intermittently autonomous and eventually autonomous.

# 5. Evaluation of the concept

In this chapter, evaluation objectives are outlined, from which methods are derived and applied to determine how the prototype system performs, with respect to said objectives. Finally, results of the evaluation and derived potential adjustments are presented. All evaluations are conducted in simulation, as it is a central aspect of the implementation performed in this work.

## 5.1. Objectives

On a high level, the key goals of the system developed in this thesis are enabling developers to program, simulate and monitor MRS. With respect to development, important aspects like extensibility and modularity, as well as possible limitations and advantages have been discussed in comparison to other approaches in Chapter 3. In this chapter, focus lies on evaluating the feasibility of simulating large MRS with RuMROS* empirically, with respect to performance and scalability, as this is a basic requirement for making the system usable in real-world scenarios that require many robots to work together. Therefore, the main goals of this chapter are determining application performance by capturing system and application metrics, as well as finding scalability limits of the system. Following these goals, research questions are formulated as follows:

1. $RQ_1$: Given certain constraints with respect to computations on ROS nodes, messaging and group assignment, how many robots can a typical consumer machine (desktop and mobile x64-based architecture) simulate using the prototype framework?
2. $RQ_2$: What are the limiting factors for system scalability?
3. $RQ_3$: Following the observations, how can scalability and performance be improved and which system components are targets for further optimizations?

## 5.2. Methods

In this section, the evaluation approach is described in detail, with respect to previously outlined objectives and research questions. This includes the test scenario, used hardware, captured metrics and post-processing of data.

### 5.2.1. Test scenario and constraints

The test scenario is based on the exemplary warehouse scenario given in Section 4.4. Robots are assigned to three groups, each arranged in a grid and consisting of an equal number of Turtlebots. For the scalability evaluation ($RQ_1$), the total number of robots, $N_r$, is increased in increments of six (adding two robots to each group) per test run, starting with 15 robots. An automated launch script is used to launch all system components and start the test scenario specified in the behavior model after a certain delay $d_{launch}$, which is adjusted to the required launch times of the given scenario and machine. As previously described, two robots change groups at first, while all others are idle. Once this step is completed, all groups start their respective behavior: two groups move with obstacle avoidance and one group moves towards a destination in a formation. Therefore, all groups use LiDAR and Pose sensors, with their data being provided by the simulator, after the initial group switch stage. The speed of all robots is fixed throughout testing to ensure that execution times only vary with simulation speed. The behavior model is set to run its update loop every 10 milliseconds, the web UI updates values once per second and ROS nodes are set to compute on a timer every $t_{node} = 100$ milliseconds. The simulator, ARGoS 3 [99], is set to a speed of 10 ticks per second, although the actual speed may slow down depending on system load. The delay between ticks in milliseconds will be referred to as $t_{argos}$ throughout the evaluation process.

### 5.2.2. Test method and hardware

After the behavior model is started, the simulation runs the given test scenario for five minutes before terminating. From launch to finish, CPU core utilization and memory utilization are captured once per second and per sub-process of each process spawned by the system components. In addition to this, $t_{argos}$ and start and stop events for behaviors are logged by the runtime model application and ARGoS respectively, in order to detect potential stalls. The test process is conducted on a mobile machine equipped with an Intel Core i7-8565U CPU and 16GB of RAM running Ubuntu 24.04.2 LTS, as well as a desktop machine with an AMD Ryzen 7 3700X CPU and 32GB of RAM running Ubuntu 24.04.3 LTS with matching ROS package versions. The mobile machine uses integrated graphics while the desktop machine has a dedicated GPU, but GPU load was found to be negligible during testing, and is therefore not part of the captured metrics to keep measuring overhead minimal. All data is additionally post-processed to obtain median values for each metric and system process per test run, so they can be compared for multiple runs with different $N_r$. As the main comparison metric, median was chosen over mean, as it represents the average throughout the duration of a test run better due to initial spikes in load during the launch process.

After conducting the performance evaluation with per-process and component-specific met-

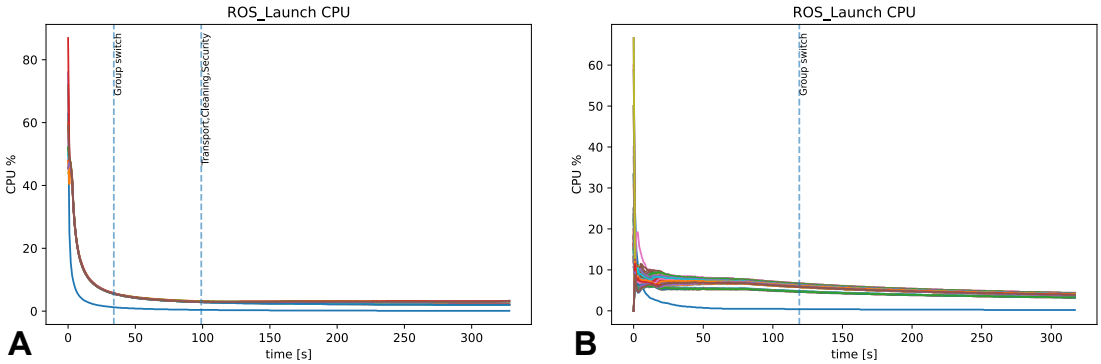


Figure 5.1.: Initial CPU load spike of ROS nodes during launch for a) $N_r = 15$ and b) $N_r = 105$ robots on the mobile machine. Each line represents a sub-process in the ROS launch process, most of which are robot nodes. Dashed vertical lines show at which point in time the runtime model starts robot and group behaviors.

rics, the test scenario is evaluated again, without capturing these metrics for $N_r$, and in increments of 30, starting at 15. During this, *ros2trace*[1] is used to determine ROS and DDS middleware metrics, like messaging delays and individual callback durations, helping to identify potential performance bottlenecks.

Finally, in order to empirically compare the performance of the runtime model to other approaches, callback latencies for the most frequently received message type, `Pose` messages, are collected across different runs, using Java's native high precision time function `System.nanoTime()`. However, as the results showed hardly any variation of note with increasing $N_r$, they are shown in Figure A.1 in Chapter A and briefly discussed in Chapter 6.

## 5.3. Results

During testing, the system exhibited a load spike during the setup process, as ROS nodes have to be initialized and both ARGoS and the runtime model have to create ROS and MQTT topic publishers and subscriptions, respectively. The ROS launch process is by far the most significant factor when many robots are simulated, this is shown in Figure 5.1.

Initially, the CPU spikes briefly, as the launch script is spawning sub-processes for each node. For 15 robots, initialization finishes for all nodes within ten seconds, while taking considerably longer for 105 robots, which is why the start of the initial behavior had to be delayed significantly, from $d_{launch} = 30$s to $d_{launch} = 120$s. This happens because all robot node processes require 5-10% of a CPU core during launch — that is a theoretical maximum of 1050% or just over ten fully utilized cores with perfect scheduling at $N_r = 105$ — causing the CPU to be overloaded and ROS nodes to be initialized very slowly. On the desktop machine, CPU usage also converges towards 5% once all nodes are initialized,

[1] `https://github.com/ros2/ros2_tracing`

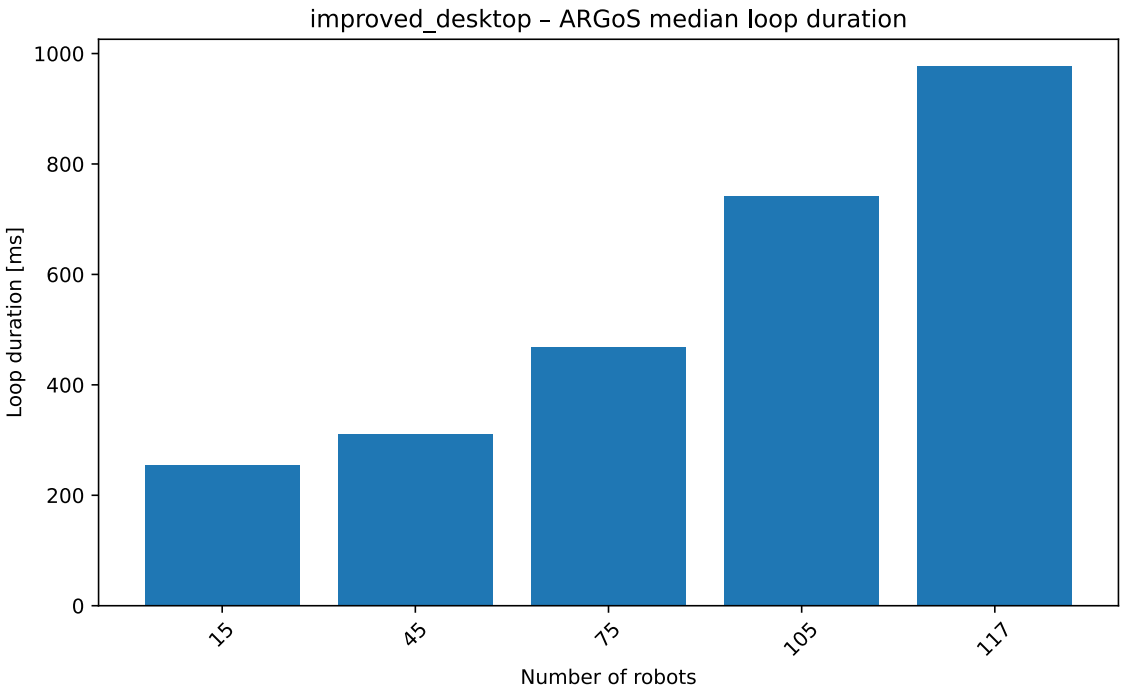


Figure 5.2.: Median duration of computing one time step of the simulation in ARGoS 3 on the desktop machine per test run with different numbers of robots ($N_r$)

though significantly quicker due to having twice the number of cores of the mobile machine as well as more powerful individual cores, and as such does not require the behavior model to start with additional delay.

Despite much better hardware specifications, even the much more powerful desktop machine was only able to handle $N_r = 117$ robots, after which some robots started missing command messages with the initial parameters. Increasing the delay between ROS $t_{node}$ loop executions to 500 milliseconds helped alleviate this issue without sacrificing accuracy, as ARGoS loop execution times ($t_{argos}$), shown in Figure 5.2, exceed this value for higher robot counts. Compared to the baseline with 15 robots, a simulation with 45 robots runs at 70%, declining to 38% at 75 robots and further to 21% of the original speed at 105 robots on the mobile machine (values rounded to nearest percent). As a result of the increasingly long loop durations, simulation speed drops, as is apparent in Figure 5.1: with 15 robots, the behavior model easily gets to the stage where all robots start their behaviors after two robots switch groups and move to their target group. At $N_r = 45$, this stage is delayed by over 50 seconds and at $N_r = 75$ by almost 3 minutes.

An additional test run with $N_r = 145$ robots was conducted in an attempt to extrapolate the previously found utilization values beyond what is considered to be doable with the current implementation of the bridge between ARGoS and ROS, but caused the communication between the runtime model and web interface to fail, as the current implementation sends the whole model as a single UDP message to the web UI. With large model sizes, i.e. high numbers of available behaviors, nodes in the behavior model, robots, groups or all in junction, the maximum transmission unit of the UDP socket is exceeded and errors occur.

This is merely a limitation of the prototype system due to time constraints and can be fixed by splitting the model into multiple packets. Since both machines that ran the tests encountered issues beyond $N_r = 117$ robots, this has no influence on the results of the scalability evaluation.

Figure 5.3 shows the total median system load of RuMROS* for different test runs, as well as the system components that mainly consume it. Main consumers are ROS nodes and ARGoS, the former exhibit roughly 3-10% of CPU load per node process, depending on the machine's CPU, but roughly constant throughout test runs. In sum however, the CPU utilization of nodes shows superlinear growth with $N_r$. ARGoS was first tested with a bridge implementation that created one ROS node per simulated robot, but this limited the maximum number of robots it was able to simulate to about 65 on the desktop machine before the simulator became unresponsive during launch. Therefore, for the main tests, this was changed so that the common superclass for robots initializes only a single node, which all subscriptions and publishers of individual robots are then registered to. This way, median utilization stays between 70% and 90% throughout. ARGoS CPU utilization decreases after $N_r = 45$ on the mobile machine and $N_r = 75$ on the desktop machine, which also corresponds to the increased incline in latency growth of $t_{argos}$ at this point. Similarly, on the desktop machine, CPU utilization of ROS nodes stagnates. A possible cause for this, as the CPU of the desktop machine is nowhere near fully utilized, is a combination of process switching overhead in ROS nodes and the sub-optimal single-node implementation of ARGoS, both of which will be discussed further in Section 5.4. Beyond $N_r = 105$ on the mobile machine and $N_r = 117$ on the desktop machine, ARGoS started to freeze, therefore these numbers constitute found upper limits of the scalability test.

### 5.3.1. Memory utilization

The memory load, also shown in Figure 5.3, plays a significant role on mobile systems which are usually more limited in capacity. It is almost entirely caused by the sub-processes of ROS nodes, as ARGoS, which has the second highest memory usage, only consumes at most 1.5% of memory at a time on the mobile machine. Up to $N_r = 105$ on the mobile machine and $N_r = 75$ on the desktop machine, there is a linear increase proportional to $N_r$, starting at 2.5 GB of memory use with 15 robots and increasing by about 5 GB for every 30 robots up to a total of just over 17.5 GB at 105 robots. For the mobile machine, this is a main limiting factor, as it only has 16 GB of memory, so the remaining 1.5 GB spill over into swap space on disk, further slowing down the system. As the growth is roughly linear, these results can be extrapolated. The desktop machine is not limited by memory and in theory would be able to simulate roughly an additional 70 robots before reaching its memory limit. With 64 GB of memory, over 360 robots should be feasible and with 128 GB over 720, if other limiting factors are disregarded.

### 5.3.2. Tracing findings

The effect of the initial spike in CPU load during launch is also reflected in callback durations (Figure 5.4). The time spent in callbacks of the MQTT client node shows this most clearly, increasing superlinearly with increasing $N_r$. After the launch process concludes however,

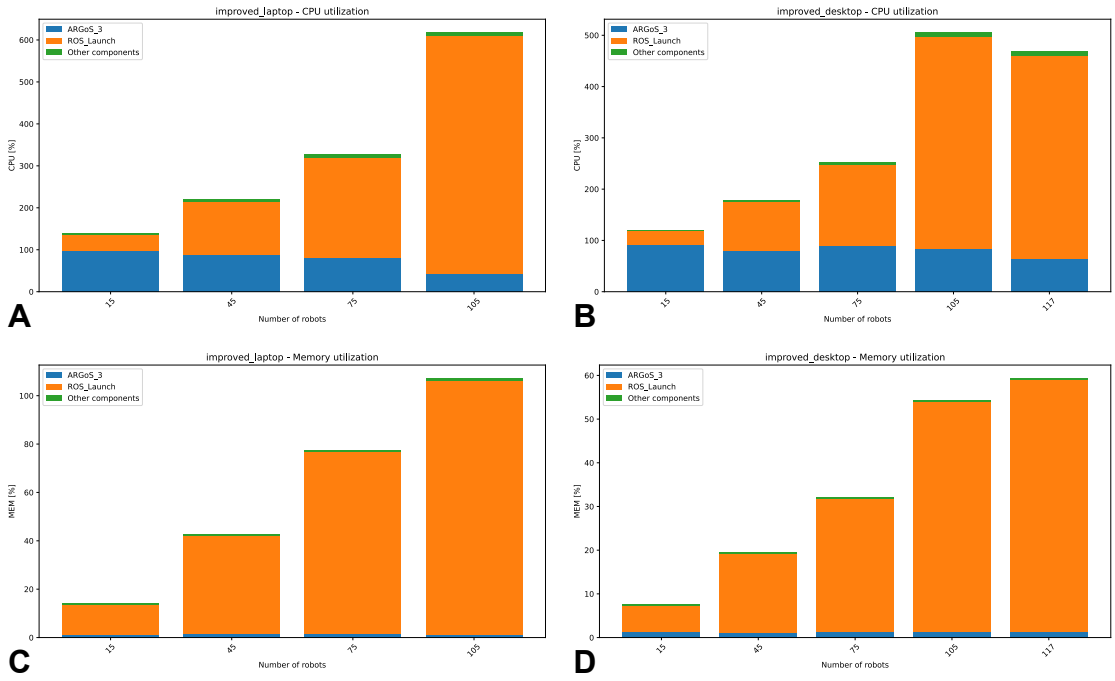


Figure 5.3.: Median system load percentage for CPU of a) the mobile and b) the desktop machine, as well as for memory of c) the mobile and d) the desktop machine per run with different numbers of robots ($N_r$)

callback times are manageable, with the MQTT client taking roughly 3 milliseconds on average with $N_r = 75$ robots to process a message. The total time spent in callbacks is almost negligible at just over one second at $N_r = 15$ and just over 5.5 seconds at $N_r = 75$ during a five minute simulation with additional launch time. This suggests that the MQTT client is not a performance bottleneck in the prototype implementation.

With 15 robots, roughly a quarter million messages were published, increasing to 800 thousand at $N_r = 75$. The mean numbers of messages per second thus amounted to 279, 663 and 917 for the respective robot counts as in the diagrams. Associated process data shows that messages originate from processes distributed across all CPU cores, further supporting the hypothesis of the existence of substantial overhead due to context switching when sender and receiver are situated on the same CPU core.

Finally, delays between sending and receiving messages via the middleware (rmw_fastrtps_cpp) were evaluated, results shown in Figure 5.5. Again, the fully loaded CPU during launch is reflected in delays, which, apart from two outliers, reach up to 16 seconds for $N_r = 15$, 36 seconds for $N_r = 45$ and over 90 seconds at $N_r = 75$ robots. Although delays stabilize after launch to a constant less than 5 milliseconds for all tested numbers of robots, publish-take delays see a sudden incline beyond $N_r = 45$, similar to the previously observed simulation loop durations (Figure 5.2). Whereas with 15 and 45 robots the median delays are around 0.5 milliseconds, they nearly double to just over 0.9 milliseconds at $N_r = 75$.

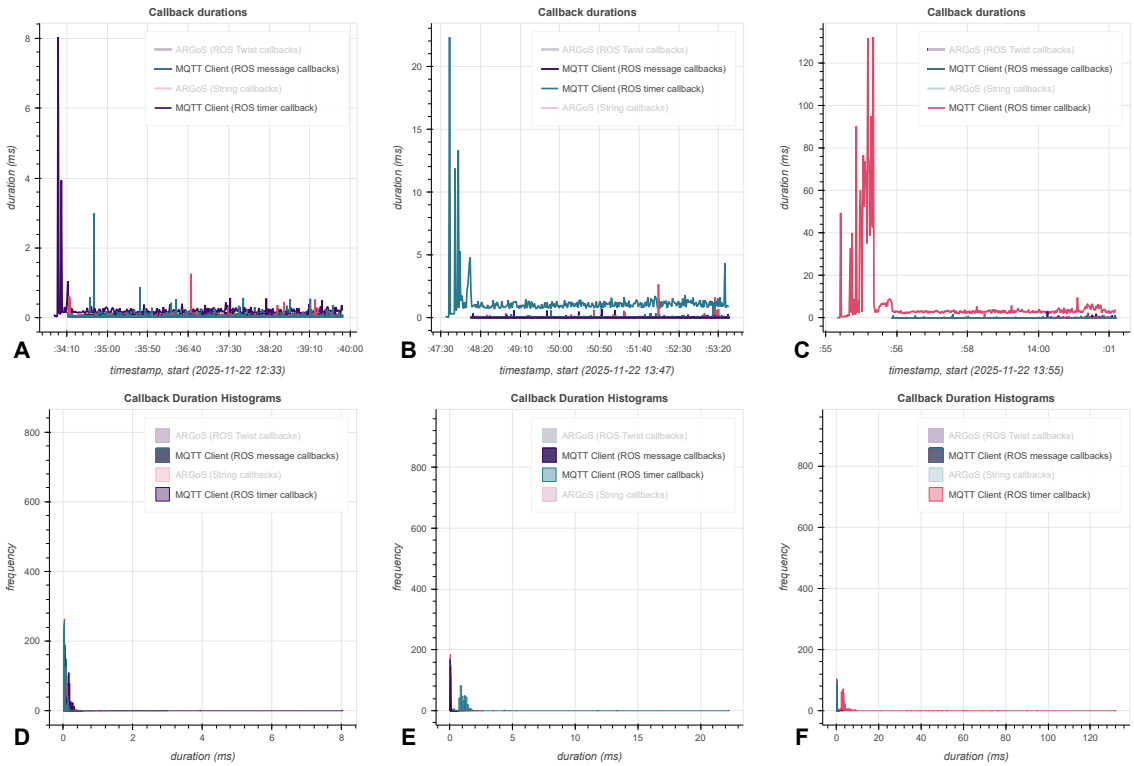


Figure 5.4.: Callback durations and associated histograms of the MQTT client node with $N_r = 15$ (a,d), $N_r = 45$ (b,e) and $N_r = 75$ (c,f) robots. All shown tests were conducted on the mobile machine.

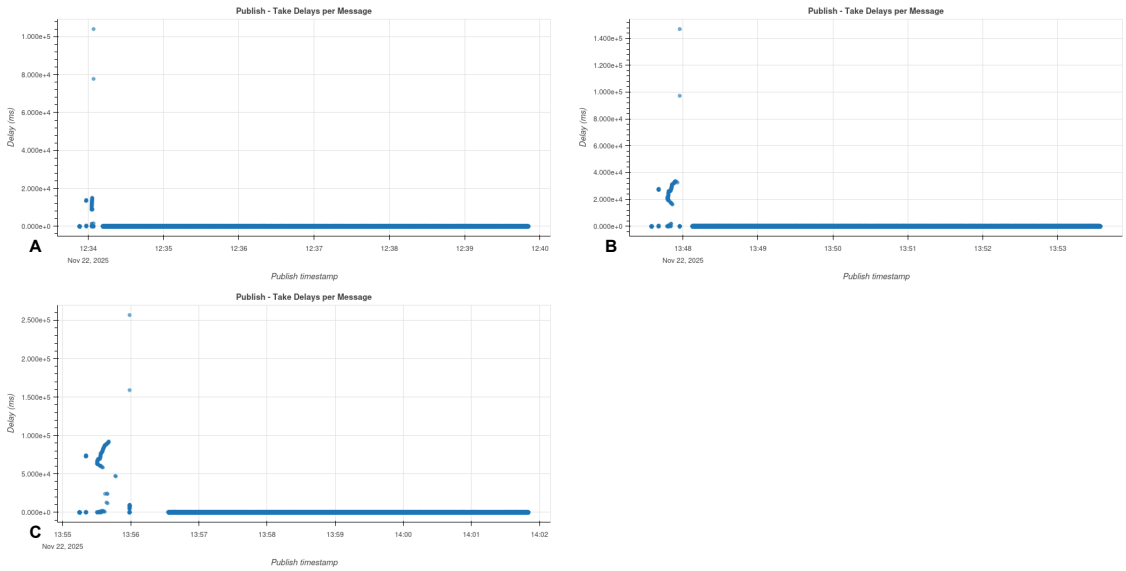


Figure 5.5.: Delays between sending and receiving a message by time sent for $N_r = 15$ (a), $N_r = 45$ (b) and $N_r = 75$ (c) robots. All shown tests were conducted on the mobile machine.

## 5.4. Answers to research questions

In the following sections, answers to the research questions, as outlined in Section 5.1, are given based on the results.

### 5.4.1. $RQ_1$: Scalability limits

Regarding $RQ_1$, it was possible to simulate, monitor and control 75 robots on the mobile machine and 105 robots on the desktop machine with the initial parameters. To mitigate launch delays, $t_{node}$ proved to be the most influential parameter, as increasing it significantly reduced the initial CPU load, helping to prevent ARGoS from freezing during launch. With $t_{node} = 500$ milliseconds, it was possible to simulate up to 105 robots on the mobile machine and 117 robots on the desktop machine. Behavior model startup delays had to be adjusted dynamically to account for increasing launch times of both ROS nodes and ARGoS, but stayed below the duration of the experiment up to 105/117 robots. When exceeding the found current maximum numbers of robots on the respective machines, freezes occurred in the ARGoS simulator during launch, rendering the test unable to run.

### 5.4.2. $RQ_2$: Limiting factors and components

System memory is an important consideration, the test showed that 16 GB barely suffice for simulating around 100 robots without exceeding physical memory size. While the desktop machine was not able to reach its memory limit, it is likely to become a limiting factor should future performance optimizations be applied that lift the current scalability limits significantly. The authors of ARGoS were able to natively[2] simulate 10.000 E-puck robots [99]. The prototype implementation of RuMROS* would require over 1.6 terabytes of RAM for an equivalent number of Turtlebots.

The CPU is mainly a limiting factor during launch, during which both machines reached full CPU utilization. Since utilization drops considerably after launch, further backed by the fact that the desktop machine was not able to simulate twice the number of robots, despite having twice as many cores as the mobile machine, the CPU is not a main limiting factor during the simulation run itself. As mentioned previously, the decrease in CPU load could be explained as a combination of process switching overhead, mainly between ROS robot control nodes, and the single-node implementation of the ROS controllers for ARGoS. The simulator in particular nearly reaches full utilization of a single core, likely already approaching the limits, as ROS node processes were also situated on every core. The system components other than ARGoS and ROS nodes — runtime model, MQTT client node and web interface — showed comparatively low resource consumption and are not considered limiting components in the prototype system.

[2] 'Natively' in this context means directly implementing simple behavior in C++, without having to integrate ROS into an ARGoS controller.

### 5.4.3. $RQ_3$: Candidates for improvements

From the observations made regarding $RQ_2$, the current key issues are related to sub-optimal CPU utilization, making the predominant consumers of CPU time, ARGoS and ROS nodes, main candidates for optimizations. ARGoS has the shortest loop times at the minimum number of robots tested at around 188 milliseconds. For reference, a native implementation of simple behavior without ROS ran at about 100 milliseconds per tick, which corresponds to the set number of ten ticks per second, so there is room for improvement, given that messaging delays and time spent in message callbacks are insignificant once fully launched. Since ARGoS is reaching the limits of what is possible on a single core, implementing multiple threads that each concurrently handle a ROS node with a limited number of publishers and subscribers could help alleviate this issue while improving performance. This should also help decouple nodes from the frontend to prevent freezes of the UI. An early version of the robot controller library used one ROS node per simulated robot, but this had considerably worse performance than the current implementation with a single node. However, creating few nodes with a limited number of publishers and subscribers, each in a separate thread will likely not run into this issue, as there are will be significantly fewer nodes than robots. Compared to the processes ROS launch uses, threads also have less overhead, working around the potential switching overhead issue.

RuMROS* uses Python for the implementation of ROS nodes. When launched, a process is created for each node. To decrease overhead and improve performance, ROS 2 comes with a mechanism that allows for a single process to handle multiple nodes via *composition*.[3] However, this requires nodes to be explicitly programmed for the respective interface, which the ROS 2 version used in this thesis, *Jazzy*, currently only supports for nodes written in C++.[4] The only permanent solution currently available is thus to re-write the robot control nodes in C++, but as a workaround, this limit could also be overcome by making use of the networking capabilities of ROS 2, splitting nodes over multiple machines on a network for large simulations.

[3] See `https://design.ros2.org/articles/roslaunch.html` for more details on ROS launch and composition.
[4] `https://docs.ros.org/en/jazzy/The-ROS2-Project/Features.html`

# 6. Related work

Many of the approaches and systems discussed in previous chapters covered aspects that RuMROS* also aims to address, such as controlling MRS with decentralized algorithms [58] or in a hybrid manner [59], runtime modeling [124] or simulation [100] of MRS. RuMROS* is a novel framework that allows for programming, runtime monitoring and simulation of hybrid MRS with ROS 2. To the best of this thesis' author's knowledge, no other systems that offer this combination of features currently exist. Therefore, in this chapter, focus lies on systems as closely related to the main aspects of RuMROS* as possible. In order to keep performance metrics and features comparable, all approaches subject to comparison in this chapter are situated in the field of robotics, with focus on ROS and ROS 2-based ones.

## 6.1. Approaches focusing on multi-robot systems

As RuMROS* is targeted towards hybrid MRS, with both decentralized and centralized control infrastructure, such systems are the main focus point of this section. The decentralized movement patterns implemented in the prototype are closely related to the patterns for mobile ground robots of ROS2Swarm [58], featured in Chapter 3. RuMROS* also offers a library of decentralized behaviors, but moreover, also features centralized infrastructure in form of the runtime model, to configure and adaptively compose such behaviors with the additional behavior modeling auxiliary classes. This has the additional benefit of offering guidance on how to utilize the patterns on a higher level to program the MRS as a whole, which is an important concern for pattern-based approaches to programming MRS [51]. RuMROS* offers a movement behavior library comparable to that of ROS2Swarm, but lacks the decision-making patterns, as it makes use of the runtime model for control over when patterns start and stop and how they should behave. ROS2Swarm was not tested on swarms with more than seven robots, as in decentralized systems, only performance of individual nodes is a concern. RuMROS* as a framework that utilizes patterns, additionally has the advantage of taking the development process into account. By integrating the ARGoS simulator, quick tests of basic behavior blocks as well as more complex composed behaviors with larger swarms can be conducted in simulation to aid both the design process of basic behaviors and compositional logic. Some of the authors of ROS2Swarm also recently proposed MacroSwarm, a framework that uses Scala to compose behaviors [1], which does

provide better guidance for composition, but doesn't currently support ROS in contrast to RuMROS*.

Another interesting system for comparison, due to its similarities to RuMROS*, is HiSARM [59]. Although it does not support ROS at the time of writing, the authors consider it a potential future enhancement. As the architectural differences were already covered in Chapter 3, this section concentrates on comparing the programming approaches and evaluating performance. In RuMROS*, behaviors can be implemented by dedicated developers on the ROS side and have to be declared with a unique name on the JastAdd [50] side, so they can be called from the UI and used as building blocks in composition. HiSARM uses a DSL for declaring the mission specification, including teams, transitions, modes as well as services, while RuMROS* uses JastAdds *JRAG* and *JADD* files with their respective syntax. This means that RuMROS* uses a more generic specification language while HiSARM's is more tailored to the control of robotic systems. Since their DSL includes concrete keywords to hide more complex features of the underlying state machine mechanism to which it is ultimately translated to, it allows them to specify missions and scenarios in a much more compact way than RuMROS*. However, a central drawback of this approach is that it restricts developers to the exact modeling features the developers provided within the DSL, while RuMROS* allows its users to adjust and expand the grammar and thus the structure of the specification language. In terms of the MOF, RuMROS* offers additional flexibility for the mission specification, due to making use of JastAdd as a meta-language (modeling level M3). This enables custom language features to be modeled, whereas HiSARM has a fixed DSL on level M2, allowing only modeling of concrete models on M1. In practice, even without modifying the structural definitions, the semantic specification can be enriched with special methods that act as "syntactic sugar". This helps to simplify the code required for the specification of the HFSM for the behavior model, and can be tailored to the given use case. While this does require additional development effort once, it can alleviate the drawback of the exemplary implementation of the behavior model being more bulky in comparison to its equivalents of DSL-based approaches like HiSARM.

### 6.1.1. Simulation of MRSs

Pitonakova et al. compared open-source simulators focused on robotics, showing that ARGoS is a suitable choice for testing simple algorithms with swarms [102], which form the foundation of the simple nodes offered by RuMROS*. When using ARGoS natively and without ROS 2 integration, simulating thousands of simple wheeled robots is possible [100]. This number is much lower in RuMROS* despite using the same simulator, primarily due to ROS 2 node CPU and memory consumption. This begs the question of how many robots other projects that employ simulation of large MRS with ROS can simulate, and whether they overcome the performance issues described in Chapter 5. This section focuses on such projects in order to answer the questions above.

### 6.1.2. Simulation with ARGoS

While the initial performance claims of ARGoS claimed simulation of swarms orders of magnitude larger than Gazebo [100], this is not the case for RuMROS*, which has ROS nodes

steer the simulated robots. For using ARGoS with any version of ROS, bridges have been developed previously, but either make no performance claims[1] or are still under development[2] and have not been evaluated yet. To the best of this thesis' author's knowledge, there are also no projects that use ARGoS with ROS while making performance or scalability claims. For other simulators, papers with performance claims that also explicitly integrate ROS for simulation are scarce, often focusing on the system's structure, functionality and usability, without a dedicated performance and scalability evaluation [71, 103]. The systems featured in papers that do offer such insights are compared to RuMROS* in the following section.

### 6.1.3. Simulation with other simulators

Mañas-Álvarez et al. evaluated the performance and scalability of an experimental scenario with up to 40 robots with a complexity comparable to RuMROS* (moving with collision avoidance) on a powerful mobile workstation machine for Gazebo and Webots [74]. They found that Gazebo was unable to simulate their test scenario with 40 robots, due to CPU resource constraints. Webots on the other hand managed all test scenarios up to 40 robots. In comparison, the developed initial version of RuMROS* is able to simulate over twice the number of robots that Gazebo can handle, while for Webots further scalability tests beyond 40 robots are required in order to make a comparison.

*MAES* [2] is a custom simulation tool designed for efficient and realistic simulation of swarm algorithms for exploration and coverage with support for ROS 2. In terms of feature support, it is more mature than RuMROS*, providing visualization tools and realistic wireless communication simulation as well as randomly generated maps. For debugging and visualization, RuMROS* includes a simplistic LiDAR visualization ROS node, based on matplotlib.[3] Since RuMROS* is the first iteration of a framework, with further development of auxiliary tools, as well as user extension in mind, it lacks a more extensive toolchain. In terms of performance, MAES was able to simulate up to 5 robots with ROS 2 on a mobile machine, compared to the over 100 robots that RuMROS* was able to simulate with ARGoS. The authors did not capture individual CPU and memory usage data per process, but from the total usage with one, three and five robots, a roughly ten percent increase in CPU usage and roughly 250 to 400 MB of memory per robot can be inferred, about twice that of the ROS nodes developed in this work. Since the CPU they used should have very comparable single-core performance to the mobile CPU used in testing of this project, this inferred data is comparable to RuMROS*. While their nodes are about twice as complex, they reach the overall system CPU limit much quicker than RuMROS* with ARGoS in comparison, showing that RuMROS* is more suitable for simulations of larger swarms with lightweight nodes.

In order to circumvent the problem of ROS 2 performance overhead in simulation, the authors of MAES opted to offer Unity[4] as an alternative and much more efficient simulation platform, allowing up to 120 robots to be simulated. This does however negate the benefits of directly developing with ROS 2 nodes, as code has to be written in C# in Unity and

[1] argos_bridge connects ARGoS to ROS (version 1), see `https://github.com/BOTSlab/argos_bridge`.
[2] argos3-ros2-bridge connects ARGoS to ROS 2 and is used in a modified form in RuMROS*, see `https://github.com/CPS-Konstanz/argos3-ros2-bridge`.
[3] `https://matplotlib.org/`
[4] `https://unity.com/`

then ported to C++ or Python for ROS 2. RuMROS* in contrast aims to offer large-scale simulations natively with ROS 2 in order to avoid having to write code twice.

## 6.2. Runtime modeling approaches

The authors of GRuM evaluated the set-model-latency (SML) of their runtime monitoring approach [124]. It measures the time between receiving an event and it being reflected (set) in the runtime model, explicitly excluding the messaging delays. For a test scenario with 25 drones, they report SMLs of roughly 5 to 13 milliseconds, depending on the property for which it was measured. Unfortunately, they don't fully clarify what actions and components exactly are part of the pipeline that they measure the SML on, only that communication delays are excluded, making a direct comparison to RuMROS* difficult. However, the internal architecture around the generated runtime modeling platform is similar to RuMROS*, in that it also uses Java for implementation and MQTT to propagate updates to the runtime model. For this reason, the callback durations of the MQTT client node in addition to latencies introduced by RAGConnect callbacks are used to provide performance context in this section.

As previously shown in Figure 5.4, even for 75 robots, three times the number of robots used for evaluation of GRuM, callback durations on the MQTT client node generally stayed below five milliseconds after launch. On the runtime model component, pose message callback delays represent the for setting time of the most frequently received message type in the AST-based model. Figure A.1 shows that they consistently stayed well below one millisecond after launch, with medians around 0.08 milliseconds. The total median set latencies of RuMROS*, excluding messaging delays, are therefore below six milliseconds, even for 75 robots. This is comparable to the low end of the property update latencies of GRuM. In addition to this, the authors of GRuM quote around 1530 property changes per minute, while pose updates in RuMROS* alone account for $75 \times 2 \times 60 = 9000$ updates per minute (at 75 robots, with each robot updating its pose every 500 milliseconds). Consequently, the observed delays show that even with three times the number of robots and much more frequent updates, the processing delays of RuMROS* are within the same order of magnitude as those of GRuM, even if their method of evaluation is not entirely clear in detail.

# 7. Conclusion

In this chapter, the evaluation results of the framework developed in this thesis are summarized and put into context of the design choices outlined in Section 1.3. It also tackles to which degree the goals outlined in Section 1.2 were achieved and thus what problems were solved. Finally, remaining issues and future work are discussed.

## 7.1. Results

The framework developed in this thesis was evaluated in simulation using a warehouse simulation with multiple robot groups executing complex behaviors that required active LiDAR sensing. The evaluation demonstrated that the prototype is capable of simulating more than one hundred robots on a single machine, reaching up to 105 robots on a mobile system and 117 robots on a desktop system before stability limits were encountered.

System resource usage statistics revealed that while CPU utilization peaked during system launch, leading to delays on launch, it remained below the theoretical maximum available through multi-core processing and simultaneous multithreading. Memory consumption emerged as a limiting factor, as each additional robot required approximately 167 MB of memory, resulting in total usage of around 17.5 GB for 105 robots. Network and ROS middleware performance did not pose a bottleneck, with local message latencies remaining low and callback overhead negligible compared to overall simulation time.

The dominant resource consumers were ARGoS and ROS 2 nodes, while runtime model, MQTT client, and web UI contributed only marginally to system load. Simulation speed decreased noticeably with increasing system size: scenarios with more than 100 robots ran close to real time and significantly slower than smaller configurations. Compared to native ARGoS simulations without ROS integration, performance was reduced, reflecting the overhead introduced by the integration of ROS.

A comparison with related work was limited due to the scarcity of systems combining runtime modeling with ROS. Nevertheless, the achieved model-set latencies were comparable to GRuM [124], despite the much larger system size.

## 7.2. Design choices and result interpretation

The evaluation results reflect the fundamental design trade-offs underlying RuMROS*. While ARGoS is capable of simulating thousands of simple robots in isolation [99], the integration of ROS 2 inevitably reduces scalability due to additional processes and communication overhead. Despite this limitation, ARGoS still enables significantly larger simulations than Gazebo, whose scalability typically saturates at around 30 to 40 robots [74]. The framework aims to provide seamless ROS 2 integration while keeping a comparatively high scalability limit. By employing a ROS bridge plugin and an ARGoS configuration generator, ROS 2 is embedded directly into both simulation and development and thus the need for simulator-specific code adaptations is eliminated, reducing development effort and avoiding inconsistencies between simulated and deployed systems. For advanced use cases, templates can be used to manually set simulation configuration variables directly.

JastAdd [50] proved well suited for framework development due to its separation of structural and semantic specifications. The use of the RelRAGs enabled concise modeling of hierarchical structures like robot groups and nested behavior model nodes as well as providing separation of structure from implementation, facilitating adjustments and extensions. The aspect-oriented approach allowed crosscutting modeling concerns, such as environment modeling, robot capability declaration, runtime behavior, and causal connections to both the robots and UI, to be modularized cleanly, supporting extensibility and maintainability.

## 7.3. Achieved goals

The primary system modeling objectives were largely achieved. The runtime model benefits from a clear separation between structure and semantics, fulfilling the goal of modular system modeling ($G_1$).

The framework's support for runtime modeling ($G_2$) was successfully extended to MRS through explicit support for groups and hierarchical organizational structures. Limited runtime reconfiguration is possible by reassigning individual robots between groups, but full dynamic restructuring of organizational hierarchies was not realized, resulting in partial fulfillment of this objective ($G_3$).

The HFSM-based behavior model enables bottom-up construction of complex system behavior and supports runtime adaptation through conditional transitions. The GUI was also extended to include an overview of the behavior model, while retaining and enhancing existing supervision features for human operators, addressing $G_4$ and $G_5$. The existing Gazebo-based workflow remains supported ($G_6$) through target-specific compilation, ensuring backward compatibility, although advanced multi-robot abstractions are exclusive to the ARGoS-based target, as it is intended for modeling MRS with a hybrid structure. By including a library of decentralized behaviors as well as a centralized controller, the infrastructure for hybrid systems is provided, achieving $G_7$.

To address scalability limitations of Gazebo, ARGoS was seamlessly integrated as an alternative simulator ($G_9$), synchronized to the specified scenario in the runtime modeling component. The results demonstrate that, despite performance overhead from ROS 2, the

framework scales to more than twice the robot count typically feasible with Gazebo on a single machine. As such, the simulation scalability objective ($G_8$) is considered achieved.

## 7.4. Solved problems

RuMROS* addresses several limitations of the original framework and related approaches. Employing ARGoS for simulation of MRS significantly improves scalability compared to Gazebo and other approaches that integrate ROS. However, evaluation results indicate that available system resources are not yet fully exploited, leaving room for future improvement. By shifting simulation and system specification to the higher-level JastAdd model, RuMROS* reduces fragmentation between development and simulation. The direct integration of ROS 2 extends this to later deployment of the system by reducing inconsistencies, as robot code does not have to be adapted to the simulator. Dedicated modeling constructs for MRS, a reusable ROS behavior library, and explicit communication mechanisms for organizational units resolve prior difficulties in addressing robots and processing their data, without overloading the runtime model.

The HFSM-based behavior modeling approach abstracts from low-level ROS code while enabling structured composition of common multi-robot movement patterns, with conditional transitions to support adaptivity based on system or environmental state. These structures provide developers with higher-level guidance on implementing adaptivity compared to approaches focusing on providing only behaviors on the ROS layer. The GUI was extended to expose multi-robot concepts at runtime, including group relations and active behavior states. This promotes transparency of the underlying ROS-based system while enabling human supervision and intervention at runtime.

Overall, RuMROS* combines a runtime modeling approach with ROS 2 integration for hybrid MRS, by offering improved scalability and additional MRS-specific modeling abstractions compared to the initial system RuMROS [117]. The use of JastAdd promotes separation of concerns and modeling hierarchical structures, as well as avoiding rigidity typical of DSL-based approaches by allowing the modeling language itself to evolve alongside the framework.

## 7.5. Future work

In addition to improving performance, feature support should also be extended in future work. This includes supporting more robot types, evaluating control scenarios with more centralization, and supporting a broader subset of ARGoS simulation features in modeling, as currently not all concepts of its configuration are supported in JastAdd. The JastAdd-based runtime modeling approach should be evaluated through user studies, to assess its impact on developer experience compared to established DSL-based solutions. Improvements to tooling, such as editor support and analysis features, would likely be necessary for practical adoption.

Regarding adaptivity, restructuring organizational units via global operators such as Buzz offers [98] would be a considerable improvement towards fully reconfigurable units at runtime.

Even more promising with respect to adaptivity is context-role oriented programming [45], which has recently been applied to robot swarms [46]. Its role-based approach provides more flexibility for organizational structures. Alternative structures such as behavior trees should also be evaluated in place of the HFSM-based approach, as they can be better represented by the tree structure of JastAdd's AST.

Finally, formal verifiability is essential for real-world deployment. Integrating verifiable modeling mechanisms, either as an alternative or extension to the current behavior modeling mechanism, represents a key direction for advancing RuMROS* toward safety-critical applications. A promising candidate is Petri nets, a modeling approach which natively supports formal verification and has already been employed for ROS-based robotic applications [33].

# Bibliography


[1] Gianluca Aguzzi and Mirko Viroli. “MacroSwarm: A Scala Framework for Swarm Programming”. In: *Science of Computer Programming* 239 (Jan. 2025), p. 103182.

[2] Malte Z. Andreasen et al. “MAES: A ROS 2-Compatible Simulation Tool for Exploration and Coverage Algorithms”. In: *Artificial Life and Robotics* 28.4 (Nov. 2023), pp. 757–770.

[3] Tamio Arai and Lynne Parker. “Editorial: Advances in Multi-Robot Systems”. In: (Feb. 2003).

[4] H. Asama, A. Matsumoto, and Y. Ishida. “Design Of An Autonomous And Distributed Robot System: Actress”. In: *Proceedings. IEEE/RSJ International Workshop on Intelligent Robots and Systems ’. (IROS ’89) ’The Autonomous Mobile Robots and Its Applications*. Sept. 1989, pp. 283–290.

[5] Richard Balogh and David Obdrálek. “Using Finite State Machines in Introductory Robotics”. In: *Robotics in Education*. Ed. by Wilfried Lepuschitz et al. Cham: Springer International Publishing, 2019, pp. 85–91.

[6] Nelly Bencomo, Sebastian Götz, and Hui Song. “Models@run.Time: A Guided Tour of the State of the Art and Research Challenges”. In: *Software & Systems Modeling* 18.5 (Oct. 2019), pp. 3049–3082.

[7] Nelly Bencomo et al., eds. *Models@run.Time: Foundations, Applications, and Roadmaps*. Vol. 8378. Lecture Notes in Computer Science. Cham: Springer International Publishing, 2014.

[8] Gerardo Beni. “From Swarm Intelligence to Swarm Robotics”. In: *Swarm Robotics*. Ed. by Erol ahin and William M. Spears. Berlin, Heidelberg: Springer, 2005, pp. 1–9.

[9] Kenneth P. Birman. “CORBA: The Common Object Request Broker Architecture”. In: *Guide to Reliable Distributed Systems: Building High-Assurance Applications and Cloud-Hosted Services*. London: Springer London, 2012, pp. 249–269.

[10] Gordon Blair, Nelly Bencomo, and Robert B. France. “Models@ Run.Time”. In: *Computer* 42.10 (Oct. 2009), pp. 22–27.

[11] Marco Blumendorf, Sebastian Feuerstack, and Sahin Albayrak. "Multimodal User Interfaces for Smart Environments: The Multi-Access Service Platform". In: *Proceedings of the Working Conference on Advanced Visual Interfaces*. AVI '08. New York, NY, USA: Association for Computing Machinery, May 2008, pp. 478–479.

[12] N. Bonjean et al. "ADELFE 2.0". In: *Handbook on Agent-Oriented Design Processes*. Ed. by Massimo Cossentino et al. Berlin, Heidelberg: Springer, 2014, pp. 19–63.

[13] Darko Bozhinoski et al. "MROS: Runtime Adaptation for Robot Control Architectures". In: *Advanced Robotics* 36.11 (June 2022), pp. 502–518.

[14] Jürgen Branke et al. "Organic Computing – Addressing Complexity by Controlled Self-Organization". In: *Second International Symposium on Leveraging Applications of Formal Methods, Verification and Validation (Isola 2006)*. Nov. 2006, pp. 185–191.

[15] Torgny Brogårdh. "Present and Future Robot Control Development—An Industrial Perspective". In: *Annual Reviews in Control* 31.1 (Jan. 2007), pp. 69–79.

[16] H. Bruyninckx. "Open Robot Control Software: The OROCOS Project". In: *Proceedings 2001 ICRA. IEEE International Conference on Robotics and Automation (Cat. No.01CH37164)*. Vol. 3. May 2001, 2523–2528 vol.3.

[17] Cindy Calderón-Arce, Juan Carlos Brenes-Torres, and Rebeca Solis-Ortega. "Swarm Robotics: Simulators, Platforms and Applications Review". In: *Computation* 10.6 (June 2022), p. 80.

[18] Y.U. Cao et al. "Cooperative Mobile Robotics: Antecedents and Directions". In: *Proceedings 1995 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human Robot Interaction and Cooperative Robots*. Vol. 1. Aug. 1995, 226–234 vol.1.

[19] Raja Chatila et al. "Toward Self-Aware Robots". In: *Frontiers in Robotics and AI* 5 (Aug. 2018).

[20] David T. Coleman et al. "Reducing the Barrier to Entry of Complex Robotic Software: A MoveIt! Case Study". In: Italy, 2014.

[21] Michele Colledanchise and Petter Ögren. *Behavior Trees in Robotics and AI: An Introduction*. July 2018. arXiv: `1709.00084 [cs]`.

[22] Edson de Araújo Silva et al. "A Survey of Model Driven Engineering in Robotics". In: *Journal of Computer Languages* 62 (Feb. 2021), p. 101021.

[23] Alex Luiz De Sousa, André Schneider De Oliveira, and Marco Antônio S. Teixeira. "Multi-Agent Robot Systems: Analysis, Classification, Applications, Challenges and Directions". In: *2024 IEEE International Conference on Industrial Technology (ICIT)*. Mar. 2024, pp. 1–8.

[24] Tom De Wolf and Tom Holvoet. "Design Patterns for Decentralised Coordination in Self-Organising Emergent Systems". In: *Proceedings of the 4th International Conference on Engineering Self-Organising Systems*. ESOA'06. Berlin, Heidelberg: Springer-Verlag, May 2006, pp. 28–49.

[25] Ada Diaconescu. *Goal-Oriented Holonic Systems*. Jan. 2017.

[26] Ada Diaconescu. *Organising Complexity: Hierarchies and Holarchies*. Jan. 2017.

[27] Marco Dorigo, Guy Theraulaz, and Vito Trianni. "Swarm Robotics: Past, Present, and Future [Point of View]". In: *Proceedings of the IEEE* 109.7 (July 2021), pp. 1152–1165.

[28] Ali Dorri, Salil S. Kanhere, and Raja Jurdak. "Multi-Agent Systems: A Survey". In: *IEEE Access* 6 (2018), pp. 28573–28593.

[29] Alexey Dosovitskiy et al. "CARLA: An Open Urban Driving Simulator". In: *Proceedings of the 1st Annual Conference on Robot Learning*. 2017, pp. 1–16.

[30] G. Dudek et al. "A Taxonomy for Swarm Robots". In: *Proceedings of 1993 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS '93)*. Vol. 1. July 1993, 441–447 vol.1.

[31] Gregory Dudek et al. "A Taxonomy for Multi-Agent Robotics". In: *Autonomous Robots* 3.4 (Dec. 1996), pp. 375–397.

[32] Gregory Dudek et al. "A Taxonomy for Multi-Agent Robotics". In: *Autonomous Robots* 3.4 (Dec. 1996), pp. 375–397.

[33] Sebastian Ebert et al. "Distributed Petri Nets for Model-Driven Verifiable Robotic Applications in ROS". In: *Innovations in Systems and Software Engineering* 20.4 (Dec. 2024), pp. 531–557.

[34] Bruce Edmonds. "Using the Experimental Method to Produce Reliable Self-Organised Systems". In: *Engineering Self-Organising Systems*. Ed. by Sven A. Brueckner et al. Berlin, Heidelberg: Springer, 2005, pp. 84–99.

[35] Tzilla Elrad, Robert E. Filman, and Atef Bader. "Aspect-Oriented Programming: Introduction". In: *Commun. ACM* 44.10 (Oct. 2001), pp. 29–32.

[36] Debora C. Engelmann et al. "RV4JaCa—Towards Runtime Verification of Multi-Agent Systems and Robotic Applications". In: *Robotics* 12.2 (Apr. 2023), p. 49.

[37] Martin Ester et al. "A Density-Based Algorithm for Discovering Clusters in Large Spatial Databases with Noise". In: *Proceedings of the Second International Conference on Knowledge Discovery and Data Mining*. KDD'96. Portland, Oregon: AAAI Press, Aug. 1996, pp. 226–231.

[38] Jose Luis Fernandez-Marquez et al. "Description and Composition of Bio-Inspired Design Patterns: A Complete Overview". In: *Natural Computing* 12.1 (Mar. 2013), pp. 43–67.

[39] Daniel J. Fremont et al. "Scenic: A Language for Scenario Specification and Data Generation". In: *Machine Learning* 112.10 (Oct. 2023), pp. 3805–3849.

[40] Erich Gamma et al. *Design Patterns*. Pearson Education, Nov. 2000.

[41] Sergio García et al. "Robotics Software Engineering: A Perspective from the Service Robotics Domain". In: *Proceedings of the 28th ACM Joint Meeting on European Software Engineering Conference and Symposium on the Foundations of Software Engineering*. ESEC/FSE 2020. New York, NY, USA: Association for Computing Machinery, Nov. 2020, pp. 593–604.

[42] Avinash Gautam and Sudeept Mohan. "A Review of Research in Multi-Robot Systems". In: *2012 IEEE 7th International Conference on Industrial and Information Systems (ICIIS)*. Aug. 2012, pp. 1–5.

[43] Carlos Gershenson. “Design and Control of Self-Organizing Systems”. 2007.

[44] Daniel Graff, Jan Richling, and Matthias Werner. “jSwarm: Distributed Coordination in Robot Swarms”. In: Apr. 2014.

[45] Christian Gutsche et al. “Context-Oriented Programming and Modeling in Julia with Context Petri Nets”. In: *2024 50th Euromicro Conference on Software Engineering and Advanced Applications (SEAA)*. Aug. 2024, pp. 1–9.

[46] Christian Gutsche et al. “Context-Role Oriented Programming in Julia: Advancing Swarm Programming”. In: *2025 IEEE/ACM 20th Symposium on Software Engineering for Adaptive and Self-Managing Systems (SEAMS)*. Apr. 2025, pp. 85–95.

[47] Hadeli et al. “Multi-Agent Coordination and Control Using Stigmergy”. In: *Computers in Industry* 53.1 (Jan. 2004), pp. 75–96.

[48] James Harbin et al. “Model-Driven Design Space Exploration for Multi-Robot Systems in Simulation”. In: *Software and Systems Modeling* 22.5 (Oct. 2023), pp. 1665–1688.

[49] Görel Hedin. “Reference Attributed Grammars”. In: *Informatica* 24.3 (2000), pp. 301–317.

[50] Görel Hedin and Eva Magnusson. “JastAdd—an Aspect-Oriented Compiler Construction System”. In: *Science of Computer Programming*. Special Issue on Language Descriptions, Tools and Applications (L DTA’01) 47.1 (Apr. 2003), pp. 37–58.

[51] Christopher-Eyk Hrabia, Marco Lützenberger, and Sahin Albayrak. “Towards Adaptive Multi-Robot Systems: Self-Organization and Self-Adaptation”. In: *The Knowledge Engineering Review* 33 (Jan. 2018), e16.

[52] F.F. Ingrand et al. “PRS: A High Level Supervision and Control Language for Autonomous Mobile Robots”. In: *Proceedings of IEEE International Conference on Robotics and Automation*. Vol. 1. Apr. 1996, 43–49 vol.1.

[53] Pablo Iñigo-Blasco et al. “Robotics Software Frameworks for Multi-Agent Robotic Systems Development”. In: *Robotics and Autonomous Systems* 60.6 (June 2012), pp. 803–821.

[54] Luca Iocchi, Daniele Nardi, and Massimiliano Salerno. “Reactivity and Deliberation: A Survey on Multi-Robot Systems”. In: *Balancing Reactivity and Social Deliberation in Multi-Agent Systems*. Ed. by Markus Hannebauer, Jan Wendler, and Enrico Pagello. Berlin, Heidelberg: Springer, 2001, pp. 9–32.

[55] Matteo Iovino et al. “Comparison Between Behavior Trees and Finite State Machines”. In: *IEEE Transactions on Automation Science and Engineering* 22 (2025), pp. 21098–21117.

[56] Zool Ismail and Nohaidda Sariff. “A Survey and Analysis of Cooperative Multi-Agent Robot Systems: Challenges and Directions”. In: Nov. 2018.

[57] *Joseph Engelberger and Unimate: Pioneering the Robotics Revolution*.

[58] Tanja Katharina Kaiser et al. “ROS2swarm - A ROS 2 Package for Swarm Robot Behaviors”. In: *2022 International Conference on Robotics and Automation (ICRA)*. May 2022, pp. 6875–6881. arXiv: `2405.02438 [cs]`.

[59] Woosuk Kang et al. “A Framework for Multi-Robot Programming: From High-Level Specification to Retargetable Deployment”. In: *ACM Trans. Embed. Comput. Syst.* (July 2025).

[60] Stuart Kent. “Model Driven Engineering”. In: *Integrated Formal Methods*. Ed. by Michael Butler, Luigia Petre, and Kaisa Sere. Berlin, Heidelberg: Springer, 2002, pp. 286–298.

[61] J.O. Kephart and D.M. Chess. “The Vision of Autonomic Computing”. In: *Computer* 36.1 (Jan. 2003), pp. 41–50.

[62] Donald E. Knuth. “Semantics of Context-Free Languages”. In: *Mathematical systems theory* 2.2 (June 1968), pp. 127–145.

[63] Samuel Kounev et al., eds. *Self-Aware Computing Systems*. Cham: Springer International Publishing, 2017.

[64] Christian Krupitzer et al. “A Survey on Engineering Approaches for Self-Adaptive Systems”. In: *Pervasive and Mobile Computing*. 10 Years of Pervasive Computing' In Honor of Chatschik Bisdikian 17 (Feb. 2015), pp. 184–206.

[65] Stefan Kugele, Ana Petrovska, and Ilias Gerostathopoulos. “Towards a Taxonomy of Autonomous Systems”. In: *Software Architecture*. Ed. by Stefan Biffl et al. Cham: Springer International Publishing, 2021, pp. 37–45.

[66] Adrian Kuhn and Toon Wim Jan Verwaest. “FAME, A Polyglot Library for Metamodeling at Runtime”. In: *Workshop on Models at Runtime*. 2008, pp. 57–66.

[67] Peter R. Lewis et al. “Architectural Aspects of Self-Aware and Self-Expressive Computing Systems: From Psychology to Engineering”. In: *Computer* 48.8 (Aug. 2015), pp. 62–70.

[68] Peter R. Lewis et al., eds. *Self-Aware Computing Systems: An Engineering Approach*. Natural Computing Series. Cham: Springer International Publishing, 2016.

[69] Zhengping Li, Cheng Hwee Sim, and Malcolm Yoke Hean Low. “A Survey of Emergent Behavior and Its Impacts in Agent-based Systems”. In: *2006 4th IEEE International Conference on Industrial Informatics*. Aug. 2006, pp. 1295–1300.

[70] Huihan Liu et al. *Model-Based Runtime Monitoring with Interactive Imitation Learning*. Oct. 2023. arXiv: `2310.17552 [cs]`.

[71] Zhengguang Ma et al. “ROS-Based Multi-Robot System Simulator”. In: *2019 Chinese Automation Congress (CAC)*. Nov. 2019, pp. 4228–4232.

[72] Steve Macenski and Ivona Jambrecic. “SLAM Toolbox: SLAM for the Dynamic World”. In: *Journal of Open Source Software* 6.61 (2021), p. 2783.

[73] Steven Macenski et al. “Robot Operating System 2: Design, Architecture, and Uses in the Wild”. In: *Science Robotics* 7.66 (May 2022), eabm6074.

[74] Francisco José Mañas-Álvarez et al. “Scalability of Cyber-Physical Systems with Real and Virtual Robots in ROS 2”. In: *Sensors (Basel, Switzerland)* 23.13 (July 2023), p. 6073.

[75] Yuya Maruyama, Shinpei Kato, and Takuya Azumi. “Exploring the Performance of ROS2”. In: *2016 International Conference on Embedded Software (EMSOFT)*. Oct. 2016, pp. 1–10.

[76] Petra Mazdin and Bernhard Rinner. "Distributed and Communication-Aware Coalition Formation and Task Assignment in Multi-Robot Systems". In: *IEEE Access* PP (Feb. 2021), pp. 1–1.

[77] S.J. Mellor, A.N. Clark, and T. Futagami. "Model-Driven Development - Guest Editor's Introduction". In: *IEEE Software* 20.5 (Sept. 2003), pp. 14–18.

[78] Giorgio Metta, Paul Fitzpatrick, and Lorenzo Natale. "YARP: Yet Another Robot Platform". In: *International Journal of Advanced Robotic Systems* 3.1 (Mar. 2006), p. 8.

[79] Johannes Mey et al. "Relational Reference Attribute Grammars: Improving Continuous Model Validation". In: *Journal of Computer Languages* 57 (Apr. 2020), p. 100940.

[80] Enrico Mingo Hoffman et al. "Yarp Based Plugins for Gazebo Simulator". In: *Modelling and Simulation for Autonomous Systems*. Ed. by Jan Hodicky. Cham: Springer International Publishing, 2014, pp. 333–346.

[81] Melanie Mitchell. *Self-Awareness and Control in Decentralized Systems*.

[82] Mirko Morandini et al. "A Goal-Oriented Approach for Modelling Self-organising MAS". In: *Engineering Societies in the Agents World X*. Ed. by Huib Aldewereld, Virginia Dignum, and Gauthier Picard. Berlin, Heidelberg: Springer, 2009, pp. 33–48.

[83] Iñaki Navarro and Fernando Matía. "An Introduction to Swarm Robotics". In: *International Scholarly Research Notices* 2013.1 (2013), p. 608164.

[84] Nadia Nedjah and Luneque Silva Junior. "Review of Methodologies and Tasks in Swarm Robotics towards Standardization". In: *Swarm and Evolutionary Computation* 50 (Nov. 2019), p. 100565.

[85] Ulric Neisser. "The Roots of Self-Knowledge: Perceiving Self, It, and Thou". In: *Annals of the New York Academy of Sciences* 818.1 (1997), pp. 19–33.

[86] Oscar Nierstrasz, Marcus Denker, and Lukas Renggli. "Model-Centric, Context-Aware Software Adaptation". In: *Software Engineering for Self-Adaptive Systems*. Ed. by Betty H. C. Cheng et al. Berlin, Heidelberg: Springer, 2009, pp. 128–145.

[87] Erik G. Nilsson et al. "Using a Patterns-Based Modelling Language and a Model-Based Adaptation Architecture to Facilitate Adaptive User Interfaces". In: *Interactive Systems. Design, Specification, and Verification*. Ed. by Gavin Doherty and Ann Blandford. Berlin, Heidelberg: Springer, 2007, pp. 234–247.

[88] Farzan M. Noori et al. "On 3D Simulators for Multi-Robot Systems in ROS: MORSE or Gazebo?" In: *2017 IEEE International Symposium on Safety, Security and Rescue Robotics (SSRR)*. Oct. 2017, pp. 19–24.

[89] R. Olfati-Saber. "Flocking for Multi-Agent Dynamic Systems: Algorithms and Theory". In: *IEEE Transactions on Automatic Control* 51.3 (Mar. 2006), pp. 401–420.

[90] David St-Onge et al. *ROS and Buzz: Consensus-Based Behaviors for Heterogeneous Teams*. Oct. 2017. arXiv: 1710.08843 [cs].

[91] G. Pardo-Castellote. "OMG Data-Distribution Service: Architectural Overview". In: *23rd International Conference on Distributed Computing Systems Workshops, 2003. Proceedings.* May 2003, pp. 200–206.

[92] L.E. Parker. "ALLIANCE: An Architecture for Fault Tolerant Multirobot Cooperation". In: *IEEE Transactions on Robotics and Automation* 14.2 (Apr. 1998), pp. 220–240.

[93] Lynne E. Parker. "Distributed Intelligence: Overview of the Field and Its Application in Multi-Robot Systems". In: *Journal of Physical Agents (JoPha)* 2.1 (2008), pp. 5–14.

[94] Lynne E. Parker. "Multiple Mobile Robot Systems". In: *Springer Handbook of Robotics*. Springer, Berlin, Heidelberg, 2008, pp. 921–941.

[95] Sven Peldszus et al. "Software Reconfiguration in Robotics". In: *Empirical Software Engineering* 30.4 (Apr. 2025), p. 94.

[96] James L. Peterson. "Petri Nets". In: *ACM Comput. Surv.* 9.3 (Sept. 1977), pp. 223–252.

[97] Ana Petrovska. "Self-Awareness as a Prerequisite for Self-Adaptivity in Computing Systems". In: *2021 IEEE International Conference on Autonomic Computing and Self-Organizing Systems Companion (ACSOS-C)*. Sept. 2021, pp. 146–149.

[98] Carlo Pinciroli and Giovanni Beltrame. "Buzz: An Extensible Programming Language for Heterogeneous Swarm Robotics". In: *2016 IEEE/RSJ International Conference on Intelligent Robots and Systems (IROS)*. Oct. 2016, pp. 3794–3800.

[99] Carlo Pinciroli et al. "ARGoS: A Modular, Multi-Engine Simulator for Heterogeneous Swarm Robotics". In: *2011 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Sept. 2011, pp. 5027–5034.

[100] Carlo Pinciroli et al. "ARGoS: A Modular, Parallel, Multi-Engine Simulator for Multi-Robot Systems". In: *Swarm Intelligence* 6.4 (Dec. 2012), pp. 271–295.

[101] Lenka Pitonakova, Richard Crowder, and Seth Bullock. "Information Exchange Design Patterns for Robot Swarm Foraging and Their Application in Robot Control Algorithms". In: *Frontiers in Robotics and AI* 5 (June 2018).

[102] Lenka Pitonakova et al. "Feature and Performance Comparison of the V-REP, Gazebo and ARGoS Robot Simulators". In: *Towards Autonomous Robotic Systems*. Ed. by Manuel Giuliani, Tareq Assaf, and Maria Elena Giannaccini. Cham: Springer International Publishing, 2018, pp. 357–368.

[103] David Portugal, Luca Iocchi, and Alessandro Farinelli. "A ROS-Based Framework for Simulation and Benchmarking of Multi-robot Patrolling Algorithms". In: *Robot Operating System (ROS): The Complete Reference (Volume 3)*. Ed. by Anis Koubaa. Cham: Springer International Publishing, 2019, pp. 3–28.

[104] Morgan Quigley et al. "ROS: An Open-Source Robot Operating System". In: *ICRA Workshop on Open Source Software*. Vol. 3. Kobe, 2009, p. 5.

[105] M. O. Rabin and D. Scott. "Finite Automata and Their Decision Problems". In: *IBM Journal of Research and Development* 3.2 (Apr. 1959), pp. 114–125.

[106] Craig W. Reynolds. "Flocks, Herds and Schools: A Distributed Behavioral Model". In: *SIGGRAPH Comput. Graph.* 21.4 (Aug. 1987), pp. 25–34.

[107] Yara Rizk, Mariette Awad, and Edward W. Tunstel. "Cooperative Heterogeneous Multi-Robot Systems: A Survey". In: *ACM Comput. Surv.* 52.2 (Apr. 2019), 29:1–29:31.

[108] Alberto Rodriguez et al. “Failure Detection in Assembly: Force Signature Analysis”. In: *2010 IEEE International Conference on Automation Science and Engineering*. Aug. 2010, pp. 210–215.

[109] Eric Rohmer, Surya P. N. Singh, and Marc Freese. “V-REP: A Versatile and Scalable Robot Simulation Framework”. In: *2013 IEEE/RSJ International Conference on Intelligent Robots and Systems*. Nov. 2013, pp. 1321–1326.

[110] C. Schlegel and R. Worz. “The Software Framework SMARTSOFT for Implementing Sensorimotor Systems”. In: *Proceedings 1999 IEEE/RSJ International Conference on Intelligent Robots and Systems. Human and Environment Friendly Robots with High Intelligence and Emotional Quotients (Cat. No.99CH36289)*. Vol. 3. Oct. 1999, 1610–1616 vol.3.

[111] Christian Schlegel et al. “Composition, Separation of Roles and Model-Driven Approaches as Enabler of a Robotics Software Ecosystem”. In: *Software Engineering for Robotics*. Ed. by Ana Cavalcanti et al. Cham: Springer International Publishing, 2021, pp. 53–108.

[112] René Schöne et al. “Incremental Causal Connection for Self-Adaptive Systems Based on Relational Reference Attribute Grammars”. In: *Proceedings of the 25th International Conference on Model Driven Engineering Languages and Systems*. MODELS ’22. New York, NY, USA: Association for Computing Machinery, Oct. 2022, pp. 1–12.

[113] Debra Schreckenghost, Terrence Fong, and Tod Milam. “Human Supervision of Robotic Site Surveys”. In: *AIP Conference Proceedings* 969.1 (Jan. 2008), pp. 776–783.

[114] T. Sheridan. “Human Supervisory Control of Robot Systems”. In: *1986 IEEE International Conference on Robotics and Automation Proceedings*. Vol. 3. Apr. 1986, pp. 808–812.

[115] Herbert A. Simon. “The Architecture of Complexity”. In: *Proceedings of the American Philosophical Society* 106.6 (1962), pp. 467–482. JSTOR: 985254.

[116] Richard Soley et al. “Model Driven Architecture”. In: *OMG white paper* 308.308 (2000), p. 5.

[117] Edgar Solf. “Runtime Model Driven ROS with RAGConnect”. Minor Thesis. Dresden University of Technology, 2025.

[118] P. N. Squire and R. and Parasuraman. “Effects of Automation and Task Load on Task Switching during Human Supervision of Multiple Semi-Autonomous Robots in a Dynamic Environment”. In: *Ergonomics* 53.8 (Aug. 2010), pp. 951–961.

[119] Kasper Støy. “Using Situated Communication in Distributed Autonomous Mobile Robotics”. In: *Proceedings of the Seventh Scandinavian Conference on Artificial Intelligence*. SCAI ’01. NLD: IOS Press, Feb. 2001, pp. 44–52.

[120] Michael Szvetits and Uwe Zdun. “Enhancing Root Cause Analysis with Runtime Models and Interactive Visualizations”. In: *Proceedings of the 8th Workshop on Models @ Run.Time*. Sept. 2013.

[121] Michael Szvetits and Uwe Zdun. “Systematic Literature Review of the Objectives, Techniques, Kinds, and Architectures of Models at Runtime”. In: *Software & Systems Modeling* 15.1 (Feb. 2016), pp. 31–69.

[122] Mario Taddei. *Leonardo da Vinci's robots: new mechanics and new automata found in codices*. 2007.

[123] "The Notion of Self-aware Computing". In: *Self-Aware Computing Systems*. Cham: Springer International Publishing, 2017, pp. 3–16.

[124] Michael Vierhauser et al. "*GRuM* — A Flexible Model-Driven Runtime Monitoring Framework and Its Application to Automated Aerial and Ground Vehicles". In: *Journal of Systems and Software* 203 (Sept. 2023), p. 111733.

[125] B. Woodcroft. *The Pneumatics of Hero of Alexandria: From the Original Greek*. Inventions of the Ancients. Charles Whittingham, 1851.

[126] Peter R. Wurman, Raffaello D'Andrea, and Mick Mountz. "Coordinating Hundreds of Cooperative, Autonomous Vehicles in Warehouses". In: *AI Magazine* 29.1 (Mar. 2008), pp. 9–9.

[127] Christopher Zhong and Scott A. DeLoach. "Runtime Models for Automatic Reorganization of Multi-Robot Systems". In: *Proceedings of the 6th International Symposium on Software Engineering for Adaptive and Self-Managing Systems*. SEAMS '11. New York, NY, USA: Association for Computing Machinery, May 2011, pp. 20–29.

[128] Joshua M. Zutell, David C. Conner, and Philipp Schillinger. "ROS 2-Based Flexible Behavior Engine for Flexible Navigation". In: *SoutheastCon 2022*. Mar. 2022, pp. 674–681.

[129] Jeroen M. Zwanepol, Gustavo Rezende Silva, and Carlos Hernández Corbato. *Runtime Architecture and Task Plan Co-Adaptation for Autonomous Robots with Metaplan*. Dec. 2023. arXiv: `2312.10525 [cs]`.

# List of Abbreviations

**AG** Attribute Grammar. 28

**AOP** Aspect-Oriented Programming. 30, 31, 50

**AST** Abstract Syntax Tree. 16, 28–31, 33, 36, 47–50, 57, 66, 67, 69–73, 90, 94

**BT** Behavior Tree. 25, 26, 73, 74

**CIM** Computation Independent Model. 20

**CORBA** Common Object Request Broker Architecture. 20

**CPS** Cyber-Physical System. 24

**DDS** Data Distribution Service. 27, 80

**DSL** Domain-Specific Language. 20, 28, 30, 33, 47, 50, 51, 56–58, 72, 73, 88, 93

**FSM** Finite State Machine. 25, 26, 74

**GUI** Graphical User Interface. 6, 10, 37, 47, 48, 60, 65, 66, 75, 76, 92, 93

**HFSM** Hierarchical Finite State Machine. 17, 25, 26, 52, 57, 59, 73–76, 88, 92–94

**IDL** Interface Definition Language. 51

**JIT** Just-In-Time compilation. 50

**JSON** JavaScript Object Notation. 34, 36, 67, 72

**JVM** Java Virtual Machine. 34

**LiDAR** Light Detection And Ranging. 53, 62–64, 79, 89, 91

**MAPE-K** Monitor-Analyze-Plan-Execute over a Knowledge base. 14, 22, 24, 45, 48, 71

**MAS** Multi-Agent System. 12, 13, 50

**MDA** Model-Driven Architecture. 14, 20

**MDD** Model-Driven Development. 20, 66, 76

**MDE** Model-Driven Engineering. 13, 14, 20–22

**MOF** Meta-Object Facility. 21, 88

**MQTT** Message Queuing Telemetry Transport. 7, 32, 33, 67, 72, 80, 82–85, 90

**MRS** Multi-Robot System. 6, 9, 10, 12–20, 23, 36–40, 42, 44, 48, 50, 52, 53, 56–62, 65, 67, 69, 71, 73, 74, 76, 78, 87, 88, 92, 93

**OMG** Object Management Group. 14, 20, 21

**P2P** Peer-To-Peer networking. 52

**PIM** Platform-Independent Model. 20

**PSM** Platform-Specific Model. 20

**RAG** Reference Attributed Grammar. 28–31, 33, 36, 58, 66

**RelRAG** Relational Reference Attributed Grammar. 28, 34, 35, 66–71, 74, 92

**ROS** Robot Operating System. 6, 9, 13–15, 17, 18, 26, 28, 32–35, 51, 52, 54, 57, 58, 60–63, 65, 66, 76, 78–82, 85–89, 91–94

**ROS 2** Robot Operating System 2. 9–15, 17–20, 24, 26, 27, 31, 32, 51–55, 57, 60, 61, 64–67, 69, 71–73, 86–93

**RuMROS** Runtime Model-driven ROS. 6, 10, 13, 15–20, 26, 29, 31, 32, 34–37, 39, 49, 50, 53, 58, 60–62, 75, 93

**RuMROS*** Multi-robot extension of RuMROS implemented within the scope of this thesis. 6, 10, 18, 19, 25, 36, 39, 41, 44, 46, 48, 49, 51–61, 63–70, 72, 75–78, 82, 85–90, 92–94, 107

**SAS** Self-Aware System. 10

**SAS** Self-Adaptive System. 20, 24, 45, 60, 76

**SLAM** Simultaneous Localization and Mapping. 27

**TLA** Top-Level Architecture. 60

**UDP** User Datagram Protocol. 33, 36, 67, 81

**UI** User Interface. 6, 31, 33–37, 49, 67, 70, 73, 75, 76, 88, 91, 92

**UML** Unified Modeling Language. 29, 44

**WSGI** Web Server Gateway Interface. 36

**XML** eXtensible Markup Language. 18, 34, 66, 72

# A. Appendix

Listing A.1: A simplified version of the Python implementation of Olfati-Saber's flocking algorithm 2 [89] in RuMROS*

```
class FlockingBehavior(SwarmBehavior):
    """
    Implements simple flocking behavior without obstacle detection according to Algorithm 2
    in 'Flocking for Multi-Agent Dynamic Systems: Algorithms and Theory'
    (See https://ieeexplore.ieee.org/abstract/document/1605401). Flocking is a form of
    behavior that allows the swarm to hold a certain formation while working towards a goal.
    The organization is characterized
    by three main components:
        Cohesion   - sticking closely to nearby neighbors,
        Separation - avoiding collisions with nearby neighbors,
        Alignment  - matching the velocity of nearby neighbors.
    Which concrete behavior or formation manifests among a swarm of robots depends on the
    parameters given, see the linked paper and constructor parameters for more information.
    Requires a Ranging-And-Detection type sensor.
    """
    def __init__(self, rad_sensor: RadSensor, pose_sensor: PoseStampedSensor, target_x: float, target_y:
          float,
             neighbor_radius: float = 1.5, spacing: float = 1.0, k_align: float = 1.0,
             k_spacing: float = 2.0, k_heading: float = 0.5, cluster_epsilon: float = 0.12,
             max_speed: float = 0.5, **kwargs):
        """
        Initializes the flocking behavior.

        Args:
            rad_sensor (RadSensor): The Ranging-And-Detection type sensor of the robot.
            pose_sensor (PoseStampedSensor): The ROS2 PoseStamped message sensor of the robot.
            target_x (float, optional): The x-coordinate of the target location.
            target_y (float, optional): The y-coordinate of the target location.
            neighbor_radius (float, optional): The radius within which neighbors are sensed. Defaults to
                      1.5.
            spacing (float, optional): The spacing to hold between robots. Defaults to 1.0.
            k_align (float, optional): Alignent force gain. Defaults to 1.0.
            k_spacing (float, optional): Spacing force gain. Defaults to 2.0.
            k_heading (float, optional): Heading force gain. Defaults to 0.5.
            cluster_epsilon (float, optional): Distance below which multiple sensor detections will be
                      considered one neighbor. Defaults to 0.12.
            max_speed (float, optional): Maximum speed of the robot. Defaults to 0.5.
        """
        super().__init__('flocking', [rad_sensor, pose_sensor])
```

```
    if rad_sensor.min_range > spacing or rad_sensor.max_range < spacing or rad_sensor.min_range >
            neighbor_radius or rad_sensor.max_range < neighbor_radius:
        raise ValueError("The neighbor sensing radius or spacing parameters fall outside the RAD
            sensor's range.")

    self.target = np.array([target_x, target_y])
    self.last_velocity = np.zeros(2)
    self.last_cluster_positions = None
    self.last_cluster_time = time.time()
    self.neighbor_radius = neighbor_radius
    self.spacing = spacing
    self.k_align = k_align
    self.k_spacing = k_spacing
    self.k_heading = k_heading
    self.cluster_epsilon = cluster_epsilon
    self.max_speed = max_speed

def potential(self, r: float) -> float:
    """
    Definition of the potential function: Linear spring-like potential:
    repulsive < spacing, attractive > spacing.
    """
    return r - self.spacing

def cluster_neighbors(self, hits: List[np.ndarray]) -> List[np.ndarray]:
    """
    Clusters nearby hits using DBSCAN and returns the mean position of each cluster.
    """
    if not hits:
        return []

    # Convert np.ndarray list to 2D np array
    points = np.vstack(hits) # shape: (n_samples, 2)

    # Run DBSCAN
    clustering = DBSCAN(eps=self.cluster_epsilon, min_samples=1).fit(points)

    labels = clustering.labels_
    clustered = []

    for label in np.unique(labels):
        cluster_points = points[labels == label]
        avg = np.mean(cluster_points, axis=0)
        clustered.append(avg)

    return clustered

def update_clusters(self, clusters: List[np.ndarray], _time: float, current_velocity: np.ndarray):
    self.last_cluster_positions = clusters
    self.last_cluster_time = _time
    self.last_velocity = current_velocity

def _tick(self) -> Twist:
    twist = Twist()

    # Assume sensor index exists because super() was initialized with a RadSensor
    lidar_data: TRadData = self.sensor_data[self.rad_sensor_index]
    pose: PoseStamped = self.sensor_data[self.pose_stamped_sensor_index]
    if lidar_data:
        # Convert polar coordinates of hits to cartesian
        hits = []
        for angle, dist in lidar_data:
            if 0.0 < dist < self.neighbor_radius:
                # Polar to cartesian
                x = dist * np.cos(angle)
                y = dist * np.sin(angle)
```

```
        hits.append(np.array([x_rot, y_rot]))

# Apply clustering to nearby points (single detection per robot)
clusters = self.cluster_neighbors(hits)

# Store cluster positions on first iteration for velocity computation
current_time = time.time()
if not self.last_cluster_positions:
    self.update_clusters(clusters, current_time, np.zeros(2))
    return twist

# Compute time delta
dt = current_time - self.last_cluster_time
if dt == 0: dt = 1e-3  # Prevent divide by zero, though it shouldn't happen

# Compute spacing force
spacing_force = np.zeros(2)
for neighbor in clusters:
    dist = np.linalg.norm(neighbor) # Distance to neighbor
    if dist == 0:
        continue
    direction = neighbor / dist # Normalized direction
    phi = self.potential(dist)
    spacing_force += phi * direction

# Compute alignment force
alignment_force = np.zeros(2)
for curr, prev in zip(clusters, self.last_cluster_positions):
    delta = curr - prev
    vel_neighbor = delta / dt
    alignment_force += vel_neighbor - self.last_velocity

# Normalize
if len(clusters) > 0:
    alignment_force /= len(clusters)

# Compute heading force
current_pos = np.array([pose.pose.position.x, pose.pose.position.y])
target_vector_world = self.target - current_pos
target_dist = np.linalg.norm(target_vector_world)

# Convert target direction into robot local frame
q = pose.pose.orientation
theta = 2 * np.arctan2(q.z, q.w)

# Rotate world vector into local frame
rot_matrix = np.array([
    [np.cos(-theta), -np.sin(-theta)],
    [np.sin(-theta),  np.cos(-theta)]
])
target_vector_local = rot_matrix @ target_vector_world

heading_force = np.zeros(2)
if target_dist > 0.1:
    heading_force = target_vector_local / target_dist

total_force = (
    self.k_align * alignment_force +
    self.k_spacing * spacing_force +
    self.k_heading * heading_force
)

# Clip to max velocity
total_force = np.clip(total_force, -self.max_speed, self.max_speed)
```

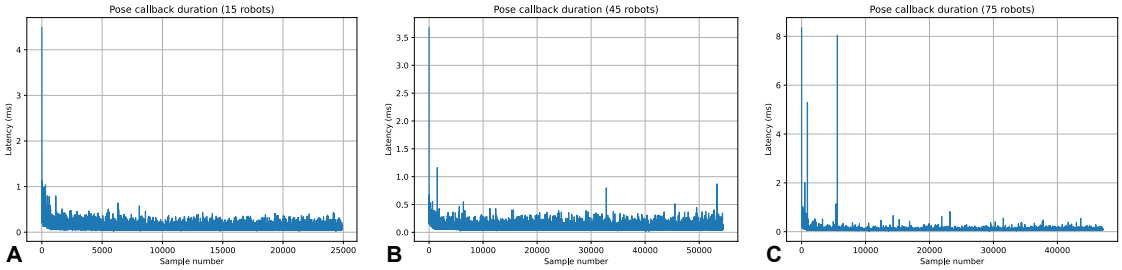


Figure A.1.: Durations of pose callbacks for $N_r = 15$ (a), $N_r = 45$ (b) and $N_r = 75$ (c) robots. All tests conducted on the mobile machine.

```
        # Update state
        # Dampen alignment for smoother movement
        self.update_clusters(clusters, current_time, 0.8 * self.last_velocity + 0.2 * total_force)

        # Convert force vector to Twist
        twist = force_to_twist(total_force,
                               epsilon=0.2,
                               max_linear=self.max_speed)
    return twist
```

**Statement of authorship**

I hereby certify that I have authored this document entitled *Multi-robot extension and evaluation of a runtime model-driven ROS 2 framework* independently and without undue assistance from third parties. No other than the resources and references indicated in this document have been used. I have marked both literal and accordingly adopted quotations as such. There were no additional persons involved in the intellectual preparation of the present document. I am aware that violations of this declaration may lead to subsequent withdrawal of the academic degree.

Dresden, 17th February 2026

Alexander Kassuba